



컴퓨터구조 및 운영체제(CAOS)



1. 컴퓨터 구조

KPC 기본반

1. 컴퓨터 구조.....	3
(1) 폰 노이만(Von Neumann) 구조.....	4
(2) 컴퓨터 구성요소.....	6
(3) 컴퓨터 명령어(Computer Instruction)	9
(4) MMU(Memory Management Unit).....	14
(5) TLB(Translation Look-Aside Buffer).....	16
(6) Network Storage(DAS/NAS/SAN/IP-SAN)	18
(7) I/O 제어방식	24
(8) GPGPU(General Purpose Graphics Processing Unit).....	27

폰노이만 구조

1. 범용 컴퓨터 구조의 확립, 폰 노이만 구조의 개요

□ 폰 노이만(Von Neumann) 구조의 정의

- 모든 명령어는 반드시 실행 이전에 메모리에 불러 들어야 하며, CPU는 한번에 단 하나의 명령어만 실행 가능한 컴퓨터 구조



Von Neumann
(1903~1957)



Alan Turing
(1912~1954)

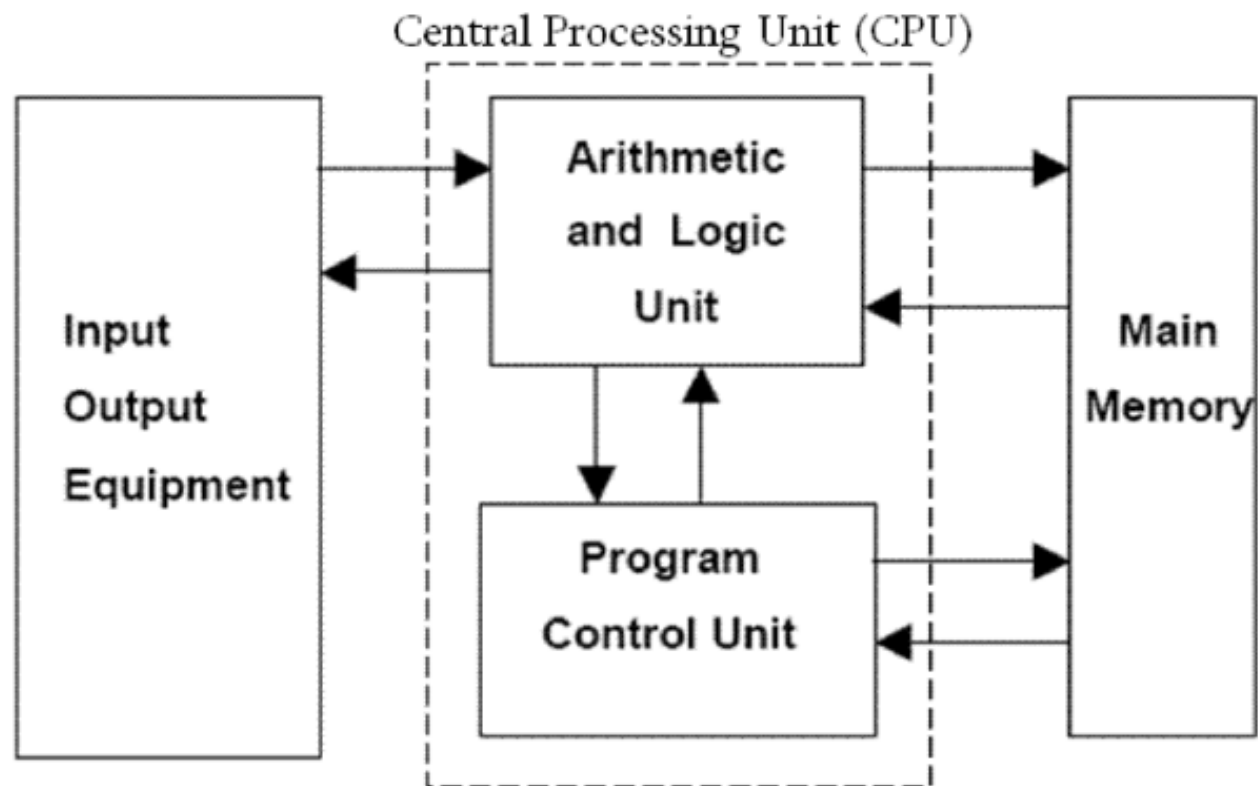
현대 컴퓨터 구조의 근간이 되는
Von Neumann Architecture

□ 폰 노이만 구조의 특징

- 프로그램 내장형(Stored Program) 컴퓨터: 미리 프로그램을 컴퓨터에 내장하여 수행하는 컴퓨터
- CPU는 한번에 단 하나의 명령어만 실행 가능
- SISD(Single Instruction Single Data stream)으로 분류

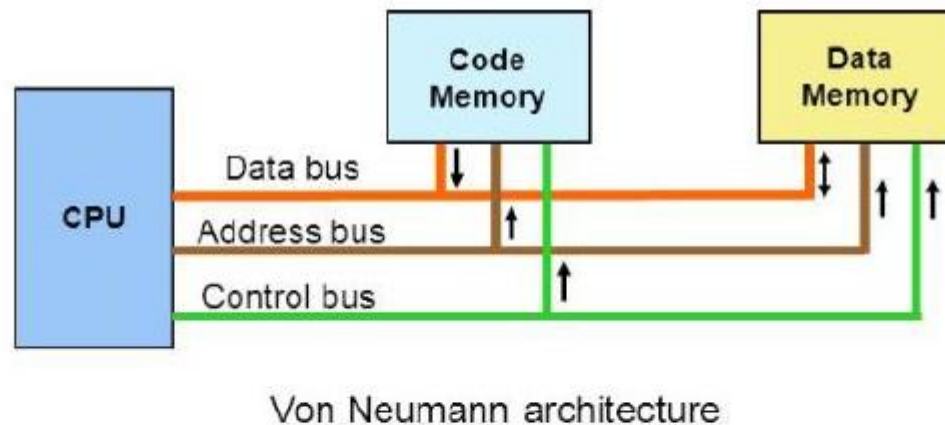
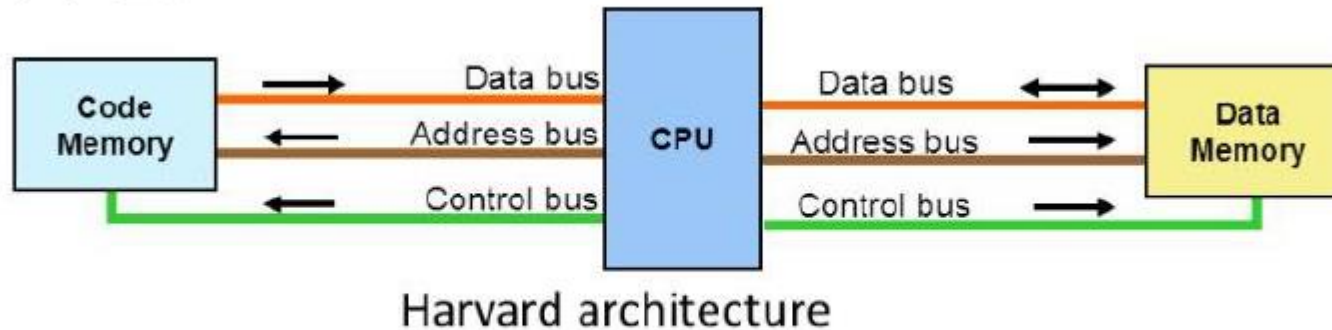
폰노이만 구조

2. Von Neumann 구조(CPU 와 주변장치)



폰노이만 구조

3. 컴퓨터 아키텍처 발전



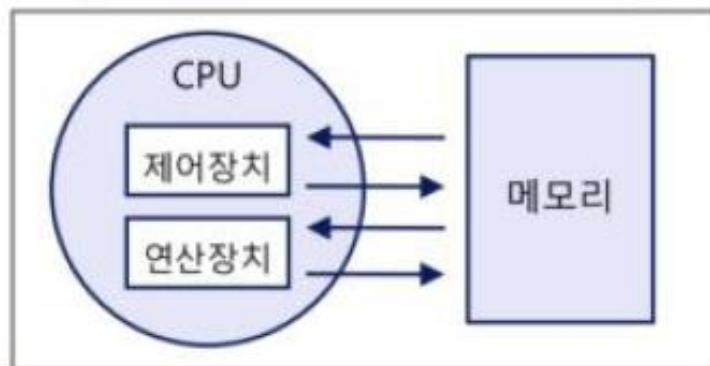
- 하나의 메모리에 명령어와 데이터를 모두 저장하는 방식으로 메모리 버스의 병목현상 발생
- 메모리 버스의 병목으로 인해 성능향상을 위한 파이프라이닝(Pipelining) 구현의 어려움 존재
- 이를 극복하기 위하여 명령 메모리와 데이터 메모리를 별도로 가지는 Harvard 구조로 발전

폰노이만 구조

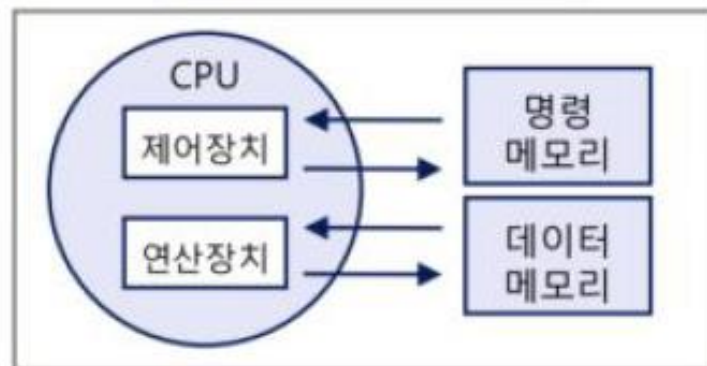
[참고] 프로그램 내장(Stored-Program) 방식

CPU 옆에 기억장치(Memory)를 붙이고, 프로그램과 데이터를 기억장치에 저장해 놓았다가 실행명령에 따라 차례로 불러내어 처리하는 방식

Von Neumann Architecture



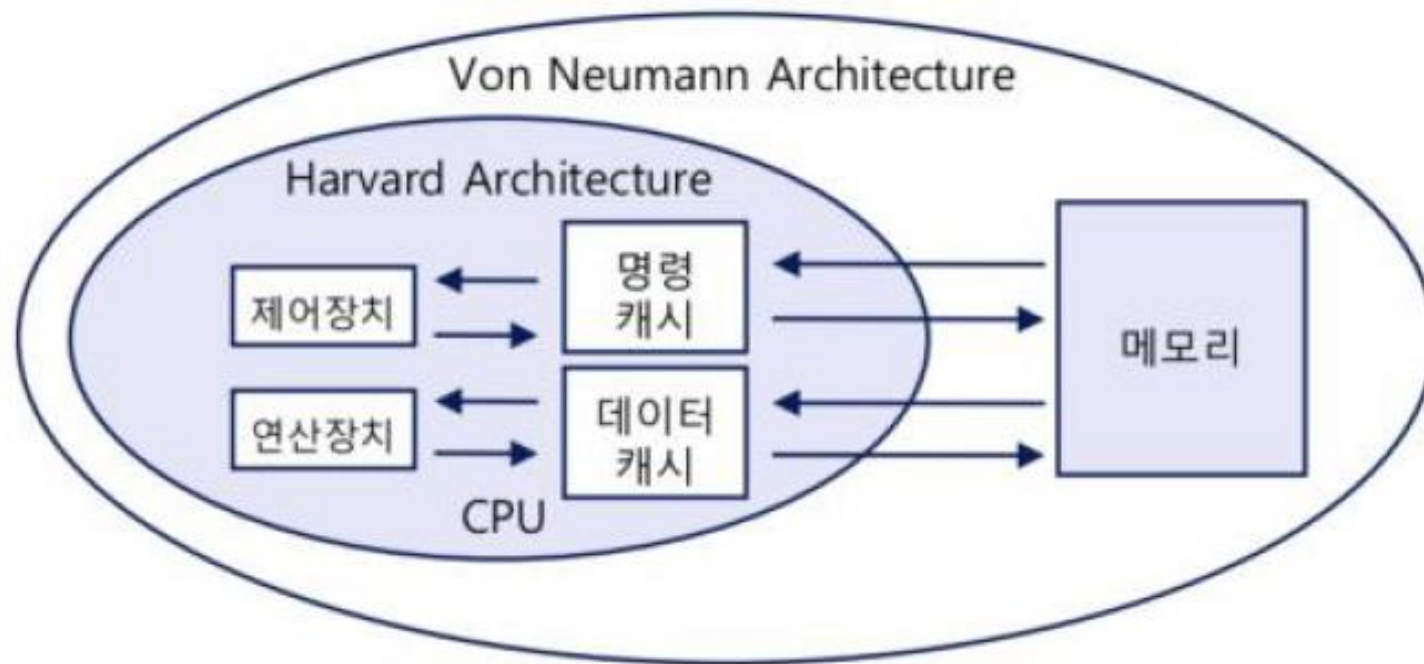
Harvard Architecture



폰노이만 구조

[참고] Von Neumann Architecture + Harvard Architecture

- 최근 고성능 CPU 설계시 하바드와 폰노이만 구조 모두 도입
- 캐시 메모리는 보통 명령용과 데이터용으로 분리됨
- 하바드 구조는 CPU 내부(CPU와 캐시의 관계)에 적용, 폰노이만 구조는 CPU 외부에 적용



MMU(Memory Management Unit)

1. 메모리 관리장치, MMU의 개요

□ MMU(Memory Management Unit)의 정의¹

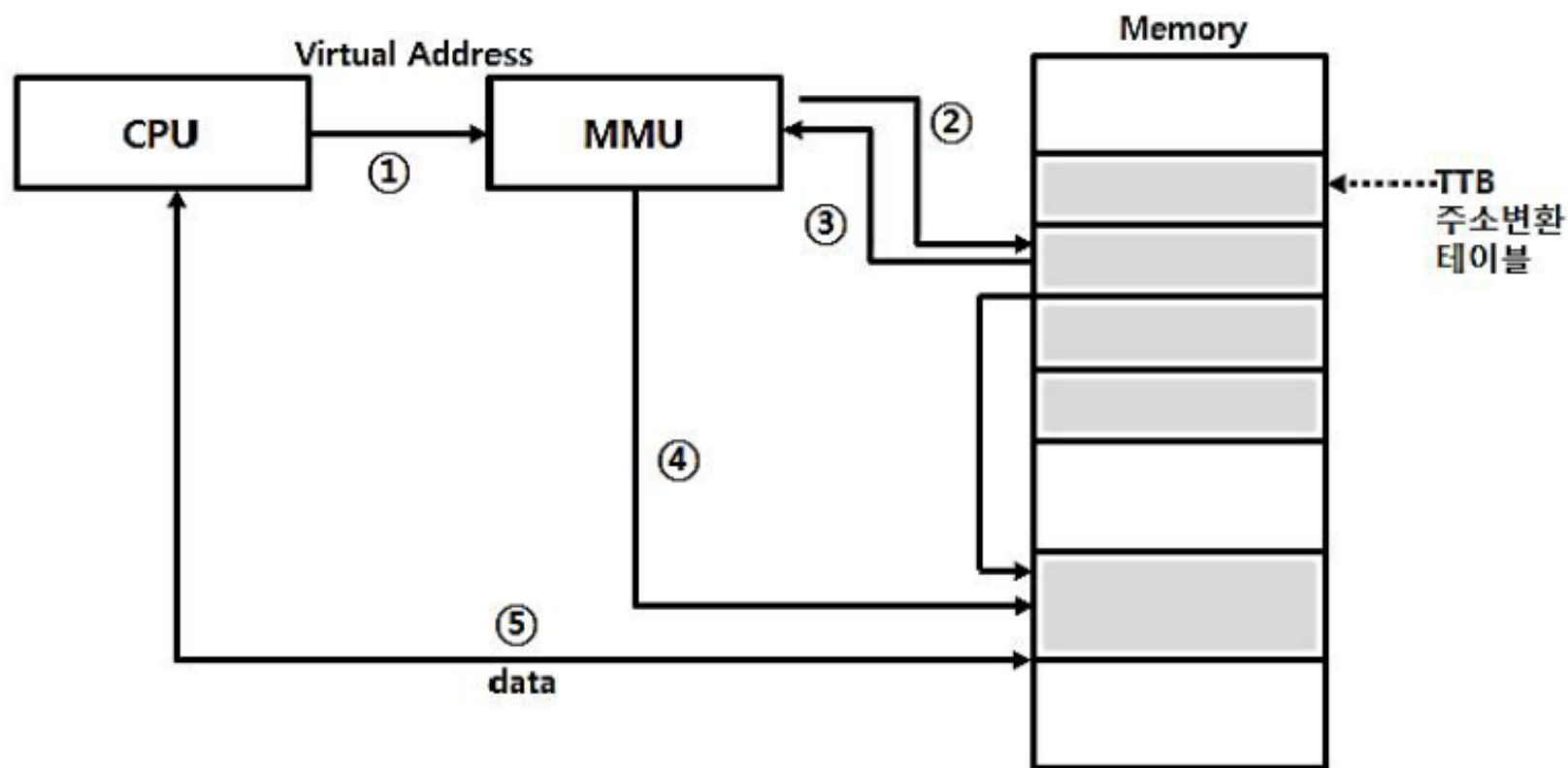
- CPU가 메모리에 접근하는 것을 관리하는 컴퓨터 하드웨어 부품
- 가상 메모리 주소를 실제 메모리 주소로 변환하며, 메모리 보호, 캐시 관리, 버스 중재 등의 역할을 담당하며 간단한 8비트 아키텍처에서는 뱅크 스위칭을 담당

□ MMU의 기능

기능	내용
주소변환	- 논리적 주소와 물리적 주소의 연결은 Memory Mapping 통해 처리
메모리 보호	- 메모리에 대한 접근행동 결정
캐시 관리	- 메모리 영역의 캐시정책 결정, TBL(Translation Look-aside Buffer)
버스 중재	- 시스템버스에 접속된 버스 마스터들이 버스를 사용할 수 있도록 조정 - 버스 요구, 버스 승인, 버스 사용 중 신호

MMU(Memory Management Unit)

2. MMU의 주소변환 과정



MMU(Memory Management Unit)

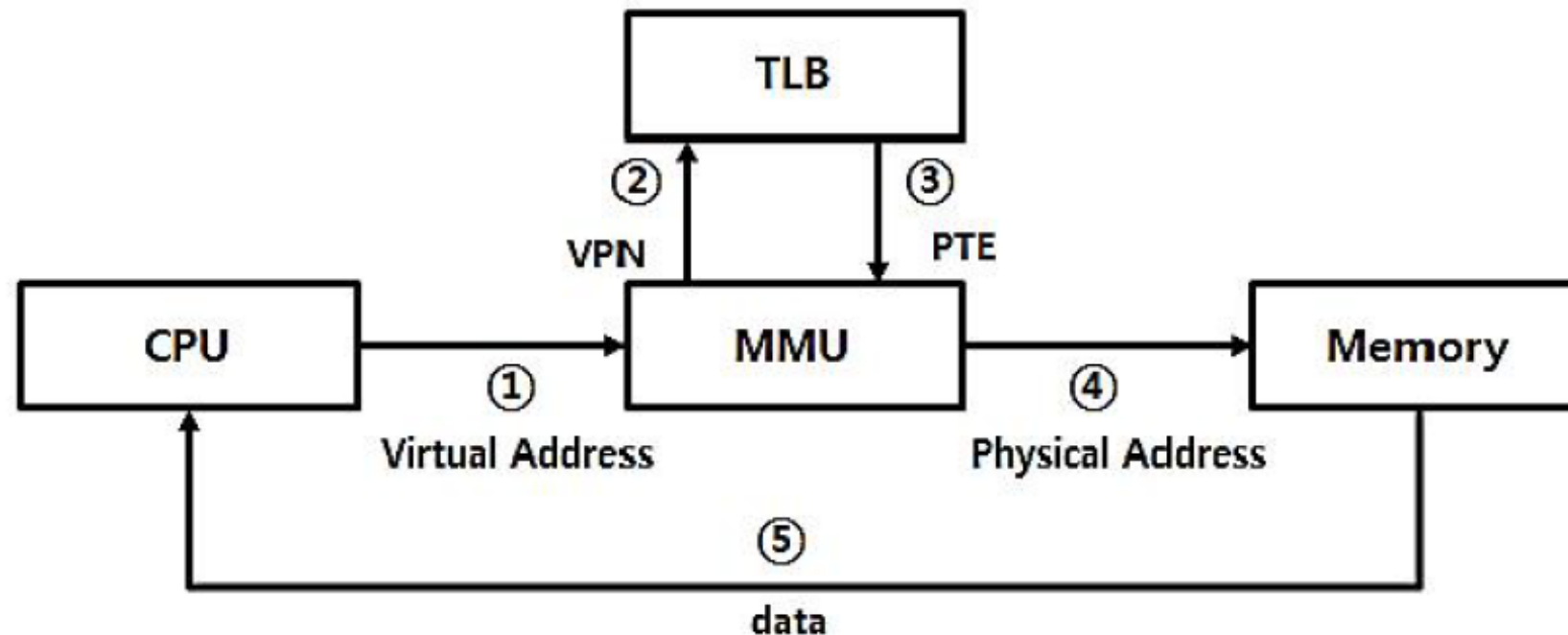
과정	내용
① 가상주소 전달	- CPU에서 메모리 접근을 위하여 MMU에 가상주소를 전달
② 페이지 테이블 검색	- MMU는 가상주소를 전달받아 메모리의 TTB(Translation Table Base Address) ² 부터 시작하여 페이지 테이블 검색
③ 페이지 테이블 엔트리 전달	- 검색된 페이지 테이블 안에서 물리주소를 찾아내어 MMU에 전달
④ 물리주소 접근	- MMU는 주소신호를 발생(RAS, CAS)
⑤ 데이터 전달	- 메모리는 해당 주소 안에 있는 데이터를 출력하여 CPU에 전달하고 가상주소를 통해 데이터 수신

¹ https://ko.wikipedia.org/wiki/메모리_관리_장치

² TTB(Translation Table Base Address) : 페이지 테이블의 시작위치를 지시하는 MMU 내의 레지스터

MMU(Memory Management Unit)

3. TLB를 활용한 주소변환 과정



① 가상주소 전달 → ② 가상주소 번호 전달 → ③ 페이지 테이블 엔트리 반환 → ④ 물리주소 접근 → ⑤ 데이터 전달

- 자주 참조되는 Page Table 주소를 캐시에 저장하여(지역성) MMU의 주소변환속도 개선
- MMU에서 Page Table를 탐색하기 전에 TLB를 먼저 탐색

TLB(Translation Look-Aside Buffer)

1. 메모리 접근성능 개선을 위한 고속캐시, TLB의 개요

□ TLB(Translation Look-Aside Buffer)의 개념

- [개념] 가상메모리 주소를 물리주소(실제 메모리주소)로 변환하는 속도를 높이기 위해 사용되는 고속 캐시 메모리

□ TLB(Translation Look-Aside Buffer)의 필요성

- 가상메모리 참조시 두 번의 물리적 액세스가 필요. 즉, 1)페이지 테이블 반입, 2)데이터 반입의 액세스가 필요. 이러한 메모리 액세스 시간을 단축시키기 위해 페이지 테이블의 일부 항목을 Locality 기반으로 적재

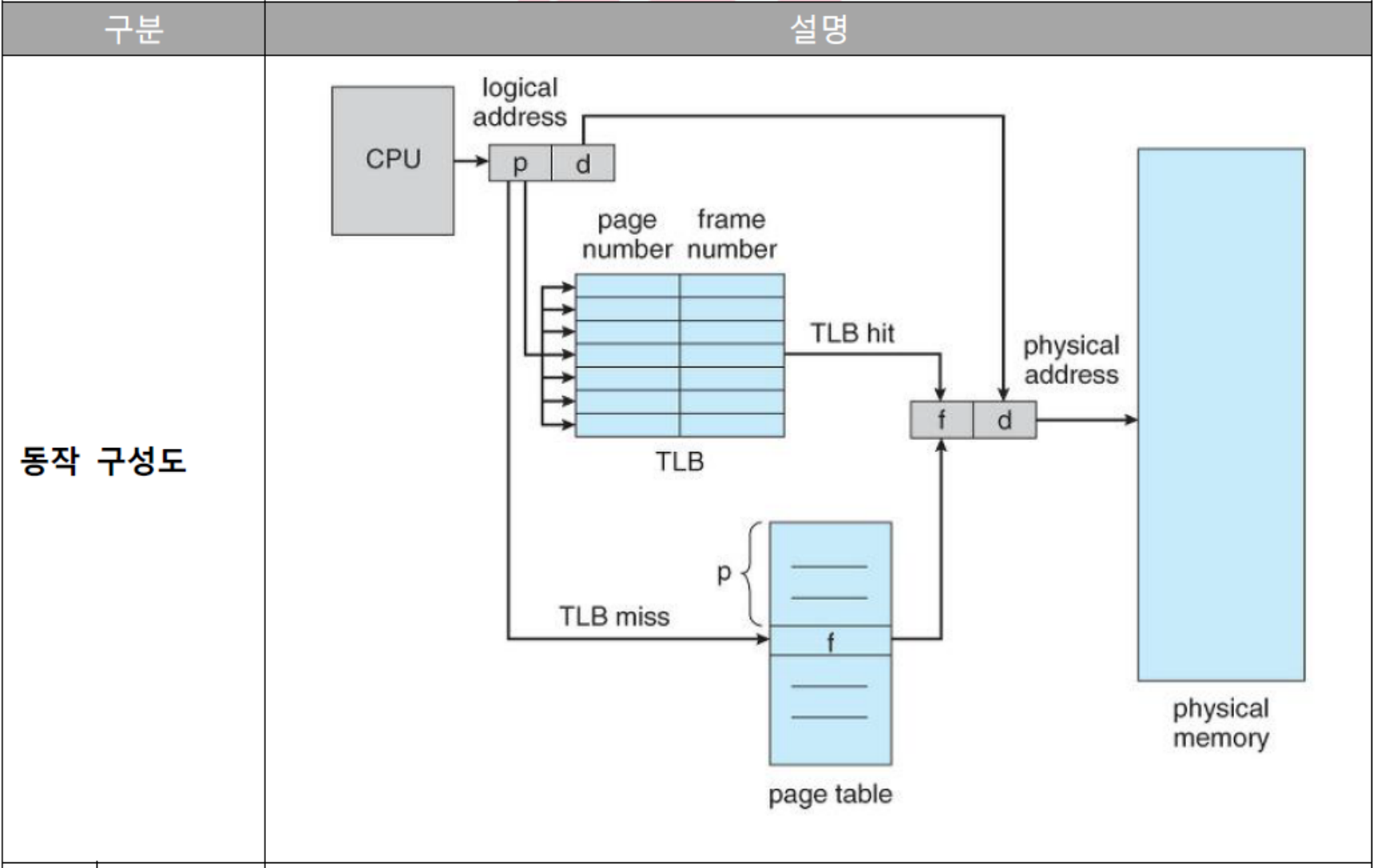
2. TLB(Translation Look-Aside Buffer)의 동작원리

□ TLB(Translation Look-Aside Buffer)의 기본원리

- 시간 지역성(temporal Locality): 최근 참조한 페이지는 가까운 미래에 다시 참조할 가능성이 높음
- 연관 사상(Associative Mapping): 가상주소에 페이지 테이블 기반으로 주소 변환

TLB(Translation Look-Aside Buffer)

□ TLB(Translation Look-Aside Buffer)의 동작구성



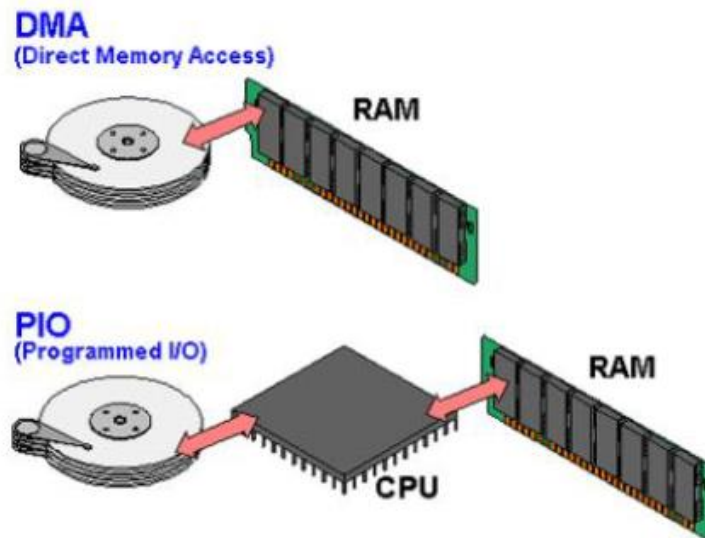
TLB(Translation Look-Aside Buffer)

동작 절차	1) TLB 우선검색	- CPU로부터 전달 받은 가상 주소로 TLB를 먼저 참조하여 이미 변환된 물리 주소가 있는지를 검색함. 이 때 사상 방식은 연관 사상 방식이 사용됨
	2) TLB Hit	- 이미 변환된 물리 주소가 TLB에 존재할 경우, 바로 프레임 주소와 변위를 통해 메인 메모리에 접근
	3) TLB Miss	- 이미 변환된 주소가 없을 경우 페이지 테이블에 접근하여 물리 주소를 얻어온 후 TLB를 갱신하고 메모리에 접근
	4)메모리 접근	- 변환된 물리 주소를 통하여 Data를 얻어 CPU에 전달

- TLB에 이미 변환된 물리주소(PFN, 페이지 프레임 넘버)가 있을 경우 즉시 해당 프레임에 접근 가능

I/O 제어방식

□ I/O 제어방식의 유형



구분	방식	내용
중앙처리장치 관여	Programmed I/O 방식	- 중앙처리장치 구동 프로그램에 의한 입출력 제어
중앙처리장치 독립적	인터럽트 I/O 방식	- 주변장치의 인터럽트 신호를 이용한 입출력 제어
	DMA 방식	- 주변장치와 메모리 간 직접 접근
	채널제어 방식	- 주변장치가 연결된 채널 제어기를 이용해 입출력 제어

- I/O 제어방식 중 DMA 방식과 채널 제어방식이 대표적임

I/O 제어방식

2. DMA 방식

□ DMA 방식의 개념

- DMA 제어기가 버스 제어권을 획득하여 중앙처리장치의 개입없이 주변장치와 메모리 간 직접 데이터를 전송하는 입출력 제어방식

구분	항목	내용
동작모드	Burst Transfer 모드	- 입출력을 모두 완료시까지 버스를 점유하는 동작 모드
	Cycle Stealing 모드	- 중앙처리장치와 번갈아 가며 입출력 처리 동작 모드
동작과정	입출력 정보 및 명령 전달	- 중앙처리장치가 DMA 제어기에 DMA 입출력 정보 및 명령 전달
	MCR(Memory Cycle Request) 신호 전송	- DMA 제어기가 중앙처리장치에 버스 제어권을 요청
	MCG(Memory Cycle Grant) 신호 전송	- 중앙처리장치가 DMA 제어기에 버스 제어권을 허용
	입출력 수행	- DMA 제어기는 주기억장치와 데이터 입출력을 수행
	입출력 반복 수행	- 데이터가 남아있으면 완료시까지 입출력 반복 수행
	인터럽트 신호 전송	- 입출력이 완료되면 DMA 제어기는 중앙처리장치에 인터럽트 전송

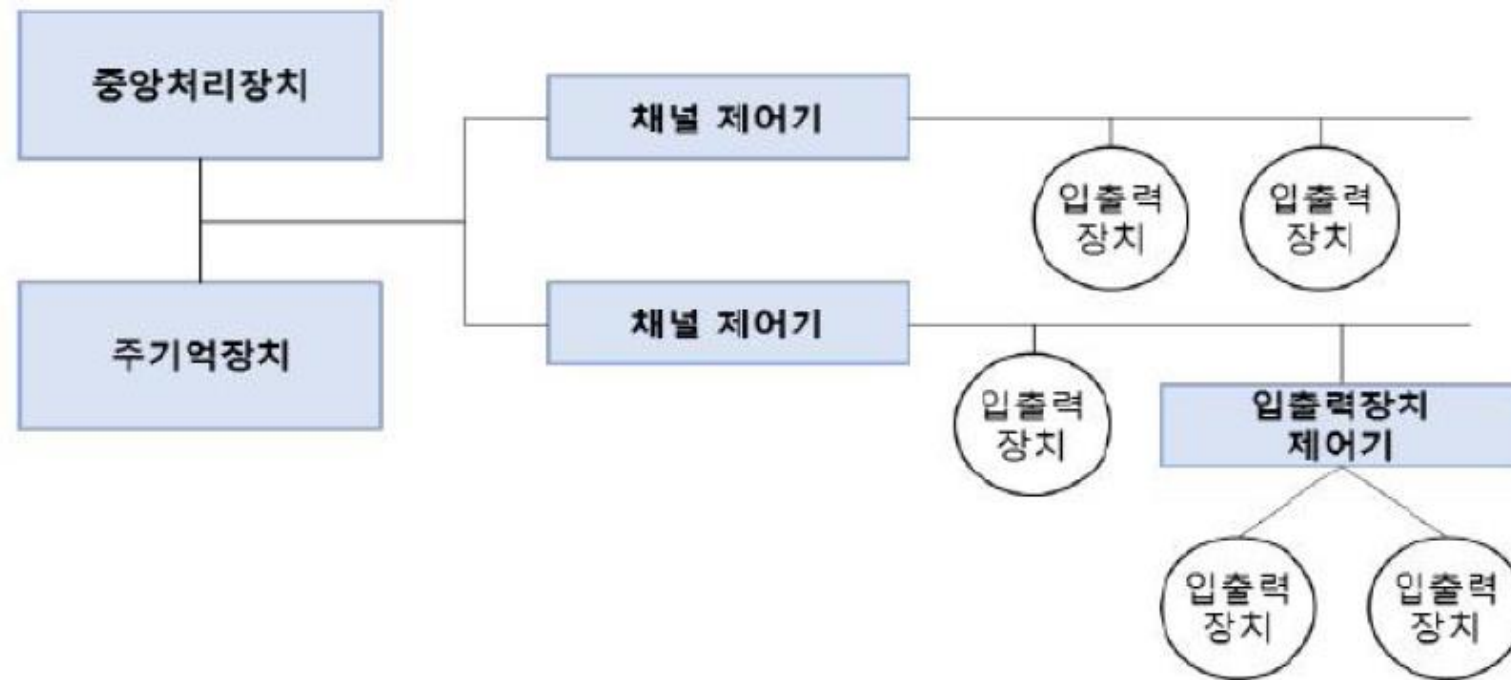
I/O 제어방식

3. 채널제어 방식

□ 채널제어 방식의 개념

- 채널은 신호를 보낼 수 있는 전송로, 여러 대의 주변장치가 연결되어 있는 채널제어기에 입출력 명령을 전달하여 중앙처리장치의 개입없이 입출력을 수행하는 방식

□ 채널제어 방식



I/O 제어방식

구분	제어과정	내용
연결방식	고정연결 채널제어기	- 여러 개의 입출력장치가 하나의 채널제어기에 고정으로 연결
	가변연결 채널제어기	- 모든 입출력장치는 여러 개의 채널제어기에 접속, 동시작업 가능
동작과정	입출력 정보 및 명령 전달	- 중앙처리장치가 채널제어기에 입출력정보 및 명령 전달
	채널제어기의 제어	- 채널제어기는 입출력장치를 선택하고 제어신호를 전달
	입출력 수행	- 채널제어기는 주기억장치와 입출력장치간 입출력을 수행
	인터럽트 신호 전송	- 입출력이 완료되면 채널제어기는 중앙처리장치에 인터럽트 전송

GPGPU

1. GPU의 수행능력 확대, GPGPU(General Purpose Graphics Processing Unit)의 개요

□ GPGPU(General Purpose Graphics Processing Units)의 정의

- GPU를 그래픽 연산 뿐 아니라, 일반 컴퓨팅 영역에도 활용하는 것을 목적으로, 중앙처리장치가 맡았던 응용 프로그램들의 계산에 GPU를 사용하는 기술

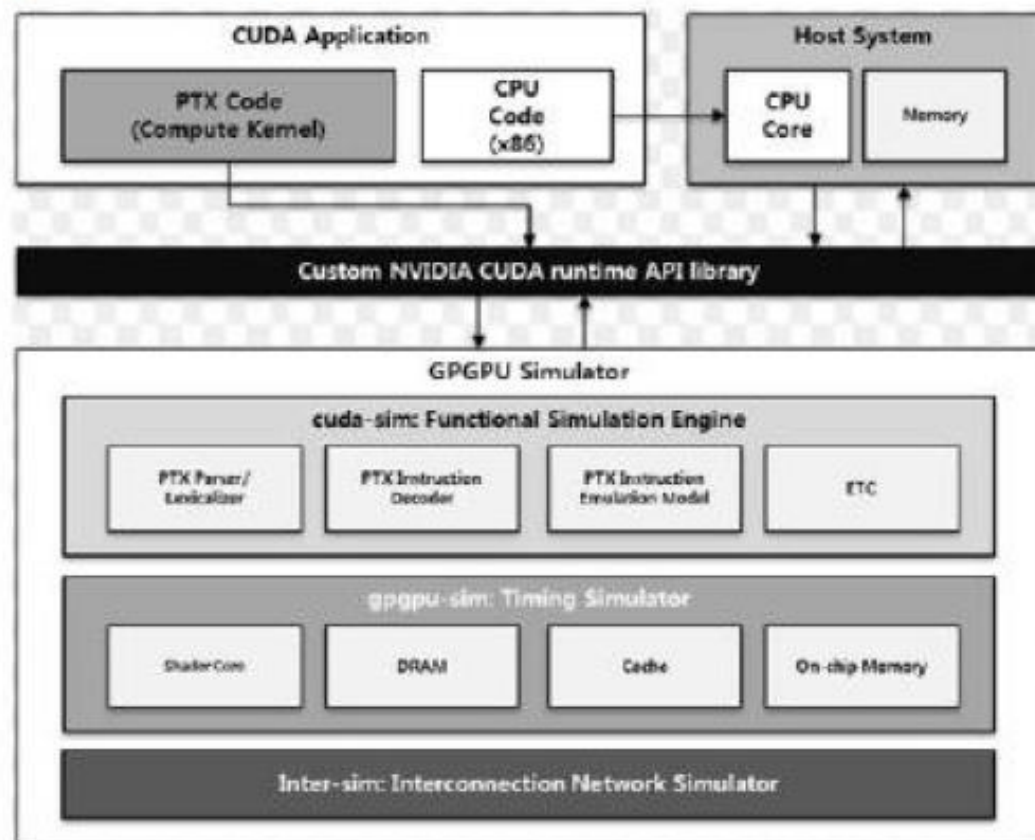
□ GPGPU의 등장 배경 및 주요 특징

구분	내용	설명
등장 배경	기계학습 성장	- 기계학습기반 딥러닝 폭발적 성장 기인
	빅데이터 처리	- 단순 비정형 등 고성능 아니어도 GPU로 처리 충분
주요 특징	초병렬 SIMD/SIMT 구조	- 제어, 캐시 부분 간소화한 코어집적/병렬처리
	플랫폼 지원	- CUDA, OpenCL 등 GPGPU 플랫폼 기반 구현 가능

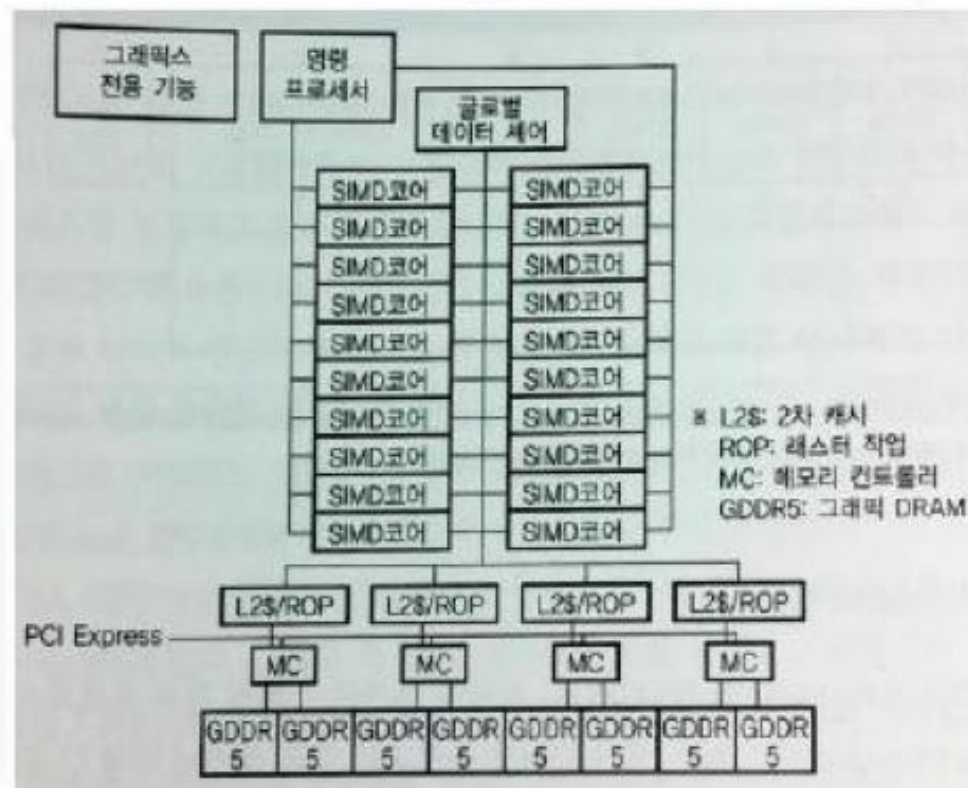
GPGPU

2. GPGPU 구성도 및 구성요소

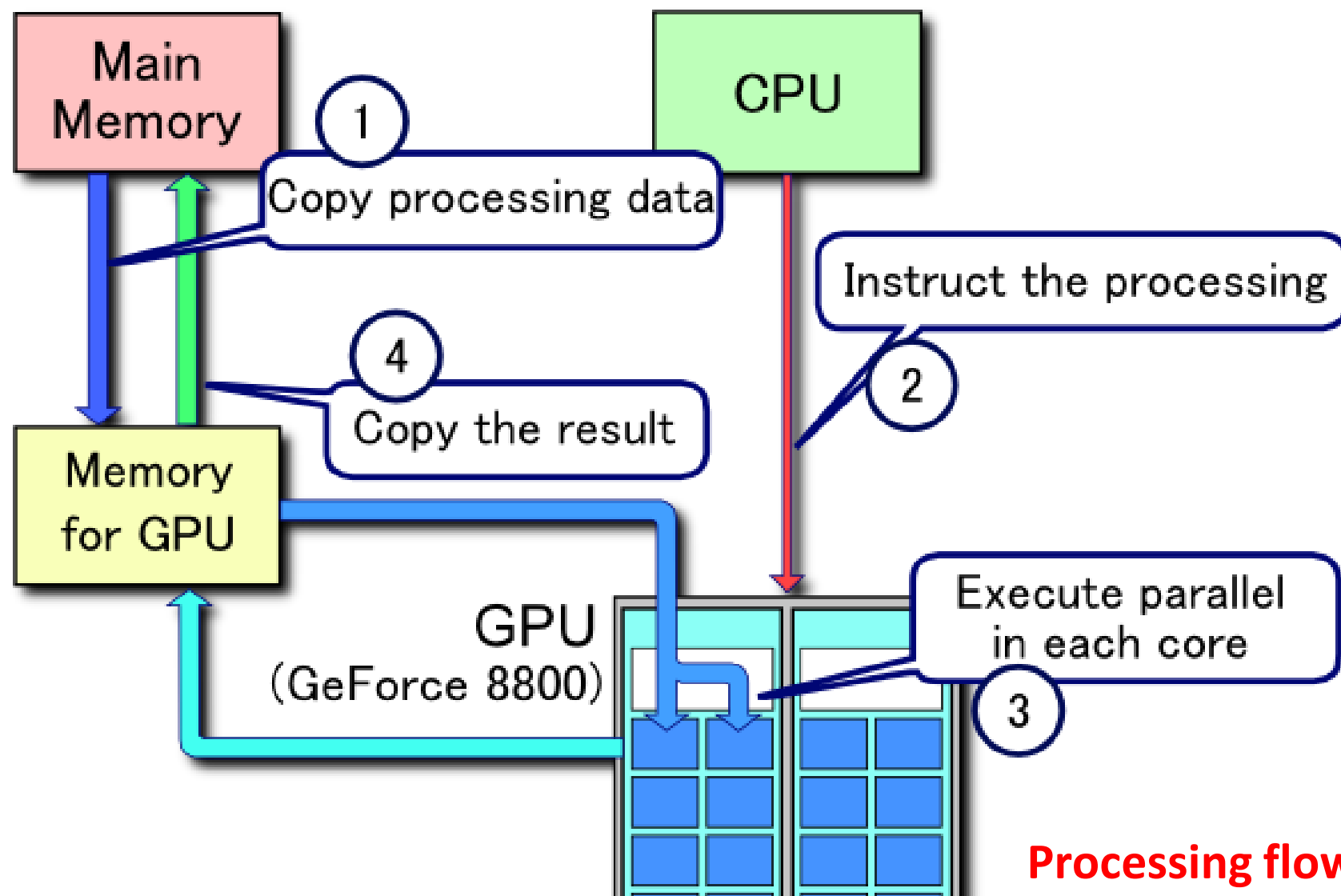
□ GPGPU 구성도



GPGPU 구성도 (AMD HD5870)



GPGPU



Processing flow on CUDA

GPGPU

3. GPGPU 플랫폼

	Controllable Hardware	Platform Dependency	Features
CUDA	GPU	NVIDIA Geforce8 ↑	Runtime API /Device API
OpenCL	Heterogeneous us CPU/GPU	Platform-independent	C/Runtime/API
OpenACC			Compiler hints, "directives"
C++ AMP		Windows 7 ↑ Visual studio 2012 ↑ DirectX 11.0 ↑	Lambda /parallel loop statement

CUDA(Compute Unified Device Architecture)

감사합니다!



컴퓨터구조 및 운영체제(CAOS)



2. 기억장치 part1

KPC 기본반

Contents

2. 기억장치.....	29
(1) 메모리 계층구조.....	30
(2) 메모리 관리기법.....	31
(3) 메모리 단편화(Fragmentation).....	34
(4) 캐시 메모리(Cache Memory).....	38
(5) 캐시 일관성(Cache Coherence).....	44
(6) 주기억장치(Main Memory)	46
(7) 메모리 인터리빙(Memory Interleaving).....	50
(8) 보조기억장치(auxiliary memory).....	53
(9) 가상 메모리(Virtual Memory).....	55
(10) 벨레디 이상현상(Belady's Anomaly).....	61

메모리 관리기법

1. 메모리 관리기법의 개요

□ 메모리 관리기법의 정의

- 다중 프로그래밍 시스템에서 운영체제에 의해 동적으로 메모리의 사용자 영역을 여러 프로세스가 상주할 수 있도록 세분화하는 과정
- 메인 메모리는 운영체제를 위한 영역과 실행 중인 프로그램을 위한 영역으로 구분됨.

메모리 관리기법

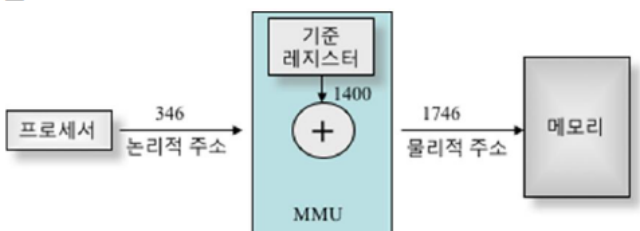
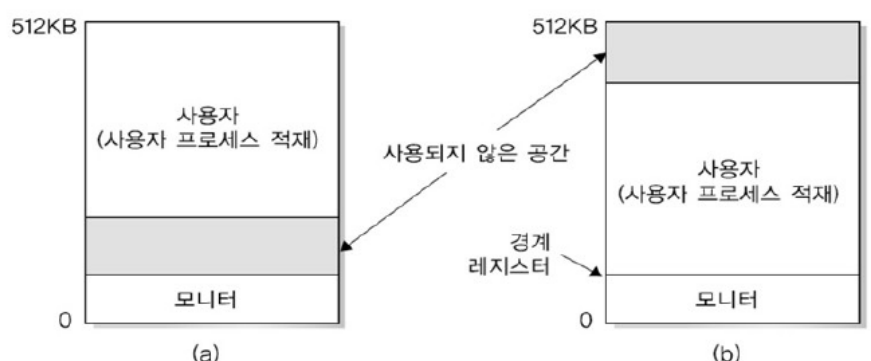
2. 메모리 관리기법의 종류 및 할당정책

□ 메모리 관리기법의 종류

기법	설명	기법
반입 정책	- 언제(When) 메모리로 적재할 것인지 결정하는 전략 - 메인 메모리에 적재할 다음 프로세스의 반입 시기를 결정	요구반입기법 예상반입기법
배치 정책	- 어디(Where)로 위치시킬 것인지 결정하는 전략 - 디스크에서 반입한 프로세스를 메인 메모리의 어느 위치에 저장할 것인지를 결정하는 방법	최초 적합 최적 적합 최악 적합
교체 정책	- 메모리의 어느 영역(What)을 교체하여 사용할 것인지 결정하는 전략 - 재배치 기법으로 메인 메모리에 있는 어떤 프로세스를 제거할 것인가를 결정	프로세스 Swap In/Out

메모리 관리기법

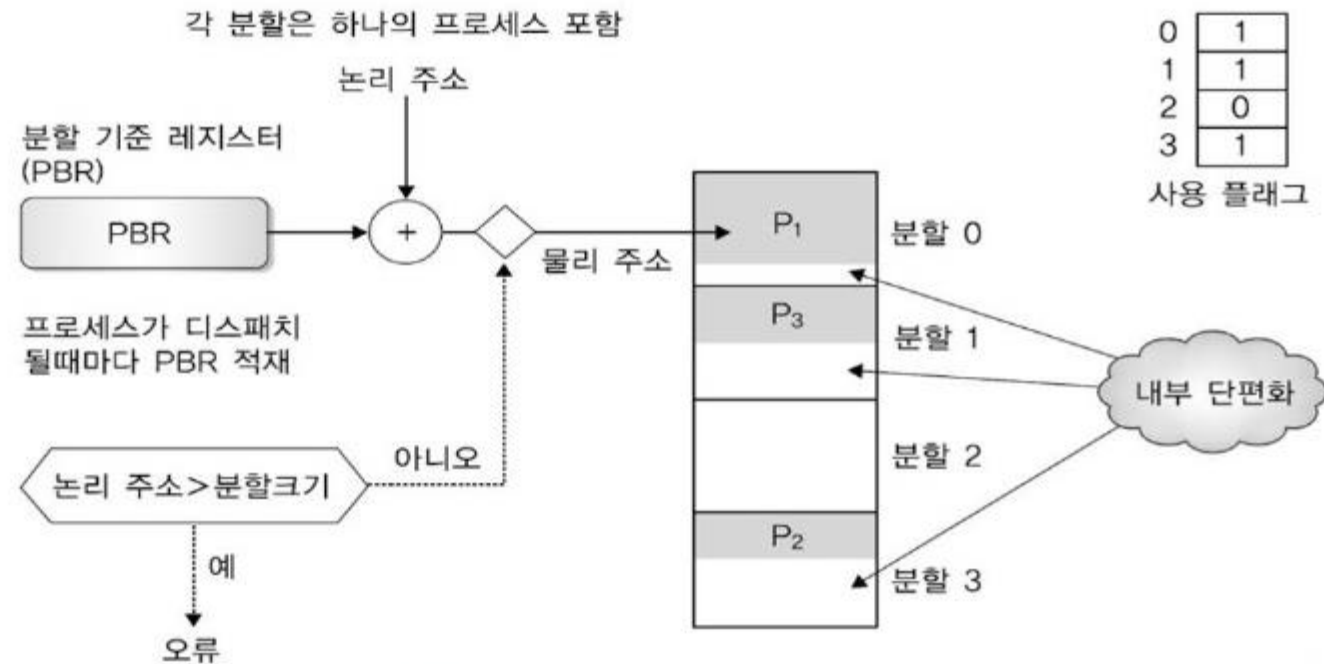
□ 연속 메모리 할당 방식

정책	설명
단일 사용자 연속 메모리 할당 (한번에 한 사용자만 시스템 사용)	- 메모리의 영역을 사용자를 위한 공간과 운영체제 상주를 위한 공간으로 구분하는 기법  - 메모리를 운영체제 루틴이 들어있는 부분(모니터)과 사용자 프로그램이 들어있는 부분, 사용되지 않는 부분으로 구분 (그림 (a)) - 사용자가 모든 메인 메모리에 대한 제어권을 가짐. (그림 (b)) • 사용자 프로그램이 주소를 잘못 지정하면 운영체제 파괴 • 운영체제 파괴 방지를 위한 프로세서 내 경계 레지스터 보유 • 사용자 프로그램이 메모리 참조할 때마다 경계 레지스터 검사 이후 실행
	 (a) (b)

메모리 관리기법

- Multiprogramming with a Fixed number of Tasks
- 메모리를 여러 개의 고정된 크기로 분할
- 고정된 크기의 분할 영역에 프로세스가 각각 할당됨.
- 물리 주소는 분할 기준 레지스터(PBR) 값에 논리 주소를 더하여 생성됨.
- 분할 영역보다 프로세스의 크기가 대부분 작으므로 내부 단편화 발생

고정분할 다중프로그래밍 (MTF)



메모리 관리기법

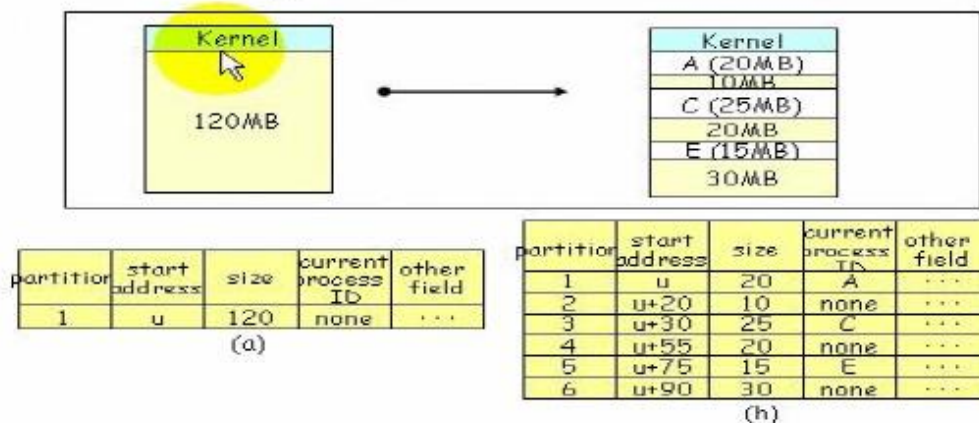
가변분할 다중프로그래밍 (MVT)

- Multiprogramming with a Variable number of Tasks
- 메인 메모리의 사용자 공간 전체를 하나의 분할 영역으로 설정하고, 이후 프로세스의 움직임에 따라 분할 형태를 동적으로 변화시키는 기법
- 내부 단편화 문제를 해결할 수 있으나, 메모리 관리를 위해 커널 자료구조의 크기가 동적으로 변화되기 때문에 오버헤드 발생

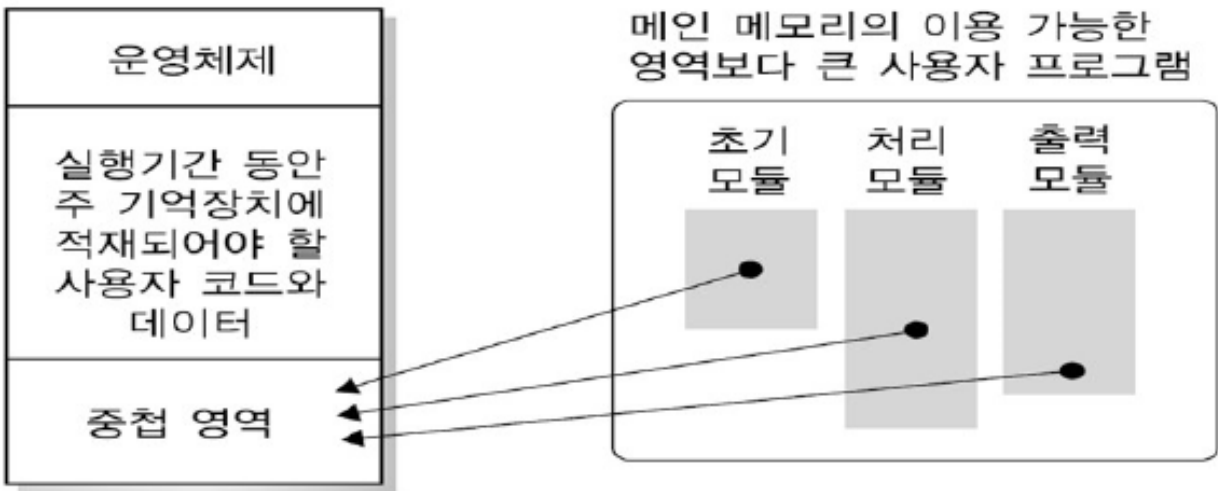
가변분할다중프로그래밍

• 시스템의 가정

- 사용자 공간 크기 : 120 MB
- (a) : 초기 상태
- (b) : 프로세스 A(20MB)가 적재된 후
- (c) : 프로세스 B(10MB)가 적재된 후
- (d) : 프로세스 C(25MB)가 적재된 후
- (e) : 프로세스 D(20MB)가 적재된 후
- (f) : 프로세스 B가 주기억장치를 반납한 후
- (g) : 프로세스 E(15MB)가 적재된 후
- (h) : 프로세스 D가 주기억장치를 반납한 후



메모리 관리기법

중첩 (Overlay)	- 프로세스의 크기가 실제 메모리 크기로 제한되는 문제점을 해결 - 운영체제 영역과 메모리 공간의 일부 영역에 프로그램(작업) 실행에 필요한 명령어와 데이터를 저장 - 중첩 구동기 영역에는 실행 기간 동안 필요한 각종 사용자 코드 등을 적재하여 필요한 시기에 해당 프로그램을 불러들여 실행함.
	

메모리 관리기법

□ 분산 메모리 할당 방식

정책	설명
페이징 기법 ¹ (paging)	- 동일 크기의 프레임으로 분할 - 페이지(page)라고 불리는 프로세스 영역들이 프레임(페이지 프레임)이라고 불리는 고정 크기 블록의 메모리 영역에 할당 - 외부 단편화 없음, 내부 단편화 발생 - 페이지가 작으면 내부 단편화 감소하나 페이지 테이블 유지 부담 (페이지 테이블 : 논리 메모리 페이지 번호와 물리 메모리 프레임 번호 매핑)
세그먼트 기법 ² (segment)	- 사용자 관점에서의 메모리 할당 (서브루틴, 모듈 등의 크기별 할당) - 컴파일러가 세그먼트를 작성, 동적(가변) 분할 기법 - 프로그램은 코드, 데이터, 스택(Stack), 세그먼트로 구성 - 세그먼트의 하드웨어 구성 : 세그먼트 테이블은 메인 메모리에 존재 - 외부 단편화 문제 발생

메모리 단편화

1. 메모리 단편화의 개요

□ 메모리 단편화(fragmentation)의 정의

- 주기억장치의 연속된 영역을 할당할 경우에 사용되지 않고 낭비되는 현상
- 기억장치(RAM, HDD 등)의 빈 공간 또는 자료가 여러 개의 조각으로 나뉘는 현상

□ 메모리 단편화의 문제점

- 기억장치의 사용 가능한 공간을 줄이거나, 읽기와 쓰기의 수행 속도를 늦추는 문제점.

메모리 단편화

2. 메모리 단편화 종류

□ 내부 단편화(Internal Fragmentation)

- m 크기의 기억 장소를 필요로 하는 어떤 작업이 $n \geq m$ 인 n 크기의 분할 영역에 적재 될 때, 그 남는 공간($n-m$)이 발생.
- 할당을 요청한 크기에 비해 실제 할당된 메모리의 크기가 커서, 해당 메모리 공간 내에서 사용하지 않는 부분이 있을 때 발생

□ 외부 단편화(External Fragmentation)

- 어떤 분할 영역이 사용되지 않고 남아 있어서 이용할 수는 있으나, 대기 중인 작업의 크기에 비해 기억 장소가 너무 작아서 그 영역 자체가 사용되지 못하는 경우를 말함

메모리 단편화

3. 메모리 단편화의 발생

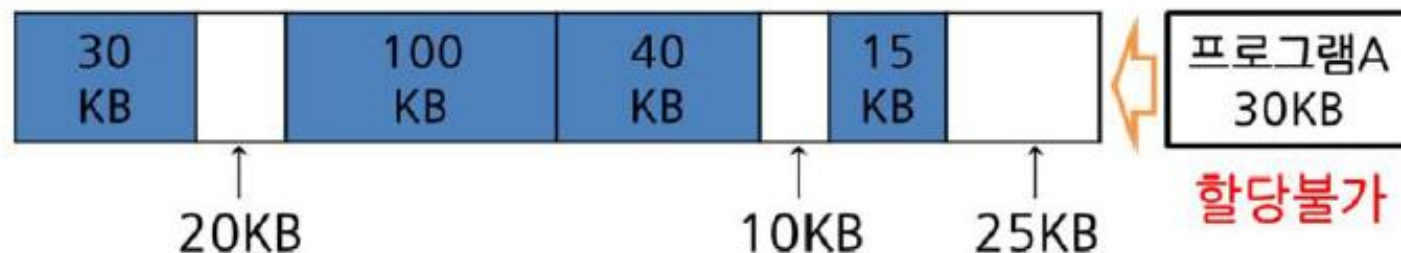
구분	내용
고정 분할	- 사용자 프로그램의 크기가 정해진 분할 크기에 정확히 맞지 않으면 내부 단편화가 발생하고, 분할이 너무 작아서 대기 중인 작업 중 하나도 맞는 것이 없을 경우 그 분할이 사용되지 않은 상태로 남아 있게 되므로 외부 단편화가 발생
가변 분할	- 가변 분할은 분할 영역의 크기가 동적으로 변경될 수 있으므로 내부 단편화는 발생하지 않지만, 외부 단편화는 발생 가능

메모리 단편화

4. 메모리 단편화 원인

□ 외부 단편화 발생 원인 : 가변 분할, 세그먼트 기법

Main Memory (가변분할, 세그먼트기법)



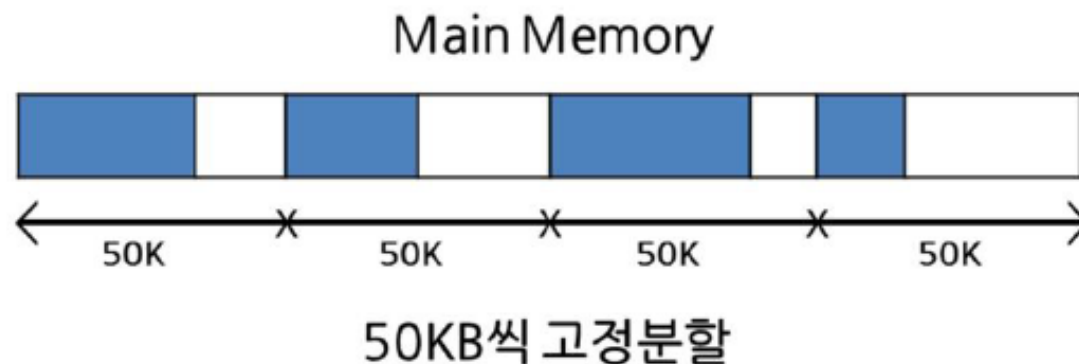
■ 할당완료

□ 빈공간이나 적재가 불가능한 공간 → 외부단편화 영역

- 발생원인 : 메모리 할당기법으로써 가변 분할 또는 세그먼트 기법 사용
- 프로그램이 요구하는 만큼 메모리를 할당하고, 이후 메모리에 프로그램이 지속적으로 할당/교체되어 사용하기 어려운 작은 공간이 발생함

메모리 단편화

□ 내부 단편화 발생 원인 : 고정 분할, 페이징 기법



■ 할당완료

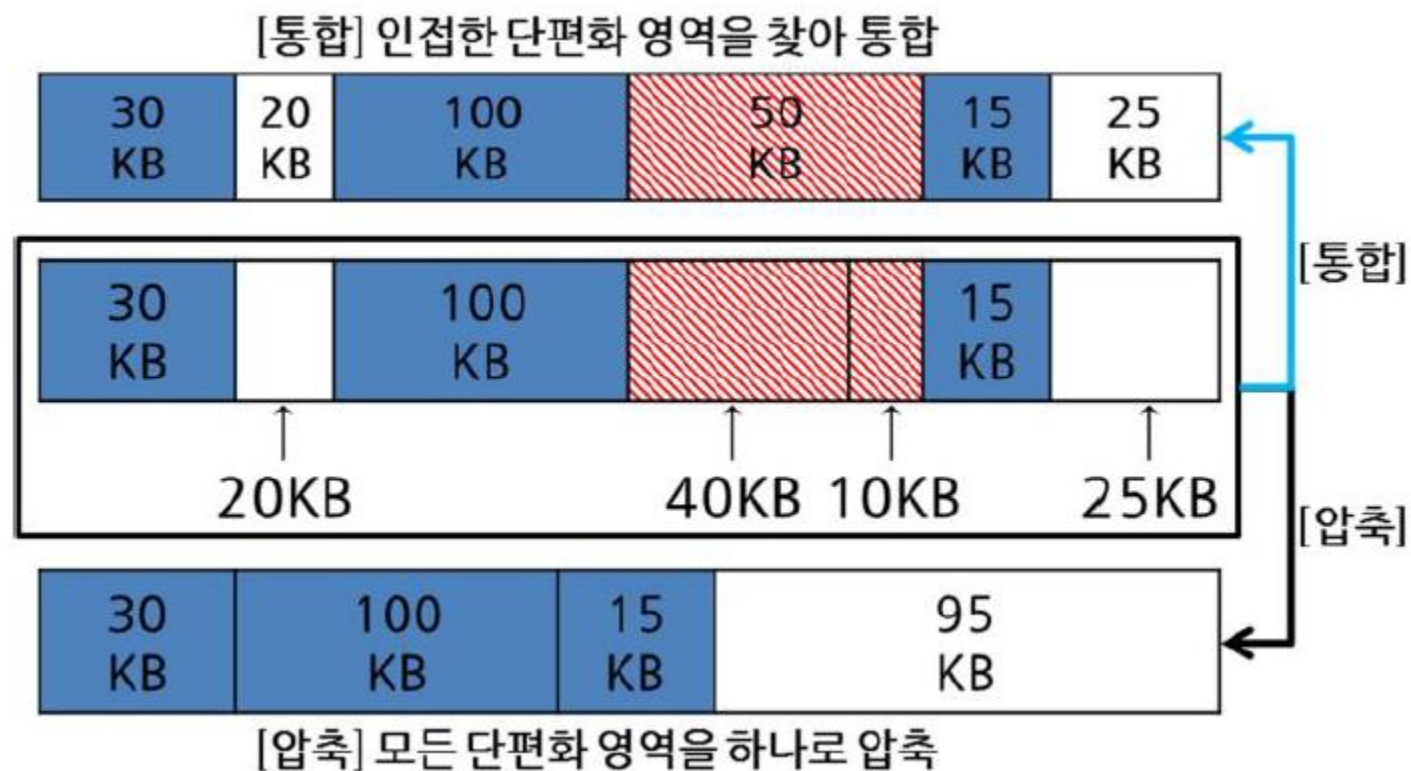
□ 고정된 분할영역 내 미사용공간 → 내부단편화 영역

- 발생원인 : 메모리 할당기법으로써 고정 분할 또는 페이징 기법 사용
- 사전에 메모리 할당 크기를 정의하고, 고정 분할 영역보다 작은 프로그램이 적재되는 경우 사용하지 않는 영역이 발생함

메모리 단편화

5. 메모리 단편화의 해결방법

□ 공통 해결방법

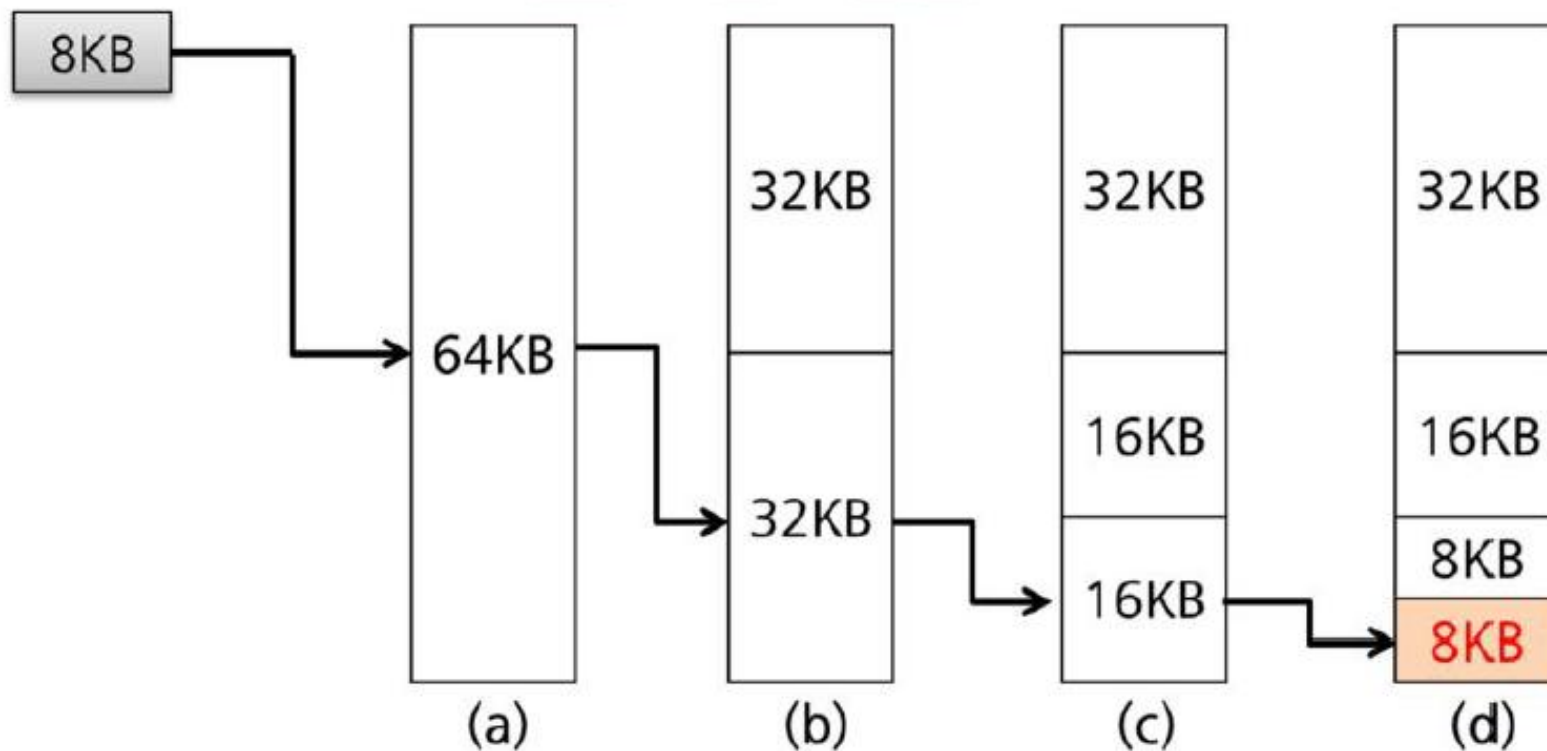


메모리 단편화

방안	내용
통합 (Coalescing)	- 가변 분할에서 하나의 작업이 끝났을 때 그 사용 영역이 다른 비어있는 분할과 인접되어 있는지를 점검하여, 만약 인접되어 있다면 이 <u>두 개의 빈 분할 영역을 하나로 통합</u>하여 효율성을 높이는 작업
압축 (Compaction)	- <u>빈 분할 영역을 주기억장치의 한 곳에 합치는 작업</u>을 말하며, 그 결과는 하나의 커다란 빈 분할 영역 생성 - 일종의 "쓰레기 수집(Garbage Collection)" 작업

메모리 단편화

□ 외부 단편화 해결 방법 - Buddy Memory Allocation

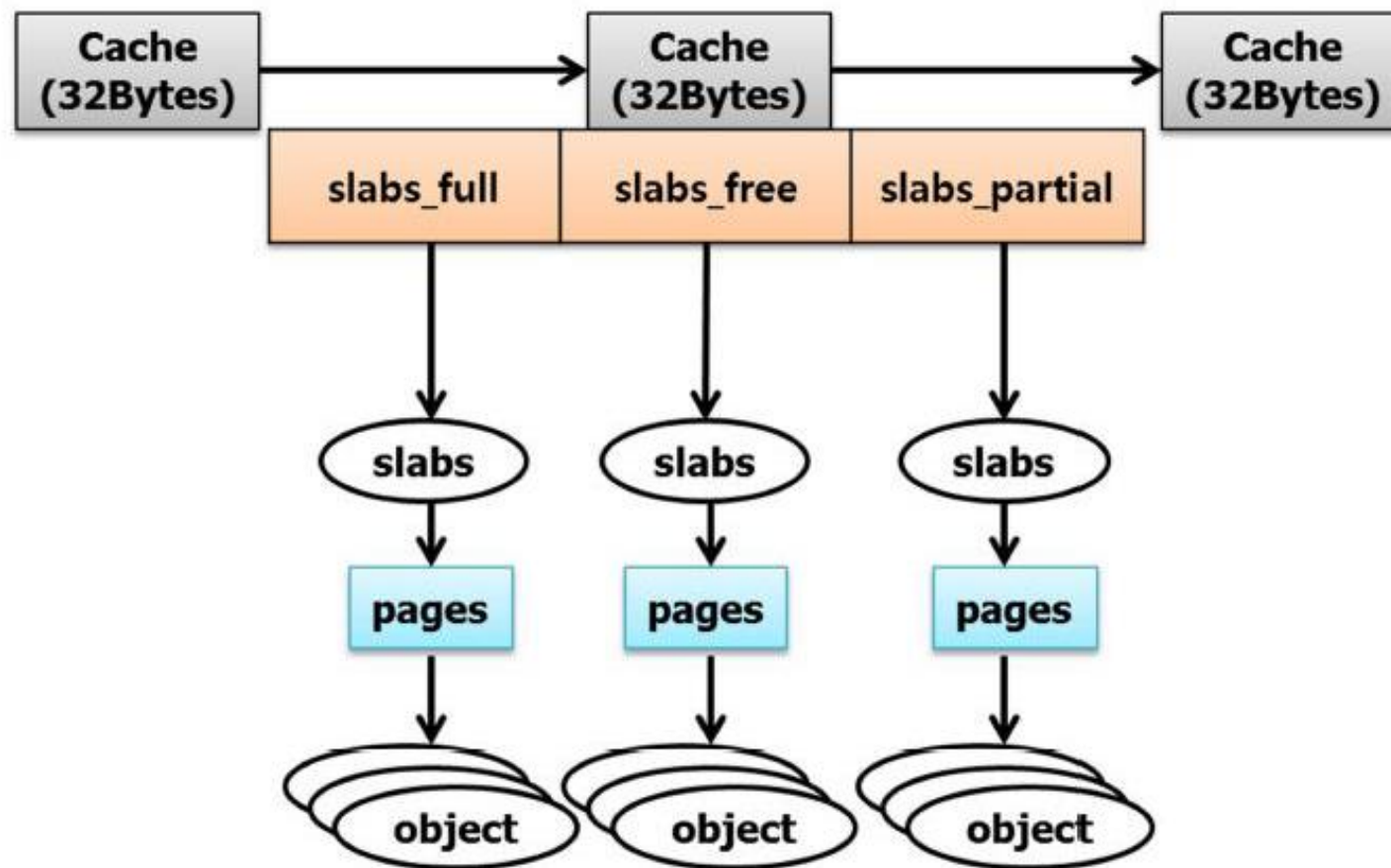


메모리 단편화

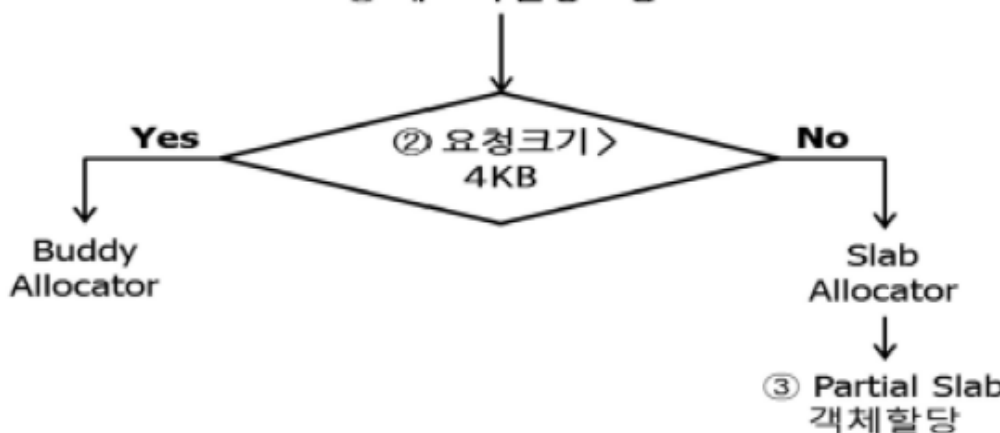
구분	내용	
개념	- 요청한 프로그램 크기에 가장 알맞은 크기를 할당하기 위해 메모리를 2n의 크기로 분할하여 메모리를 할당하는 기법	
동작원리	(a)	- 초기 64KB 버디 블록을 사용하는 시스템에 8KB 메모리 할당 요청 들어옴
	(b)	- 버디 블록을 절반으로 분할
	(c)	- 요청한 8KB보다 크기가 크기 때문에 다시 절반으로 분할
	(d)	- 블록 크기가 8KB가 되어 요청한 8KB를 할당
특징	- 메모리 해제 시, 인접한 메모리가 비어 있으면 free 버디를 합치는 과정을 반복	
한계	- 외부 단편화는 해소되었으니, 내부 단편화 문제는 여전히 남아 있음	

메모리 단편화

□ 내부 단편화 해결 방법 - Slab Allocator



메모리 단편화

구분	내용	
개념	- 페이지 프레임을 할당 받아 공간을 작은 크기로 분할하고(캐시 집합) 메모리 요청시 작은 크기로 메모리를 할당/해제하는 동적 메모리 관리 기법	
동작원리	Free Slab	- 모든 객체가 사용 가능한 상태
	Full Slab	- 모든 객체가 이미 사용중인 상태
	Partial Slab	- 일부는 사용 중, 일부는 미사용 중인 상태
동작원리	① 메모리 할당요청  <pre> graph TD A[① 메모리 할당요청] --> B{② 요청크기 > 4KB} B -- Yes --> C[Buddy Allocator] B -- No --> D[Slab Allocator] D --> E[③ Partial Slab 객체할당] </pre> - 캐싱 : 할당 받은 프레임을 미리 작은 크기로 분할하여 자주 할당되고 해제되는 크기의 캐시 구성(캐시는 slab으로, slab은 object로 구성)	

메모리 단편화

동작 메커니즘	①	- 메모리 할당 요청
	②	- 요청 크기가 최소 관리 단위(4KB)보다 크면 Buddy Allocator 최소 관리단위보다 작으면 Slab Allocator 할당 - (최소관리단위의 페이지 프레임보다 작은 요청은 내부 단편화를 유발시키므로 작은 크기의 요청은 Slab Allocator를 사용하여 작은 블록의 메모리 할당
	③	- 가장 적합한 크기의 캐시를 찾아 Partial slab으로부터 객체 할당
특징	- 내부 단편화를 최소화 할뿐 아니라 캐시를 사용하여 성능향상의 효과도 있음	

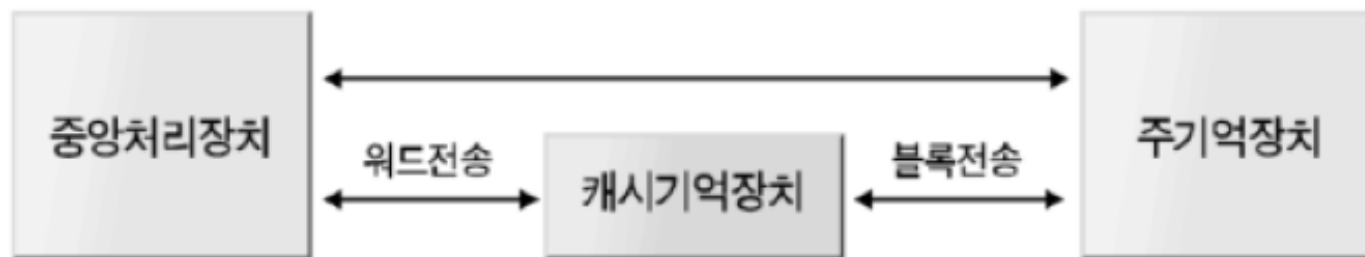
캐시 메모리(Cache Memory)

1. 캐시 메모리(Cache Memory)의 개요

□ 캐시 메모리의 정의

- CPU와 주기억장치의 속도 차이를 극복하기 위하여 CPU와 주기억장치 사이에 위치한 소형 고속 메모리(SRAM 사용)
- 주기억장치에 비해 5~10배 정도 접근 속도가 빠른 기억장치로 느리게 동작하는 주기억장치와 빠르게 동작하는 CPU의 속도 차이를 줄여 데이터와 명령어 처리 속도를 향상시키는 기억장치

□ 캐시 메모리의 필요성



- DMA 컨트롤러(Controller)가 기억장치를 사용하는 동안 CPU가 캐시를 사용하게 하여 CPU 사용 효율을 높임

캐시 메모리(Cache Memory)

□ 캐시 메모리의 특징

구분	내용	비고
Locality	- 블록 단위의 메모리 참조	시간, 공간
Mapping	- 주기억 장치와 캐시 메모리 간의 메모리 매핑 적용	직접, 연관
Coherence	- 병렬 처리시 로컬 캐시와 공유 메모리 간 데이터 일관성 유지	공유 캐시

□ 캐시 메모리 지역성(locality) : 짧은 시간 동안에 제한된 주소 공간의 일부만 참조되는 현상

구분	설 명	비고
시간적	- 최근 참조된 기억 장소가 가까운 장래에 계속 참조될 가능성이 높은 성질	순환(Looping), 서브 프로그램, LRU(Least Recently Used), Stack
공간적	- 최근에 참조될 기억 장소와 가까운 기억 장소가 가까운 장래에 계속 참조될 가능성이 높은 성질	Array
순차적	- 분기가 없는 한 데이터가 기억장치에 저장된 순서대로 순차적으로 인출되고 실행	

캐시 메모리(Cache Memory)

3. Hit Ratio & 캐시의 동작기술

구분	내용
Hit Ratio	- 캐시를 가진 컴퓨터의 성능을 나타내는 척도 - $\text{Hit Ratio} = \text{적중 횟수} / (\text{적중 횟수} + \text{실패 횟수})$ - CPU가 메모리를 참조할 경우 (Cache에서 찾음 → Hit Ratio, 주기억장치에서 찾음 → Miss Ratio)
캐시의 Write	- Write Through : 캐시에 데이터를 쓸 때, 동시에 주기억장치를 갱신 - Write Back : 캐시에서 데이터가 제거될 때, 주기억장치에 내용을 작성
캐시의 일관성 Cache Coherence	- 컴퓨터의 속도 향상을 위해 캐시의 내용과 주기억장치의 내용을 일치시킴 [Cache Coherence 유지 기법]

캐시 메모리(Cache Memory)

1) 공유 Cache 사용

- 모든 프로세서들이 하나의 Cache 사용
- 구조가 간단하나 프로세서 간 충돌이 심함

2) 공유변수를 Cache에 저장하지 않는 방법

3) 버스감시 메커니즘 이용

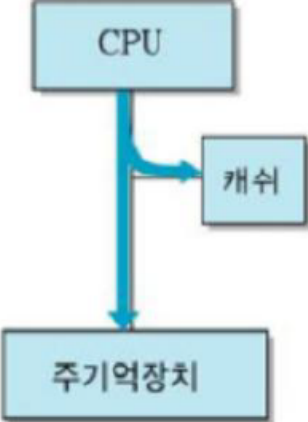
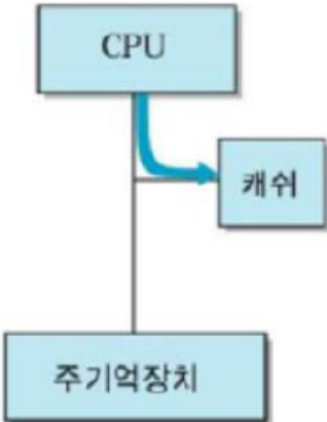
- 스누프 제어기 : 변경에 대한 자신의 Cache 블록 상태 제어
- 쓰기 갱신 (Write update or Broadcast) 방법 : 다중 읽기/쓰기 가능, 공유 블록 갱신 시 다른 모든 Cache에 갱신되는 정보가 보내짐
- 쓰기 무효 (Write invalid) 방법 : 다중 읽기, 한번에 하나만 쓰기

4) 디렉토리 기반 Cache coherence 유지 기법

- 쓰기 동작을 할 경우 다른 Cache의 블록을 무조건 무효화 시킨 후 쓰기 동작을 배타적으로 수행
- 캐시의 전역 정보 상태를 디렉토리에 저장하여 일관성 유지
- Full Map, Limited, Chained 방식

캐시 메모리(Cache Memory)

4. 캐시 메모리 쓰기 정책 비교

구분	Write Through	Write Back
구성도		
개념	- 쓰기 동작 시 캐시와 주기억장치에 동시에 쓰는 방식	- 캐시에만 쓰기를 하고 해당 데이터가 swap-out될 때 주기억장치에 복사하는 방식
장점	- 구조가 단순함 - 캐시와 주기억장치의 일관성 유지	- 기억장치 쓰기 동작의 횟수 최소화 및 쓰기 시간 단축
단점	- 버스의 트래픽 양이 많아짐 - 주기억장치 쓰기 포함으로 쓰기 시간이 길어짐	- 캐시와 주기억장치 일관성 유지 어려움(Cache Coherency 문제) - 블록 교체 시 캐시의 상태 비트(dirty bit) 확인 절차 필요

캐시 메모리(Cache Memory)

5. Mapping Process

- 주기억장치로부터 캐시메모리로 데이터를 전송하는 작업
- 캐시 메모리 라인 수가 메인 메모리의 블록의 수 보다 적기 때문에 Mapping이 필요

분류	내용
Associative Mapping (연관사상)	- 빠르고 융통적임(메모리 워드(임의의 부분), 번지와 데이터를 함께 저장) - 메모리의 각 블록이 캐시의 어느 라인에나 적재 가능(블록 단위 아님) - Cache Hit Ratio는 높아지나 회로가 복잡해지고 처리속도가 느림
Direct Mapping (직접 사상)	- Associative Mapping 보다 항상 빠르지만 경우에, 특정 상황에서 비효율적임. - 메모리 참조 요청 시 CPU 번지의 태그 필드와 캐시의 태그 필드 비교 - 메모리 각 블록이 캐시 특정 라인으로 적재(캐시와 메모리가 1:1 대응) - 회로 구현이 용이, 간단, 처리속도가 빠름 - Hit율이 떨어질 수 있으며, Cache Miss Ratio가 큼
Set Associative Mapping (집합연관사상)	- 직접 사상과 연관 사상의 장점을 결합 - 캐시와 메모리의 대응이 N : 1 (N-way Set Associative)

캐시 메모리(Cache Memory)

6. 캐시 교체 방식

- 캐시 메모리의 용량이 한계점에 도달했을 때 그 내용을 교체 하기 위한 알고리즘

구분	내용
LRU	- Least Recently Used - 사용되지 않은 채로 가장 오래 있었던 Block 교체
FIFO	- First In First Out - Cache 내에 가장 오래 있었던 Block 교체
LFU	- Least Frequently Used - 사용빈도수가 가장 낮은 Block 교체
Random	- 후보 블록 중 한 블록을 임의로 선택
Optimal	- 향후 가장 참조되지 않을 Block을 교체 → 실현 불가능

감사합니다!



컴퓨터구조 및 운영체제(CAOS)



2. 기억장치 part2

KPC 기본반

Contents

2. 기억장치.....	29
(1) 메모리 계층구조.....	30
(2) 메모리 관리기법.....	31
(3) 메모리 단편화(Fragmentation).....	34
(4) 캐시 메모리(Cache Memory).....	38
(5) 캐시 일관성(Cache Coherence).....	44
(6) 주기억장치(Main Memory)	46
(7) 메모리 인터리빙(Memory Interleaving).....	50
(8) 보조기억장치(auxiliary memory).....	53
(9) 가상 메모리(Virtual Memory).....	55
(10) 벨레디 이상현상(Belady's Anomaly).....	61

가상 메모리(Virtual Memory)

1. 물리 공간의 한계성을 극복하여 효율적인 자원 이용을 돕는 가상 메모리의 개요

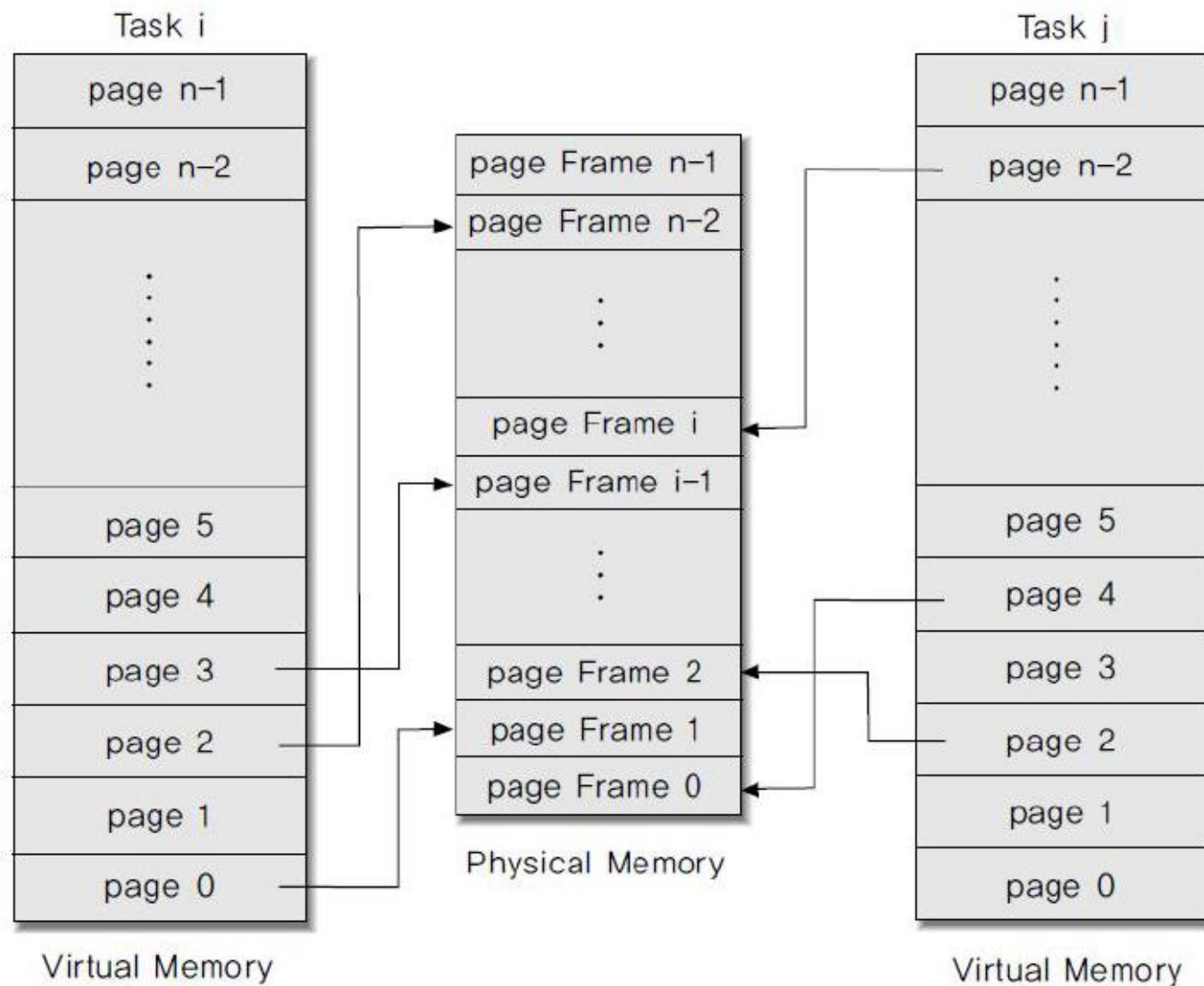
□ 가상 메모리(Virtual Memory)의 정의

- 사용자에게 주기억장치보다 더 큰 용량의 가상 기억 공간을 제공하는 기억장치관리 기법
- 주기억장치 안의 프로그램 양이 많아질 때, 사용하지 않는 프로그램을 보조기억장치 안의 특별한 영역으로 옮겨서, 그 보조기억장치 부분을 주기억 장치처럼 사용할 수 있는데, 이때 사용하는 보조기억장치의 일부분을 가상 메모리라고 함

□ 페이지 적재

- 페이지는 가상 메모리를 일정한 고정 크기로 분할하는 단위를 의미하며, 페이지 프레임은 물리 메모리를 일정한 고정 크기(예 : 인텔 펜티엄 프로세서에서는 4KB, 알파칩의 경우 8KB)로 분할한 단위를 의미
- 일반적으로 페이지와 페이지 프레임의 크기는 동일
- 예를 들어, 인텔 펜티엄 프로세서를 사용하고 장착된 메모리의 크기가 512MB라고 가정하면 물리 메모리는 512MB/4KB개의 페이지 프레임 단위로 분할되고, 가상 메모리의 4GB도 마찬가지로 4KB 크기로 분할됨, 분할된 각각의 영역을 하나의 페이지라고 함

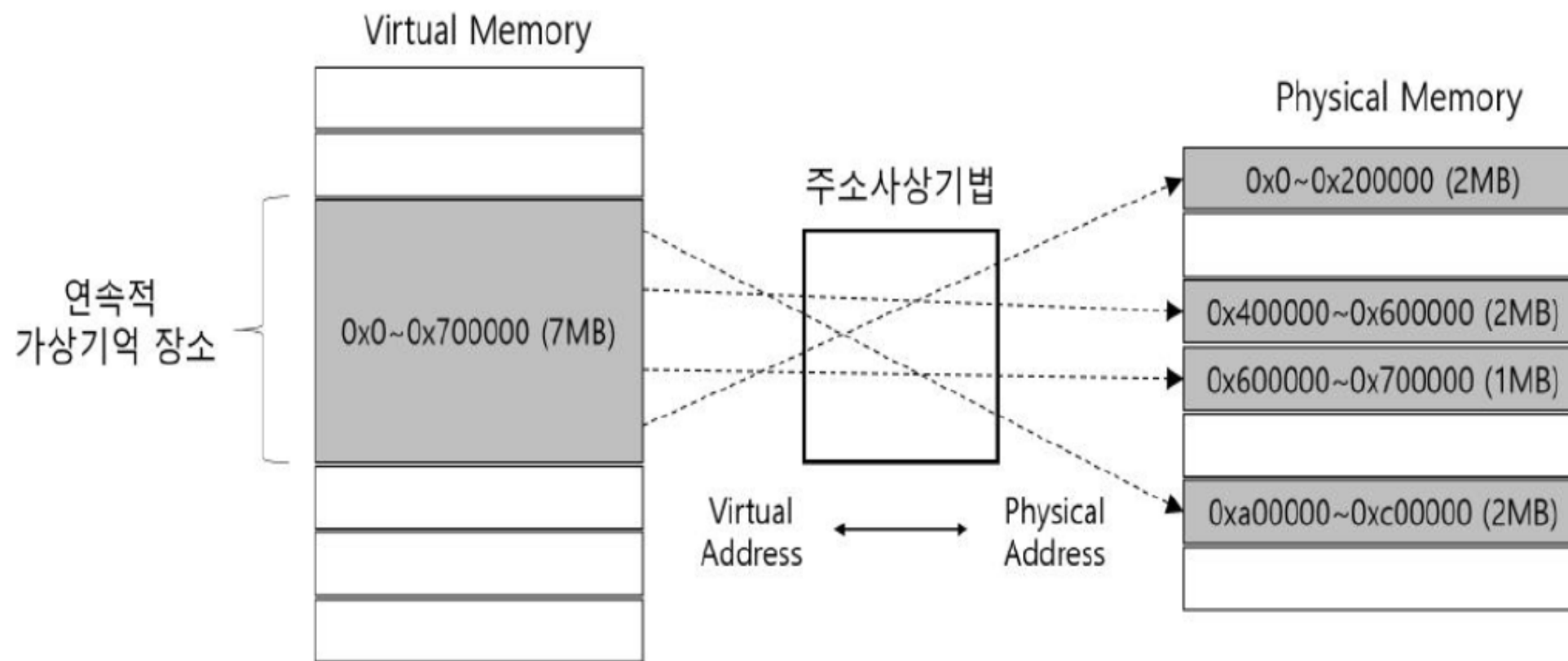
가상 메모리(Virtual Memory)



가상 메모리(Virtual Memory)

2. 가상 메모리 관리 및 연관 개념도

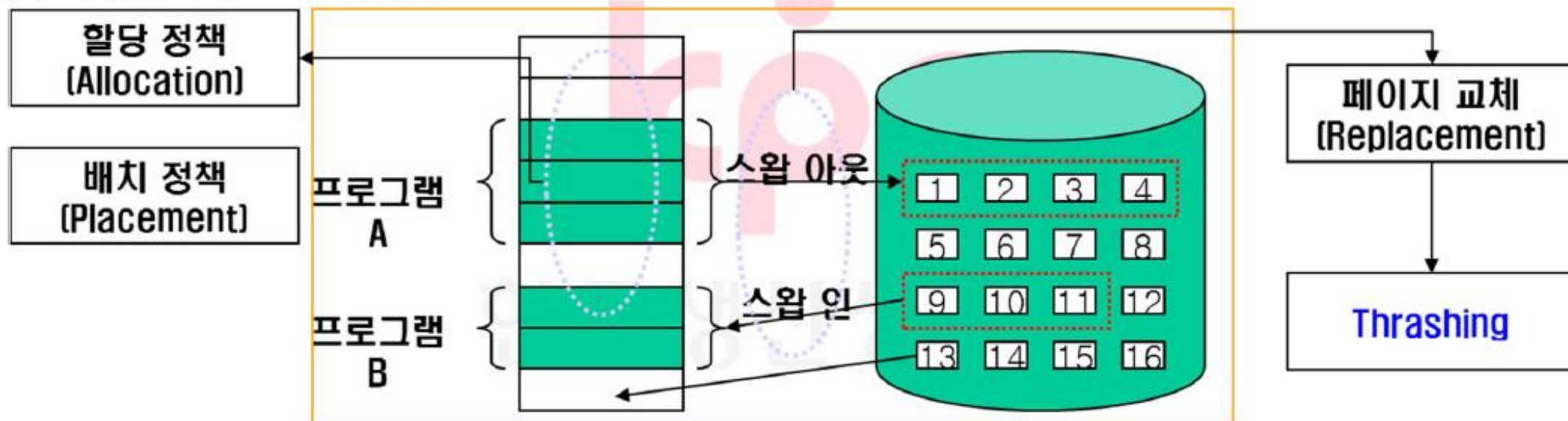
□ 가상 메모리 관리



- 각 프로그램에 실제 메모리 주소가 아닌 논리 메모리 주소를 전달하여 메모리를 확장하는 효과
- 하드 디스크의 큰 공간을 활용하여 주기억장치보다 더 큰 공간을 제공 가능

가상 메모리(Virtual Memory)

□ 가상메모리 연관 개념도



- 각 프로세스에게 메인 메모리를 할당하고, 디스크(보조기억장치)의 어느 항목을 호출할 것인지, 어느 곳에 배치할 것인지, 무엇을 교체할 것인지 결정하는 메커니즘

가상 메모리(Virtual Memory)

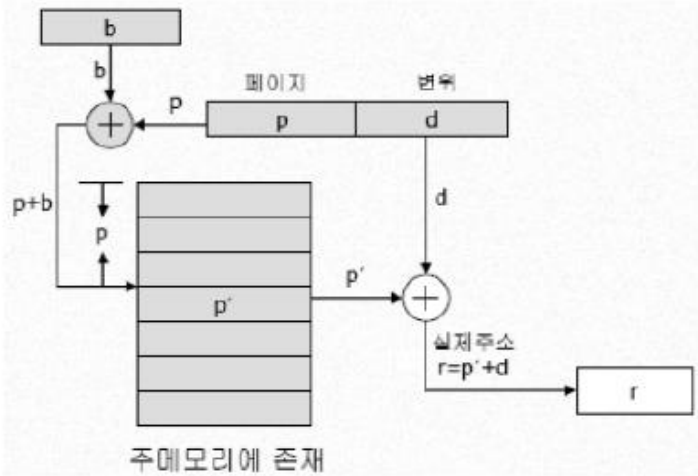
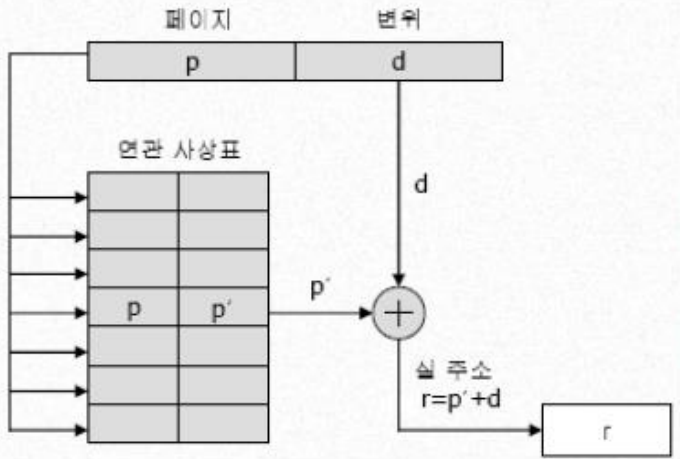
3. 가상 메모리 관리 기법 및 주소 사상기법

□ 가상 메모리 관리기법

관리정책	내용
할당 정책	- How Much - 각 프로세스에게 할당할 메모리의 양을 관리 - 프로세스 실행 중 메인 메모리 할당량 변화 알고리즘 - 고정할당기법과 가변 할당기법으로 구분
호출 정책	- When - 언제 어느 항목들을 보조기억장치에서 주기억장치로 가져올 것인지 결정 - 요구 호출(Demand Fetch)기법과 예측 호출(Pre Fetch)기법
배치 정책	- Where
	- 프로그램의 한 블록을 주기억장치의 어디에 배치할 것인가 관리 - First, Best, Next, Worst Fit
교체 정책	- Who - 주기억장치에 적재할 공간이 없을 경우 무엇과 교체할 것인가에 대한 관리 - FIFO, LRU, LFU, NUR

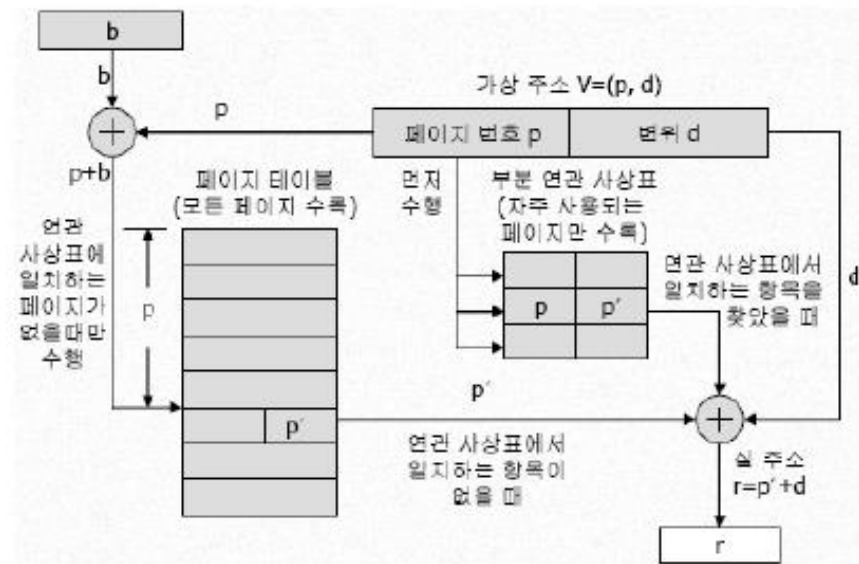
가상 메모리(Virtual Memory)

주소 사상 기법

구분	개념도	내용
Direct Mapping	 주메모리에 존재	- Page table을 주기억장치에 위치 - 데이터 접근을 위해 Page Table과 Data에 각각 한 번씩 주기억장치 접근 - 접근 시간이 길어짐
Associative Mapping		- Page table을 Associative buffer에 저장 - 속도가 빠르고 가격이 비쌈

가상 메모리(Virtual Memory)

Direct/Associative Mapping



- Page table을 주기억장치와 Associative buffer에 분산 저장
- 연관 사상 테이블 : Locality에 근거하여, 최근에 가장 많이 참조된 page만 유지
- 페이지 사상 테이블 : 연관사상테이블에서 제외된 나머지 page들을 모두 유지, 경제성과 성능 절충 : 소수의 연관 레지스터를 먼저 검색한 후 없으면 페이지 테이블 검색

가상 메모리(Virtual Memory)

5. Page Replacement(페이지 교체 알고리즘)

구분	내용
기본 방법	- 빈 프레임을 찾아 내용을 전이 시키는 것. 빈 프레임이 없으면 페이지 교체 알고리즘을 구동시켜 빈 프레임을 만든 뒤 기록함.
FIFO	- 메모리에 올라온 지 가장 오래된 페이지를 내쫓음 - 가장 간단한 알고리즘.
최적 페이지 교체 (Optimal Page)	- 앞으로 가장 오랫동안 사용되지 않을 페이지를 찾아서 교체함
LRU	- 가장 오랫동안 사용되지 않은 페이지를 교체 함 - Least recently used
LRU 근사페이지 교체	- 참조 비트를 이용하여 0,1 을 변화 시켜 일정 시간 경과 후 사용되지 않은 페이지 교체
계수 기반 페이지 교체	- LFU (Least Frequently Used) : 참조 회수가 가장 작은 페이지 교체
페이지 버퍼링 알고리즘	- 페이지 교체 기법들과 함께 사용하는 버퍼 변환 기법

가상 메모리(Virtual Memory)

6. 가상기억장치 사용시의 문제점 및 해결방안

문제점	내용	해결 방안
Demand Paging	- 요구될 때에만 Process의 Page들을 적재하는 방식 (프로그램은 한 명령어에도 여러 개의 페이지 부재를 일으킬 수 있음)	- 참조 지역성 (Locality of Reference) : 프로그램의 어느 한 특정 부분만 한동안 집중적으로 참조 하기 때문에, 페이징의 성능이 좋아 지는 현상
Page Fault Trap	- 기억장치에 적재되지 않은 Page를 사용하려는 경우 발생	1) 프로세스를 검사하여 유무 판단 2) 빈 공간(Free Frame)의 검색 3) 디스크로부터 읽도록 요구 4) 중단되었던 명령의 재 수행

가상 메모리(Virtual Memory)

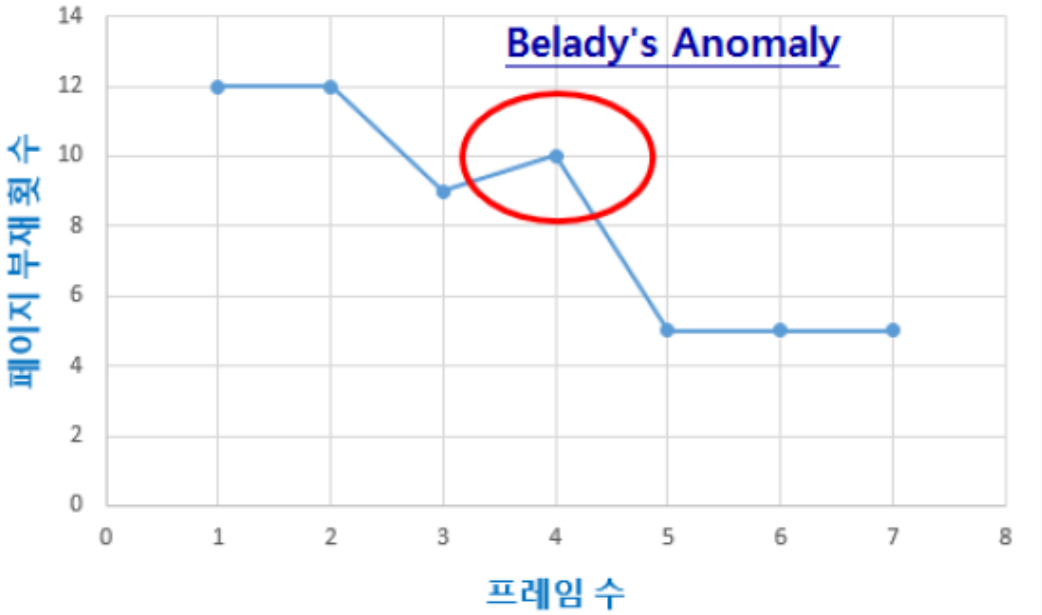
Thrashing 발생	- Page 부재가 너무 빈번하게 발생하여 CPU가 Process 수행 보다 Page 교체에 더 많은 시간을 소요하는 비정상적인 현상 발생	1) Locality Model 사용 : 프로그램들은 일정시간 동안 일정한 장소들만 집중적으로 사용하는 원리 이용 2) Working Set : 일정 시간 동안 참조되는 페이지의 집합을 주기억
		장치에 유지 3) Page Fault - Frequency Process의 Page Fault 빈도 (Frequency)에 따라 Residence Set을 조정 - PFF가 높으면 Residence set 크기 증가, 낮으면 줄임

벨레디 이상현상

1. 페이지 프레임 수와 페이지 부재율 이상관계, FIFO 알고리즘에서 벨레디 이상현상

구분	그래프	설명																
일반적인 교체기법	<table><thead><tr><th>프레임 수</th><th>페이지 부재 횟수</th></tr></thead><tbody><tr><td>1</td><td>13</td></tr><tr><td>2</td><td>7</td></tr><tr><td>3</td><td>5</td></tr><tr><td>4</td><td>4.5</td></tr><tr><td>5</td><td>4.2</td></tr><tr><td>6</td><td>4</td></tr><tr><td>7</td><td>4</td></tr></tbody></table>	프레임 수	페이지 부재 횟수	1	13	2	7	3	5	4	4.5	5	4.2	6	4	7	4	- 일반 적인 가상메모리 교체 기법에서는 프레임 수가 늘어나면 페이지 부재 횟수가 감소함
프레임 수	페이지 부재 횟수																	
1	13																	
2	7																	
3	5																	
4	4.5																	
5	4.2																	
6	4																	
7	4																	

벨레디 이상현상

FIFO 교체기법	 페이지 부재 횟수 프레임 수 <u>Belady's Anomaly</u>	- FIFO 알고리즘은 어떤 페이지를 교체해야 할 때, 메모리에 올라 온지 가장 오래된 페이지를 교체 - 일반적인 교체 기법과 달리 프레임 수가 3개에서 4개로 증가 했을 때 부재 횟수가 더 증가하는 이상 현상 발생
벨레디 이상현상	- 위 그래프처럼 프레임의 숫자가 증가해도 페이지의 부재율이 증가 - 일반적으로 프레임이 많으면 페이지 부재 횟수가 줄어드는 현상에 반대되는 현상이 나타나는 벨레디 이상현상(Belady's Anomaly)이 발생함 - 벨레디 이상현상(Belady's Anomaly)를 해결하기 위해 Locality를 고려한 LRU(Least Recently Used)를 사용함	

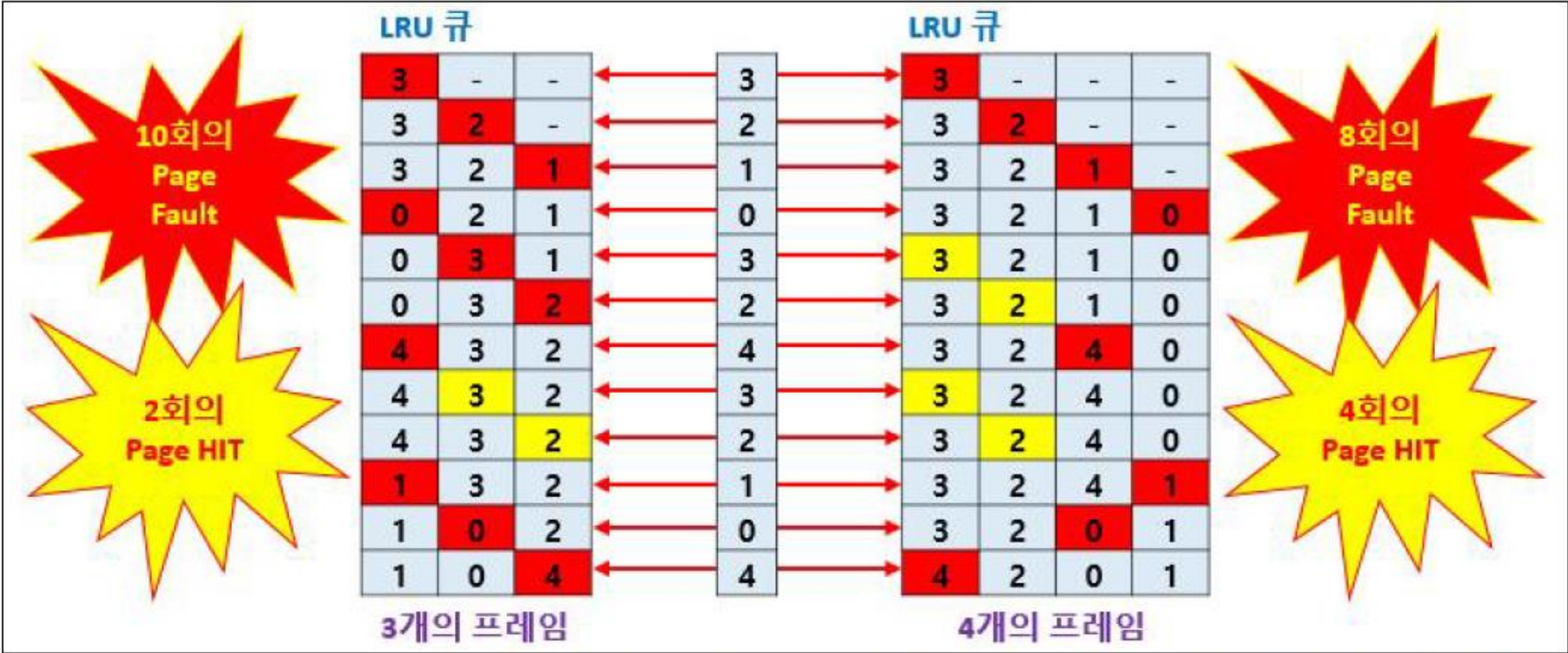
□ FIFO 알고리즘



- 프레임 수가 4개로 늘어났지만 오히려 페이지 부재가 10회로 증가하는 Belady's Anomaly 발생하므로 LRU(Least Recently Used)등 다른 기법을 사용해 이상현상 해결을 해야 함

벨레디 이상현상

□ LRU 알고리즘



- LRU(Least Recently Used)를 사용할 경우 페이지 프레임 수가 3개에서 4개로 증가할 때 페이지 부재수는 10회에서 8회로 감소해 FIFO 이상현상인 벨레디 이상현상을 해결함

벨레디 이상현상

3. 벨레디 이상현상을 해결하기 위한 방안

구분	방안	내용
페이지 교체기법 변경	LRU	- 페이지 교체 시점에서 가장 최근에 참조가 안된 페이지를 교체하는 기법
	OTP	- 앞으로 가장 오랫동안 사용하지 않을 페이지를 교체하는 기법
최적화 설계	Locality	- 시간적, 공간적, 순차적 지역성 활용
	PFF	- 프로세스의 페이지 폴트 빈도에 따라 Residence set을 조정해 PFF가 높으면 Residence set의 크기를 증가 시키고, 낮으면 줄임

감사합니다!

컴퓨터구조 및 운영체제(CAOS)

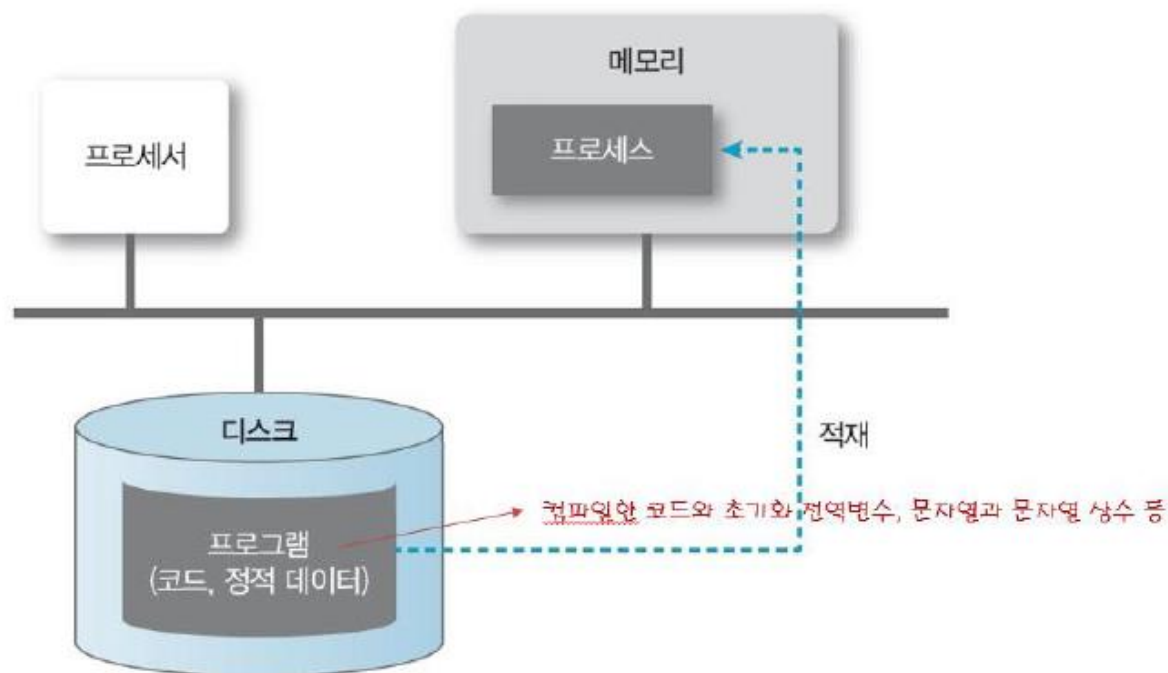
3. 운영체제(Operation System) part1

3. 운영체제(OS, Operating System)	66
(1) 운영체제(OS, Operating System)	67
(2) 프로세스(Process).....	70
(3) PCB(Process Control Block)	73
(4) 문맥교환(Context Switching).....	75
(5) 인터럽트(Interrupt).....	80
(6) CPU 스케줄링(CPU Scheduling).....	84

프로세스(Process)

1. 실행되고 있는 프로그램의 인스턴스, 프로세스의 개요

□ Process 의 정의



- 레지스터(register), 스택(stack), 포인트(point), 프로그램, 데이터 등의 집합체로 실행 중인 프로그램
- 비동기적 행위, 프로시저(procedure)가 활동 중인 것, 실행 중인 프로시저의 제어 경로 등을 의미하는 것으로 디스패치(dispatch)된 작업 단위

프로세스(Process)

□ Process 의 상태전이



- 프로세스는 상황과 조건에 따라 실행(running), 준비(ready), 대기(wait, blocked) 상태에 있다가 다른 상태로 변화되는 전이를 반복

프로세스(Process)

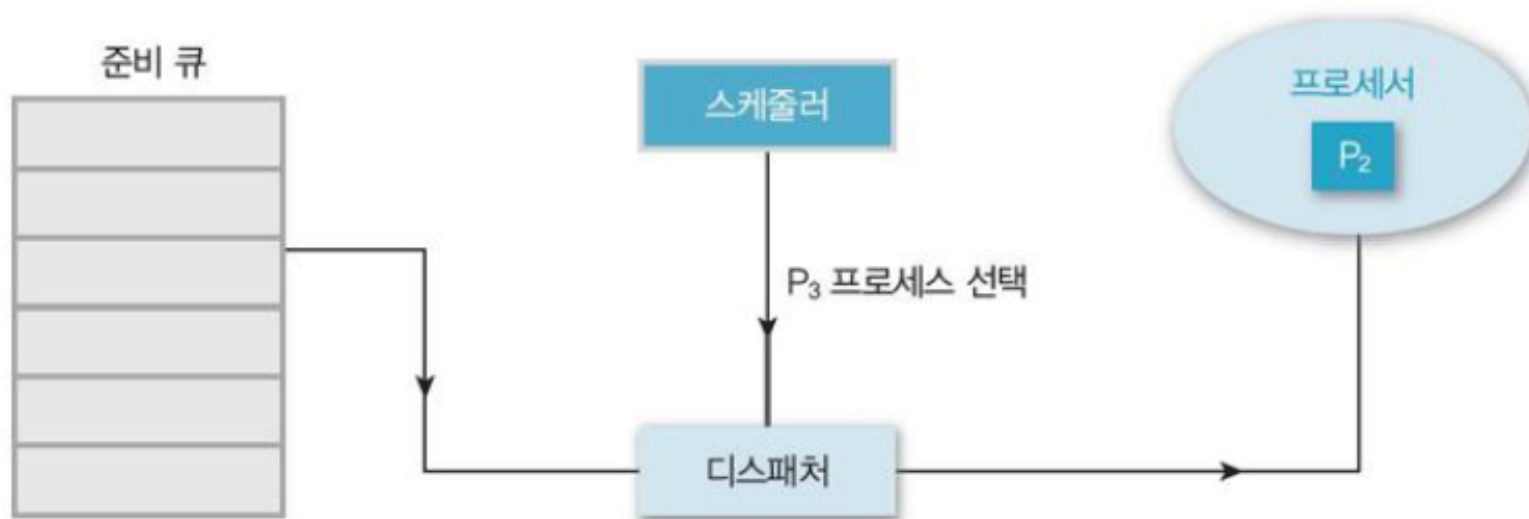
상태	설명
준비 상태	- 프로세스가 실행을 준비하고 있는 상태(프로세스가 프로세서에 할당 대기) - 디스패치를 통해 실행 상태로 전이 * 디스패치¹: 준비 상태의 프로세스가 CPU를 배정받아 실행 상태로 바뀌는 것
실행 상태	- 프로세스가 CPU를 차지하고 실행 중인 상태
	- Time run out: 시간 주기 내에 작업이 끝나지 않으면 인터럽트를 발생시켜 제어권을 운영체제로 넘기고 준비 상태로 전이 - Block: 실행 상태의 프로세스가 입출력이나 다른 작업을 수행하는 경우, 제어권을 운영체제에 넘기고 작업이 끝날 때까지 대기 상태로 전이
대기 상태	- 프로세스가 어떤 사건(Event)을 기다리는 상태 - Wake up: 입출력이나 다른 작업이 끝났을 때, 프로세스가 준비 상태로 전이하는 것

프로세스(Process)

2. 프로세스 스케줄링

□ 스케줄링 큐

- 프로세스의 스케줄링을 위해 운영체제는 여러 종류의 큐를 유지하며, 큐는 일반적으로 연결 리스트(linked list)로 구현됨



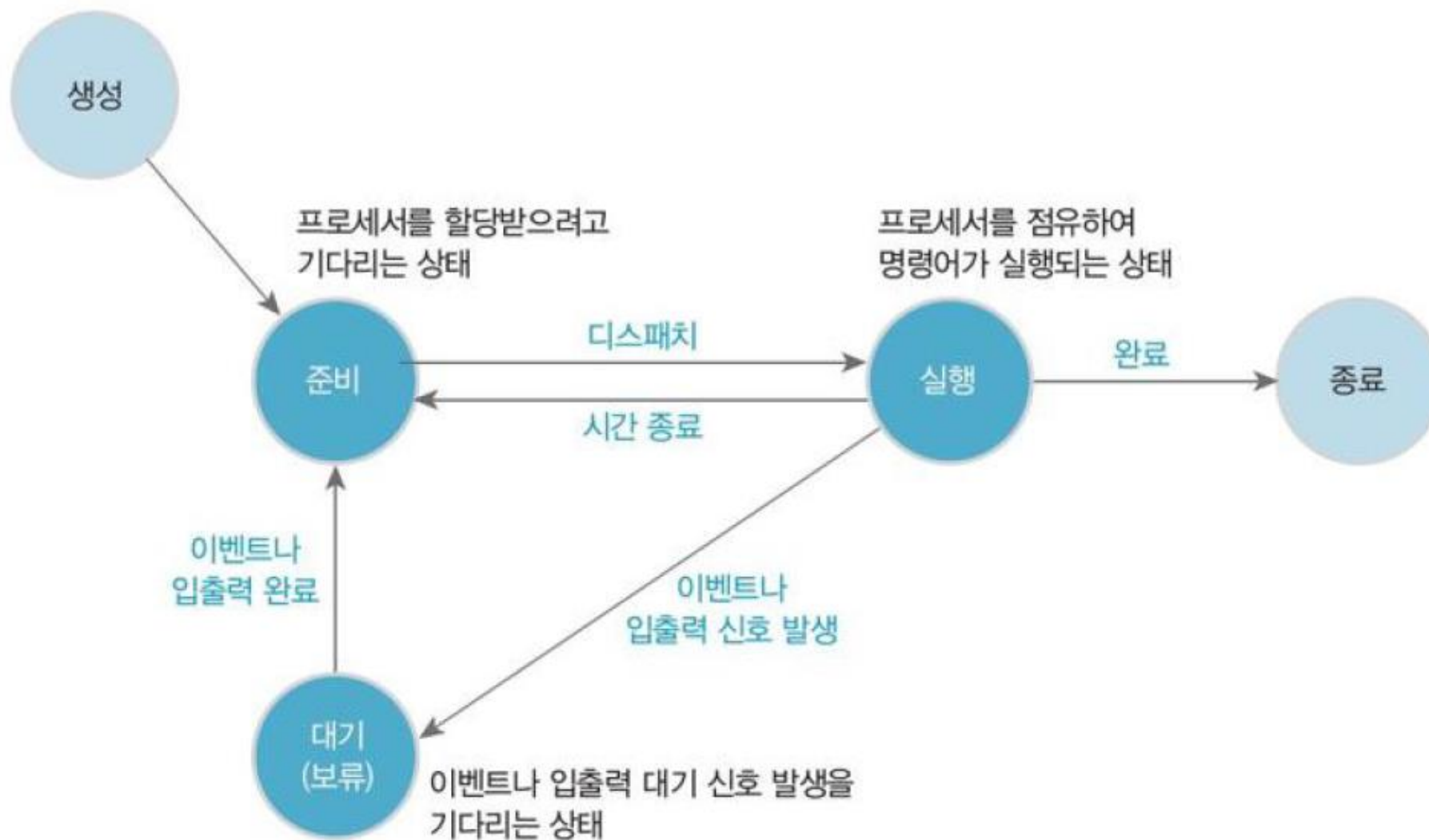
프로세스(Process)

□ 큐의 종류

구분	내용
작업 큐(job queue)	- 프로세스가 시스템에 처음 들어와서 대기하는 큐
준비완료 큐 (ready queue)	- 주기억장치에 적재되어 실행을 기다리는 프로세스를 유지하는 큐
장치 큐 (device queue)	- 장치를 사용하기 위해 기다리는 큐, 각 장치는 자신만의 장치 큐가 있음
대기 큐 (waiting queue)	- 특정한 사건마다 그 사건을 기다리는 프로세스를 유지하는 큐를 사용

프로세스(Process)

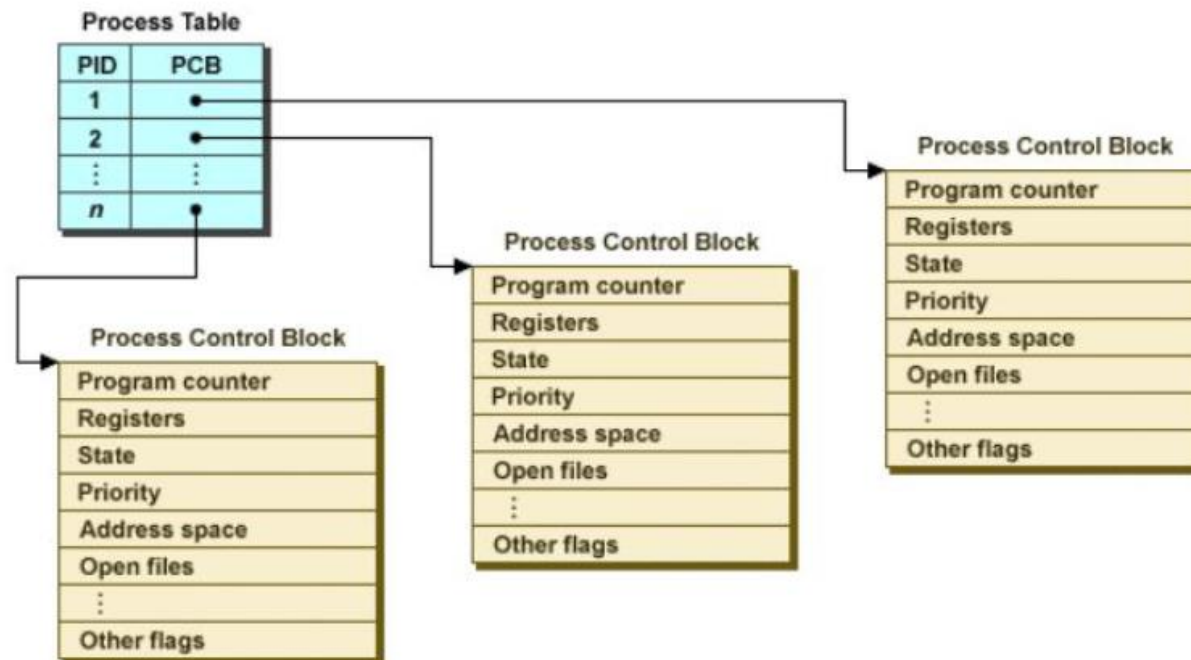
※ [참고] Process State



PCB(Process Control Block)

1. PCB(Process Control Block)의 개요

- 운영체제가 프로그램 실행을 위해 필요한 자료를 담고 있는 자료구조
- 프로세스는 운영체제 내에서 프로세스 제어 블록(PCB)으로 표현 됨
- PCB는 프로세스 생성 시 만들어지고, 메인 메모리에 유지되며, OS에서 한 프로세스의 전체를 정의함
- 프로세스 제어 블록의 정보는 문맥 교환에 이용되며, 인터럽트 처리, 자원 할당, 스케줄링 등을 수행하는 OS의 모든 모듈이 읽고 수정할 수 있음



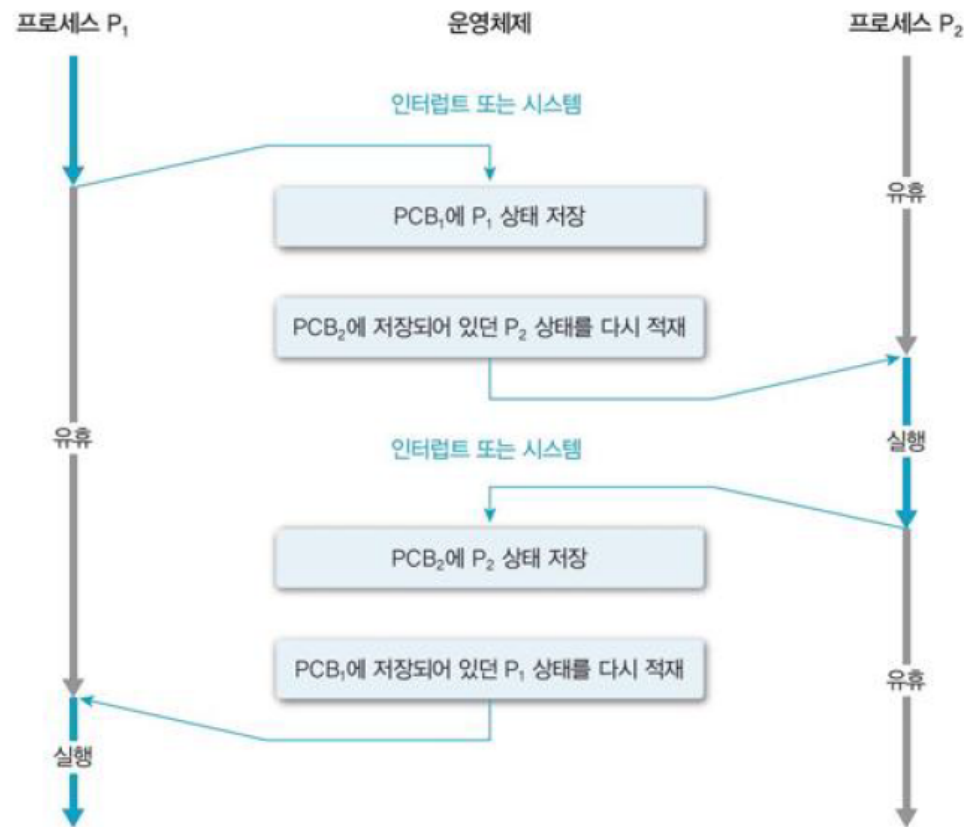
PCB(Process Control Block)

구성 정보	구성 정보의 내용 및 역할
PID(프로세스 식별자)	- 각 프로세스에 대한 고유 식별자(숫자, 색인항목)
프로세스 상태	- 생성, 준비, 실행, 대기, 중단 등의 프로세스 상태
프로그램 카운터	- 프로세스 실행을 위한 다음 명령의 주소 표시
레지스터 저장 영역	- 누산기, 인덱스 레지스터, 범용 레지스터, 조건 코드 등에 관한 정보 - 컴퓨터 구조에 따라 다른 형태 - 인터럽트가 발생 시 프로그램 카운터와 함께 저장, 다시 실행 시 원상 복귀
프로세서 스케줄링 정보	- 우선순위, 스케줄링 큐 포인터, 스케줄 매개변수
계정 정보	- 프로세서 사용시간, 실제 사용시간, 사용 상한 시간, 계정 정보, 작업이나 프로세스 번호 등
입출력 상태 정보	- 입출력 요구 프로세스에 할당된 입출력 장치, 개방된 파일 목록
메모리 관리 정보	- 메모리 영역 정의에 필요한 상한/하한 레지스터(경계 레지스터) 또는 페이지 테이블 정보

PCB(Process Control Block)

3. PCB 를 이용한 프로세스 교체, 문맥 교환(Context Switching)의 절차

- 문맥 교환(Context Switching) : 현재 실행중인 프로세스 정보를 PCB에 저장, 다른 프로세스 정보를 복구하는 작업
- 과도한 문맥 교환은 오버헤드를 초래. 적정 수준의 멀티 프로그래밍 및 스레드 사용을 통한 오버헤드 감소가 가능함



문맥교환(Context Switching)

1. 병행처리를 위한 문맥교환(Context Switching)의 이해

□ 문맥(Context)의 정의

- 여러 프로세스가 함께 수행되는 시분할 시스템 환경에서 특정 프로세스의 처리를 위해 할당된 자원이 일시적으로 반환되어 다른 선점 프로세스를 처리하고 재할당 되었을 경우, 기존 수행 중이었던 작업시점부터 재현하기 위한 정보
- Task가 사용하는 CPU 레지스터 들의 값

□ 문맥교환(Context Switch)의 정의

- 하나의 프로세스가 CPU를 사용중인 상태에서 다른 프로세스가 CPU를 사용하도록 하기 위해, 이전의 프로세스 상태를 보관하고 새로운 프로세스의 상태를 적재하는 일련의 작업
- 이전 프로세스의 상태 레지스터 내용을 보관하고, 다른 프로세스의 레지스터를 적재하여 프로세스를 교환하는 일련의 과정

문맥교환(Context Switching)

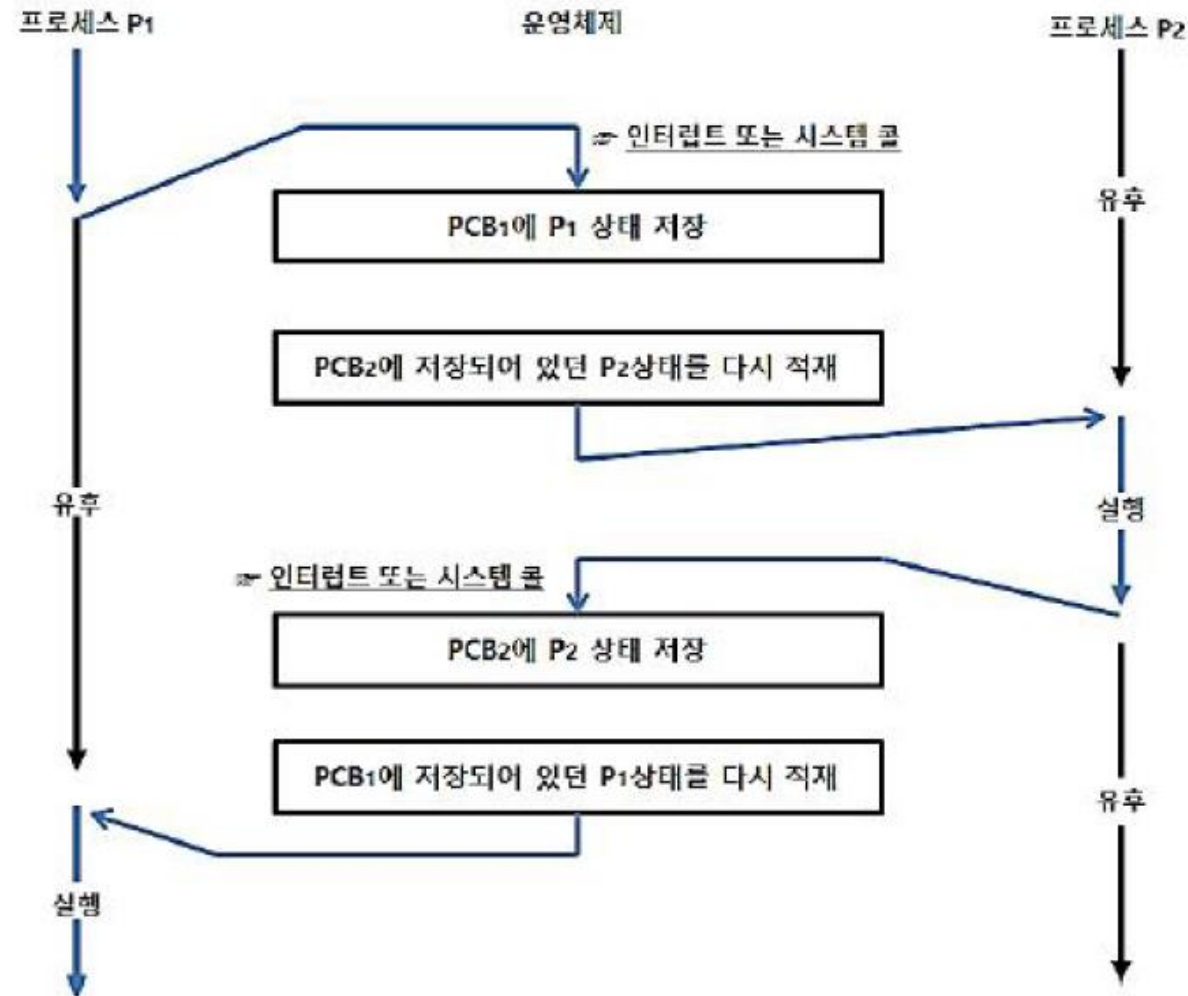
□ Context Switching 관련 개념

용어	설명
Dispatching	- 준비 상태의 프로세스들 중에서 우선순위가 가장 높은 프로세스에게 CPU를 할당
Time Quantum	- 특정 프로세스에게 할당되는 시간의 단위 - 특정 프로세스의 CPU 독점을 방지
Preemption	- Time Quantum이 초과하면 인터럽트를 통해 CPU 사용권을 빼앗는 스케줄링 기법

문맥교환(Context Switching)

2. 병행처리 수행을 위한 문맥교환 절차

□ 문맥교환 절차



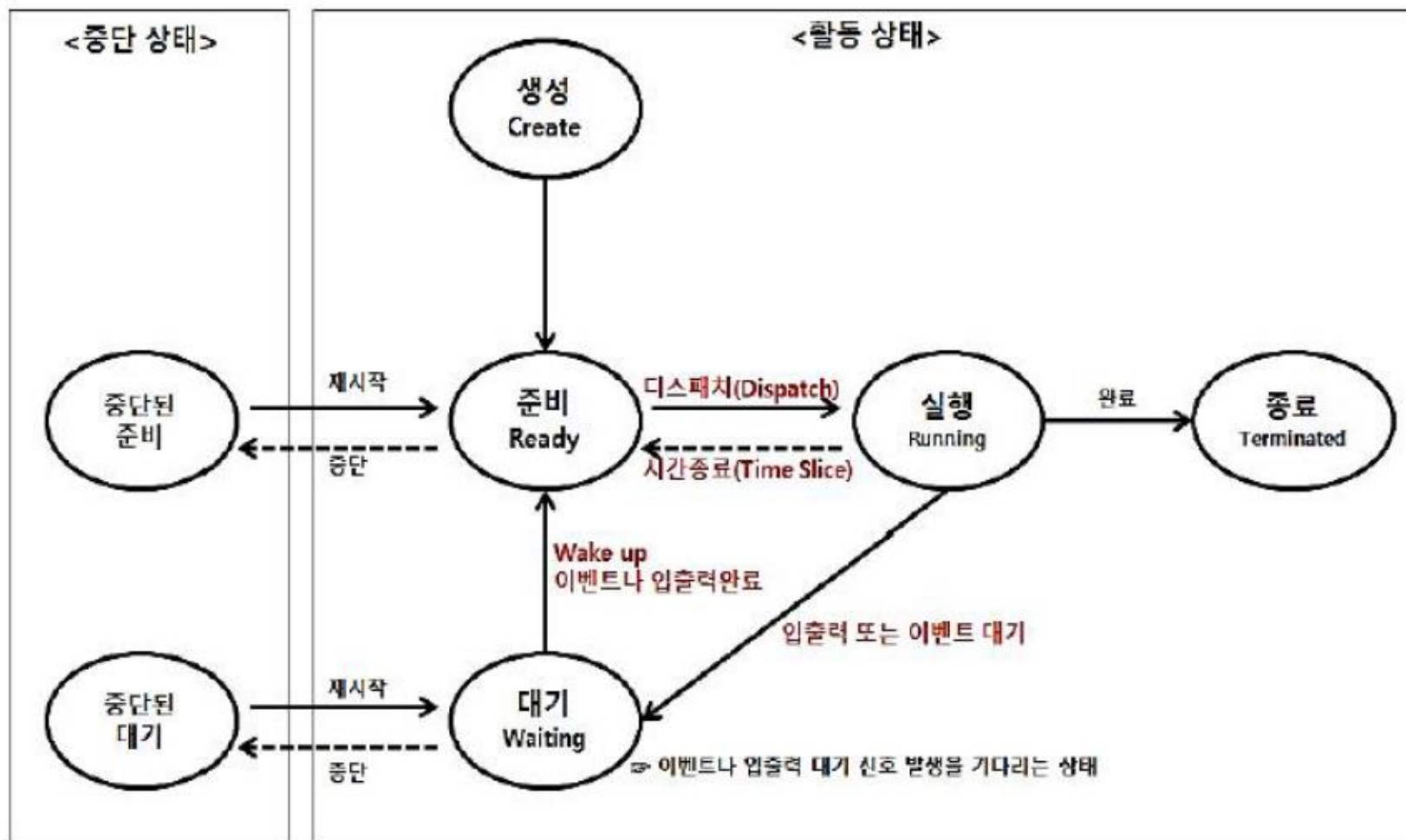
문맥교환(Context Switching)

구분	절차	내용
프로세스 P1	실행	- CPU가 할당되어 프로세스 P1이 실행중인 상태
운영체제	인터럽트 또는 시스템 콜	- 인터럽트나 시스템 호출에 의해 프로세스 P1을 대기처리
	PCB1 저장	- 프로세스 P1에 대한 PCB1을 복귀시 사용위해 저장
	PCB2 적재	- 새로 실행될 프로세스 P2에 CPU 할당하여 실행
프로세스 P2	실행	- CPU가 새로 할당되어 프로세스 P2가 실행중인 상태
운영체제	인터럽트 또는 시스템 콜	- 인터럽트나 시스템 호출에 의해 프로세스 P2을 대기 처리
	PCB2 저장	- 프로세스 P2에 대한 PCB2을 복귀시 사용위해 저장
	PCB1 재적재	- 대기 중인 프로세스 P1을 복구 후 CPU 할당하고 실행
프로세스 P1	실행	- CPU가 할당되어 프로세스 P1이 실행중인 상태

문맥교환(Context Switching)

3. 문맥교환이 발생하는 조건

□ 프로세스 상태 천이시 문맥교환 발생시점



문맥교환(Context Switching)

□ 문맥교환이 발생하는 조건

발생조건	상태전이	내용
디스패치 (dispatch)	준비 → 실행	- 준비큐의 첫번째 프로세스는 CPU를 할당 받고 실행상태로 전이 - 프로세스 스케줄러에 의해 결정된 우선순위에 따라 CPU 할당
시간종료 (time slice)	실행 → 준비	- 프로세스가 CPU 사용 가능한 할당 시간을 초과한 경우 - 시간 초과되면 PCB를 저장하고, CPU 양도 후 준비상태로 전이
입출력 (I/O)	실행 → 대기	- 실행 중인 프로세스가 I/O로 인해 I/O 전용 프로세스에게 CPU 양도 후 대기 상태로 전이 - I/O 처리는 CPU가 아닌 I/O 전용 프로세스가 담당
시스템 콜 (system call)	실행 → 대기	- 실행될 준비가 되어 있지 않은 서비스를 호출할 경우 대기 상태로 전이

문맥교환(Context Switching)

4. Context Switch 과정의 오버헤드 및 해결방안

□ 오버헤드 발생 시점

단계	실행루틴	오버헤드작업
1	현 프로세스	-
2	Interrupt 처리 루틴	현재 상태를 PCB 에 저장
3	프로세스 스케줄러	다음 실행 프로세스를 준비 Queue 에서 선택
4	Dispatch	다음 Process 의 PCB 값 복구
5	다음 프로세스	-

□ 문맥교환시 오버헤드 해결 방안

방법	해결방안
원인제거	- 문맥교환이 자주 발생하지 않도록 다중 프로그래밍 수준을 조절(낮춤)
구조변경	- 스택 중심의 시스템에서는 스택 포인트를 변경하여 프로세스 간 문맥교환을 수행
스레드 이용	- 스레드를 이용하여 문맥교환 부하 최소화

- 오버헤드 문제점의 해결을 통해 CPU가 프로세스 실행보다 문맥교환에 더 많은 시간을 소모하여 성능이 저하되는 쓰레싱(Thrashing) 현상의 발생을 억제하여 성능 저하 문제 방지

인터럽트(Interrupt)

1. 장치의 상태 변화에 효과적으로 대응하기 위한 인터럽트의 개요

□ 인터럽트(Interrupt)의 정의

- 컴퓨터가 작업을 수행하던 도중 예기치 못한 특수한 상황이 발생하여 작업을 중단하고, 그 특수한 상황을 먼저 처리한 후, 원래의 작업으로 되돌아가 나머지 작업을 계속 수행하게 되는 일련의 과정

□ 인터럽트 필요성

필요성	내용
시스템 효율성 개선	- 고속 CPU, 저속 Device간 처리 효율 향상
	- 다중 입출력 장치에 대한 우선순위
시스템 안정성 개선	- 정전과 같은 비정상 상태 발생시, 수행 결과 저장 및 복구 수행

인터럽트(Interrupt)

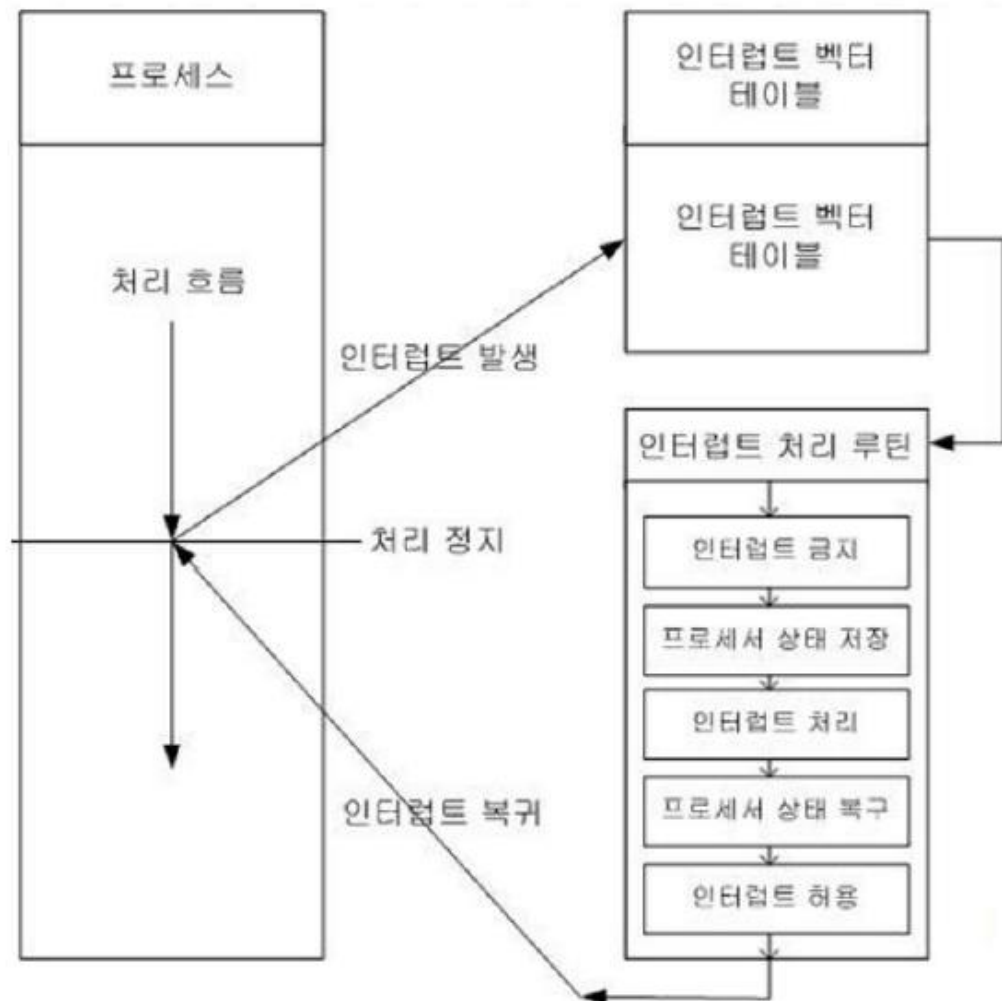
□ 인터럽트 관련 주요 개념

구분	내용
인터럽트 서비스 루틴(ISR)	- 모든 서비스 루틴들의 주소를 가지고 있는 인터럽트 벡터를 통해, 인터럽트 서비스 루틴으로 제어 신호를 전송
트랩(trap)	- 오류나 사용자 요청에 의해 소프트웨어가 발생시킨 인터럽트
인터럽트 중첩(nesting)	- 어떤 인터럽트를 처리하고 있는 중에, 새로운 인터럽트를 중첩하여 받을 수도 있고, 아닐 수도 있음

인터럽트(Interrupt)

2. 인터럽트 처리

□ 인터럽트 처리 개념도



- 1) 인터럽트 벡터 테이블의 해당 주소로 PC 값 변경(분기)
 - 외부에서 인터럽트가 발생한 경우, IRQ에 해당하는 주소로 PC 값 변경(분기) (예. 0x20 번지)
- 2) 인터럽트 벡터 테이블의 해당 주소에서, 해당 인터럽트 핸들러로 분기
 - 이미 0x20 번지에는, 해당 인터럽트 핸들러(여기에서는 IRQ 핸들러)로 분기하는 명령이 있음
- 3) 인터럽트 핸들러에서는, 현재 상태(context)를 저장하고, 해당하는 ISR을 찾아서 분기
 - IRQ 핸들러에서는 인터럽트 컨트롤러의 상태 레지스터를 확인하여, 해당하는 ISR을 결정함
예) Timer와 DMA의 인터럽트가 동시에 발생한 경우, 우선순위가 높은 장치의 ISR로 결정됨
- 4) ISR에서는 실제 해당 인터럽트에 대한 처리를 담당함, 종료 후 인터럽트 핸들러로 복귀
- 5) 인터럽트 핸들러로 돌아오면, 원래 상태(context)를 복원(restore)하게 됨.
- 6) 인터럽트 핸들러 종료 후 리턴하게 되면, 원래 수행되던 프로세스가 이어서 수행됨.
 - 이 과정에서, 인터럽트의 마스킹(masking), 수행 불가/가능(disable/enable) 등도 설정됨.

CPU 스케줄링

1. CPU의 효율적인 사용 방법, CPU(프로세스) 스케줄링의 개요

□ CPU 스케줄링(CPU Scheduling)의 의미

- 프로세스 작업 수행을 위해 언제, 어느 프로세스에 CPU를 배당할지를 결정하는 작업
- Multi-Processor 환경하에서 Processor 간의 우선순위를 저장함으로써 CPU 활용을 극대화하기 위한 방법

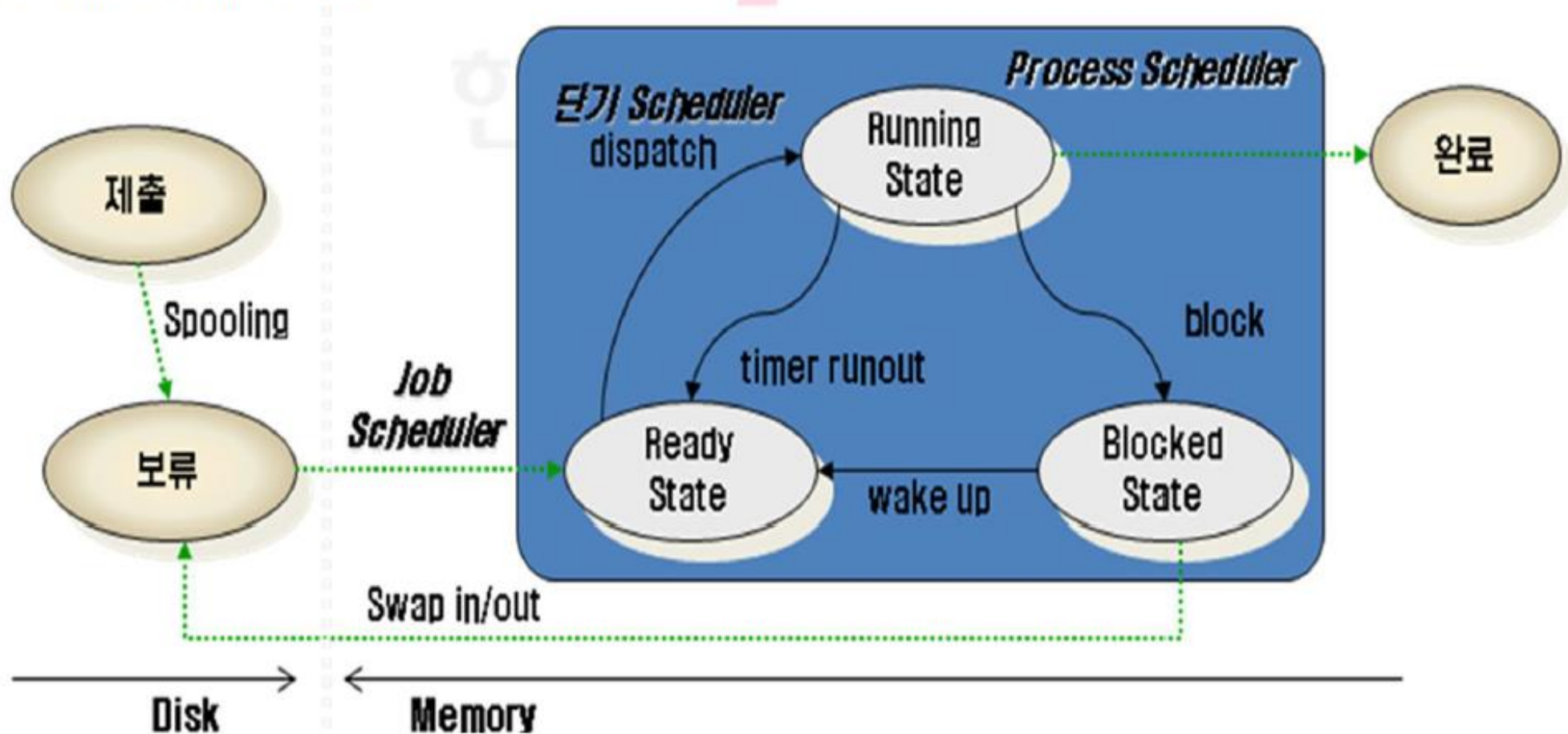
□ CPU 스케줄링 목표

요건	설명
처리능력 (Throughput) 최대화	- 주어진 시간에 최대한 많은 작업 처리 (처리된 프로세스 수/시간)
반환(경과)시간 (Turnaround time) 최소화	- 전체 작업 수행 시간을 최소화 (system in → system out 시간)
대기시간 (waiting time) 최소화	- ready queue에서 기다리는 시간 최소화
응답시간 (Response time) 최소화	- 대화식 시스템에서 첫 응답까지의 시간 최소화
CPU 이용률(Utilization) 극대화	- CPU를 정해진 시간 내 100% 가동

CPU 스케줄링

2. Process 상태 전이도와 Scheduler의 종류 및 역할

□ 프로세스 상태 전이도



CPU 스케줄링

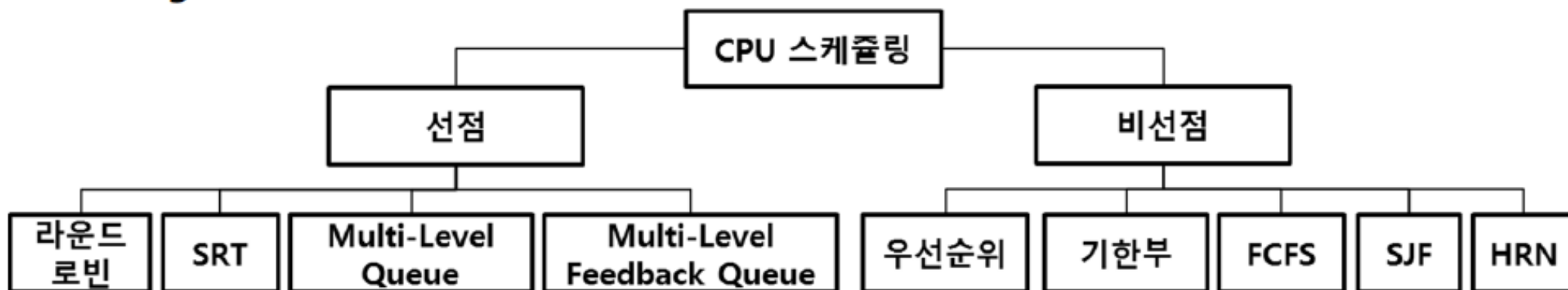
□ Scheduler의 종류 및 역할

종류	역할
Scheduling Queue	- 주 기억 장치의 할당을 기다림 (보류 상태, 디스크에 위치)
장기(Job) Scheduler	- 프로세스 선택, 주기억장치 할당 (보류 준비)
중기(Process) Scheduler	- 프로세스 수에 따라 디스크로 보냄 (대기 보류)
단기 Scheduler	- 실행 준비된 프로세스에 CPU 할당 (준비 실행)

CPU 스케줄링

3. CPU scheduling 기법과 알고리즘

□ Scheduling 기법



CPU 스케줄링

구분	선점(Preemptive) 스케줄링	비선점(Non-preemptive) 스케줄링
개념	- 한 프로세스가 CPU를 차지하고 있을 때 우선순위가 높은 다른 프로세스가 현재 프로세스를 중지시키고 자신이 CPU를 점유 가능 - 높은 우선순위를 가진 프로세스들이 빠른 처리를 요구하는 시스템에서 유용	한 프로세스가 CPU를 할당 받으면 작업 종료 후 CPU 반환 시까지 다른 프로세스는 CPU 점유 불가
장점	- 비교적 빠른 응답 - 대화식 시분할 시스템에 적합	- 응답시간 예측이 용이 - 모든 프로세스에 대한 요구를 공정하게 처리
단점	높은 우선순위 프로세스들이 들어오는 경우 오버헤드를 초래	짧은 작업을 수행하는 프로세스가 긴 작업 종료 시까지 대기

CPU 스케줄링

□ 선점 스케줄링 알고리즘

알고리즘	처리방식
RR	- Round Robin - 대화식 사용자를 위한 시분할 시스템(Time Sharing System)을 위해 고안 - 준비 큐(FCFS)에 의해 보내진 각 프로세스는 같은 크기의 CPU 시간을 할당 받음 - 프로세스가 할당된 시간 내에 처리 완료 못하면 준비 큐 리스트의 가장 뒤로 보내지고 CPU는 대기중인 다음 프로세스로 넘어감 - 할당 시간이 가장 중요하며, 일반적으로 시간 할당량은 100 밀리 초에서 1, 2초 사이 값 - 할당 시간이 크면 FCFS 와 같게 되고, 작으면 문맥 교환이 자주 발생
SRT	- Short Remaining Time - 가장 짧은 시간이 소요된다고 판단되는 프로세스를 먼저 수행 - 남은 처리 시간이 더 짧다고 판단되는 프로세스가 준비 큐에 생기면 언제라도 프로세스가 선점됨 - 긴 작업은 SJF 보다 대기 시간이 길다

CPU 스케줄링

다단계 큐	- Multi-level Queue - 작업들을 여러 종류의 그룹의 분할
	- 여러 개의 큐를 이용 상위 단계 작업에 의해 하위 단계 작업이 선점 당함 - 준비 상태 큐를 여러 종류로 분할(작업 분류별 묶음) 하지만 다른 큐로 작업 이동 불가 - 각 큐는 자신만의 독자적인 스케줄링을 가짐
다단계 피드백 큐	- Multi-level Feedback Queue - 입출력 위주와 CPU 위주인 프로세스의 특성에 따라 큐마다 서로 다른 CPU Time Slice (Quantum)를 부여 - 새로운 프로세스는 높은 우선순위, 프로세스의 실행 시간이 길어질수록 점점 낮은 우선순위 큐로 이동 (맨 마지막 단계에서는 Round Robin 처리) - 하위단계일수록 할당 시간은 증가 (공평성 부여) - MLFQ는 MLQ의 Starvation방지 및 Process간 유연한 처리 가능

CPU 스케줄링

□ 비선점 스케줄링 알고리즘

알고리즘	처리방식
우선순위 스케줄링	- 각 프로세스에 우선순위가 주어지고 우선순위에 따라 CPU 할당 - 동일한 우선순위간은 FCFS 처리 - 우선순위 결정 : 관리자에 의한 결정, 자원요구량에 의한 결정, CPU처리 시간에 의한 결정, 시스템에서 보낸 시간에 의한 결정 등 사용 - 우선순위가 높은 작업이 계속적으로 들어오게 될 우선순위가 낮은 프로세스는 Starvation 발생 → Aging 기법으로 해결 가능
기한부 스케줄링	- 작업들이 명시된 시간이나 기한 내에 완료되도록 계획 - 사용자는 사전에 작업이 요구하는 정확한 자원을 제시 - 작업시간이나 상황 등 정보를 미리 예측하기가 어려움
FCFS	- First Come First Service - 프로세스가 대기 큐(준비 큐)에 도착한 순서에 따라 CPU 할당 - 가장 간단한 스케줄링 알고리즘으로 FIFO(First Input First Out)알고리즘이라고 함 - Convoy Effect 발생 가능(Burst time이 긴 프로세스가 CPU 독점) - 단독적 사용이 거의 없으며, 다른 스케줄링 알고리즘에 보조적으로 사용 (우선순위 스케줄링, RR 스케줄링 등)

CPU 스케줄링

SJF	- Shortest Job First - 준비 큐 내의 작업 중 수행시간이 가장 짧다고 판단되는 것을 먼저 수행
	- 각 프로세스에서 CPU 버스트 길이를 비교하여 CPU 가 이용 가능해지면 가장 작은 CPU 버스트를 가진 프로세스를 할당 - 주어진 프로세스 집합에 대해서 평균 대기 시간이 최소가 되는 최적 알고리즘 - CPU 요구 시간이 긴 작업과 짧은 작업 간의 불평등 심하여, CPU 요구 시간이 긴 프로세스는 Starvation 발생 → HRN 사용
HRN	- Highest Response Ratio Next - $\text{Response Ratio} = (\text{대기시간} + \text{서비스 시간}) / \text{서비스 시간}$ - 대기중인 프로세스 중 현재 Response Ratio 가 가장 높은 것을 선택 - 대기중인 프로세스 중 현재 Response Ratio 가 가장 높은 것을 선택 - SJF의 약점을 보완한 기법으로 긴 작업과 짧은 작업 간의 불평등을 완화

CPU 스케줄링

CPU 스케줄링 기법에 관련하여 아래 물음에 답하시오.

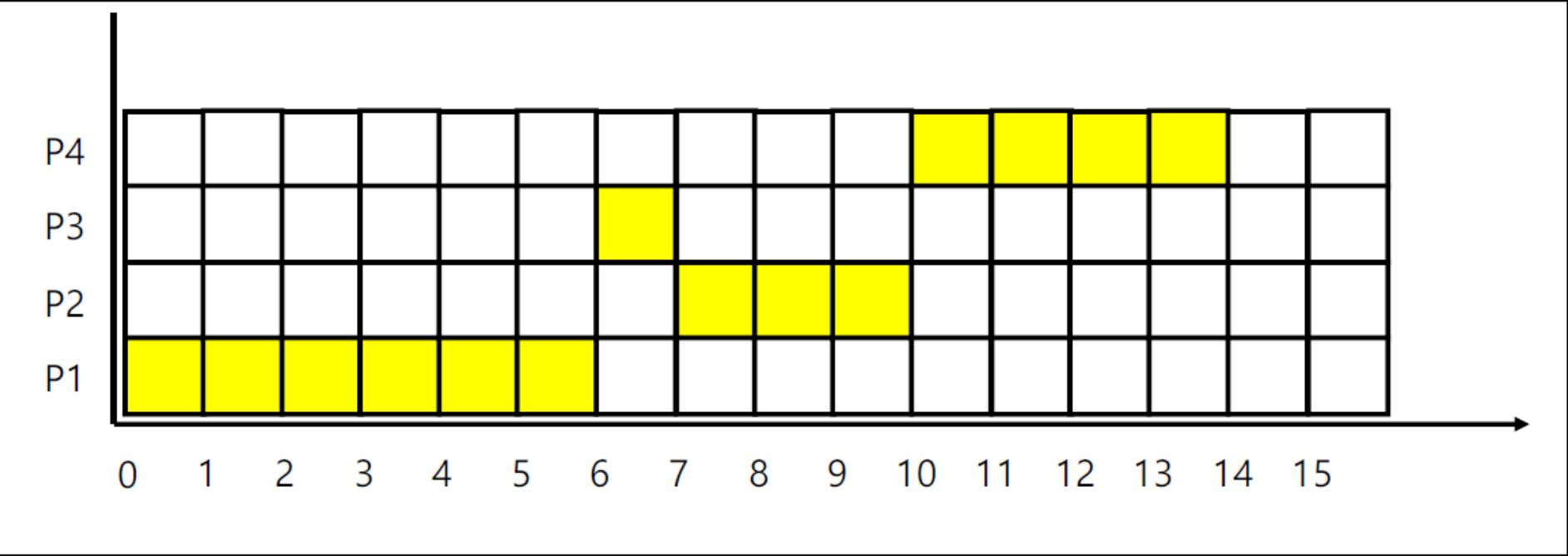
가. SJF 기법과 SRT 기법에 대해 설명하시오.

나. 주어진 사례에 SJF 와 SRT 의 평균대기시간을 구하시오

프로세스	도착시간	실행시간
P1	0	6
P2	1	3
P3	2	1
P4	3	4

CPU 스케줄링

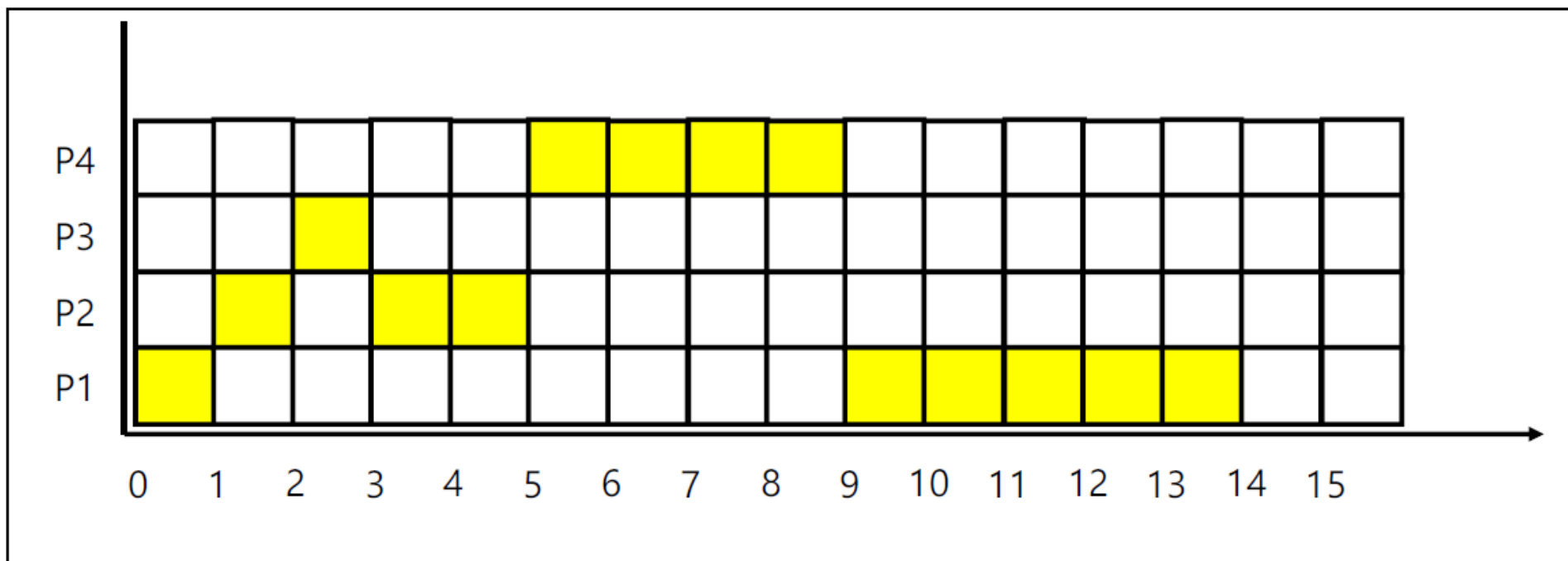
가. SJF의 프로세스 자원 점유도 및 평균 대기시간



구분	P1	P2	P3	P4	평균 대기시간
도착 시간	0	1	2	3	N/A
서비스 시간	6	3	1	4	N/A
종료 시간	6	10	7	14	N/A
반환 시간	6	9	5	11	N/A
대기 시간	0	6	4	7	4.25

CPU 스케줄링

나. SRT의 자원 점유도 및 평균 대기시간



구분	P1	P2	P3	P4	평균 대기시간
도착 시간	0	1	2	3	N/A
서비스 시간	6	3	1	4	N/A
종료 시간	14	5	3	9	N/A
반환 시간	14	4	1	6	N/A
대기 시간	8	1	0	2	2.75

감사합니다!

컴퓨터구조 및 운영체제(CAOS)

3. 운영체제(Operation System) part2

(7)	경쟁 조건(Race Condition)	88
(8)	세마포어(Semaphore).....	91
(9)	스핀락(Spinlock)	96
(10)	교착상태(Deadlock)	98
(11)	wait-die 와 wound-wait	102
(12)	뱅크스(Banker's) 알고리즘	103
(13)	지역성(Locality)	105
(14)	쓰레싱(Thrashing).....	107

경쟁 조건(Race Condition)

1. 한정된 자원을 획득하려 프로세스가 경합하는 현상, 경쟁 조건의 개요

□ 경쟁 조건(Race Condition)의 정의

- Multi-processing 시스템 환경에서 특정 자원에 대하여 여러 프로세스 가동 시에 자원(Resource)을 획득하려고 경쟁하는 상황

□ 경쟁 조건의 발생원인

- 시스템의 중요 자원의 무결성(integrity)를 보증하기 위하여 중요 자원을 접근(access)하는 경우에 자원을 locking하고, 사용 후에 release시킴
- 한 프로세스가 자원을 lock하고 있는 경우, 이 자원을 얻으려는 다른 프로세스는 wait 해야 함.
- 자원을 lock한 프로세스가 내부적인 문제로 release가 길어지는 경우에 프로세스 대기 행렬이 길어지고, 교착상태(deadlock)의 발생가능성이 커지고, 시스템의 성능(performance)에 심각한 영향

경쟁 조건(Race Condition)

2. 다중 처리기 시스템과 단일 처리기 시스템에서의 경쟁조건

□ 단일 처리기 시스템에서 경쟁 조건이 일어나는 시나리오

- 단일 처리기 시스템에서도 시분할(Time Sharing)을 통하여 여러 프로세스들이 동시에 실행
- time slice를 통하여 multi-processing으로 처리되는 시스템 환경에서 프로세스간의 자원의 lock을 통한 점유와 대기가 발생됨
- 특정 프로세스가 자원을 lock하고 sleep이나 I/O wait등으로 빠지는 경우에 다른 프로세스들이 무한 대기에 빠질 가능성이 있음
- 대기중인 프로세스 중에서 sleep & wait 등을 통하여 다음에 자원을 lock할 수 있는 기회가 생김(경쟁이 발생함)

단일처리기 시스템(Single-Processor Systems) : 하나의 범용 CPU를 가진 시스템

경쟁 조건(Race Condition)

□ 다중처리기 시스템에서 경쟁 조건이 일어나는 시나리오

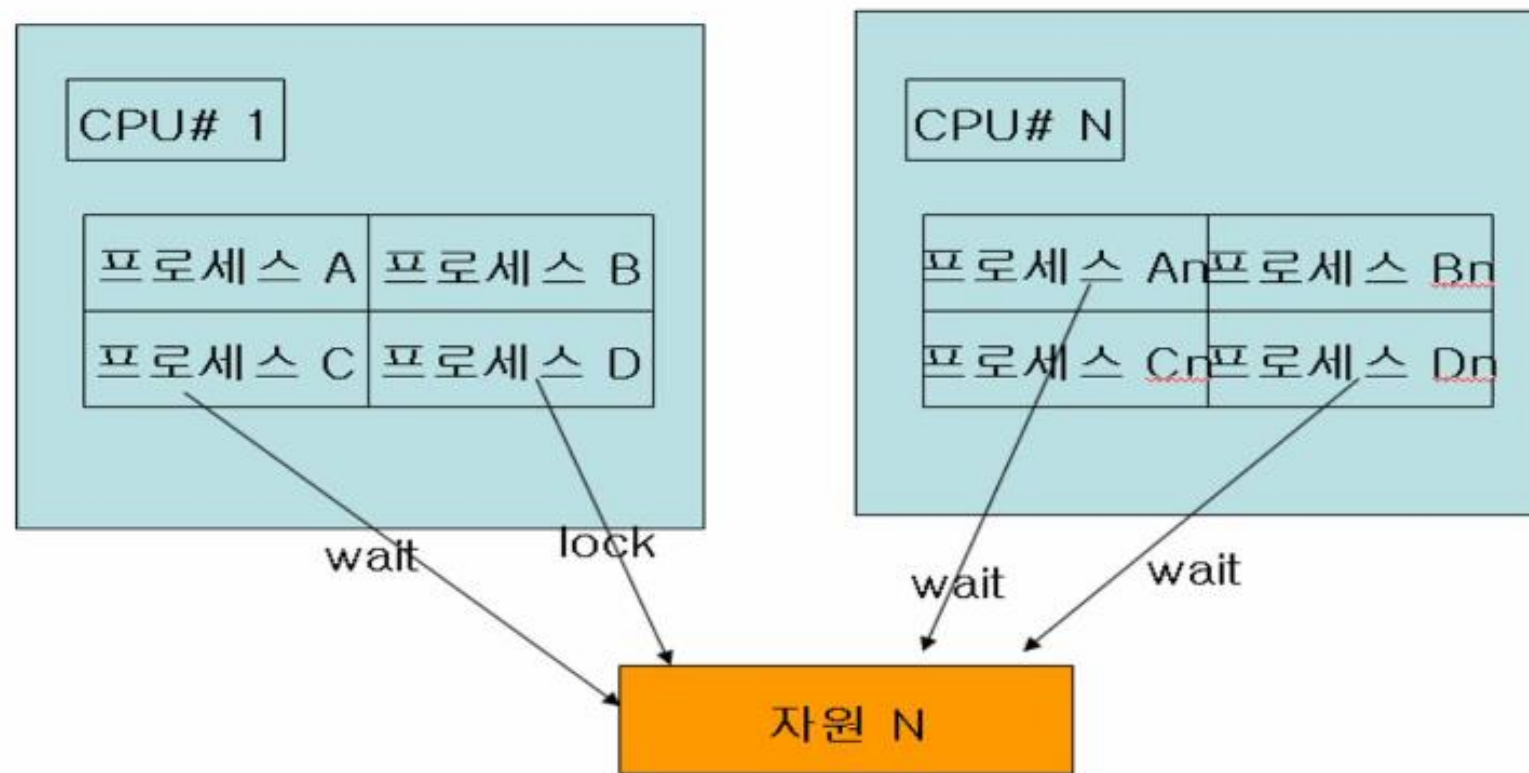
1) 다중처리기 시스템의 구성

- multi-processor 구성으로 여러 processor(CPU)에서 동시에 여러 프로세스가 실행됨
- 어느 processor에서 실행되는지 관계없이 시스템 자원에 대한 lock과 release는 단일 처리기 시스템 보다 훨씬 많은 경쟁 조건이 발생됨

2) 다중처리기 시스템에서의 경쟁 조건이 일어나는 시나리오

- 특정 CPU에서 실행되는 프로세스 D가 특정 자원을 lock하면, 실행되는 CPU에 관계없이 다른 프로세스들은 대기(wait)해야 함
- 결국 여러 CPU 상에서 실행되는 많은 프로세스 간에 경쟁이 발생함

경쟁 조건(Race Condition)



경쟁 조건(Race Condition)

3. 다중 처리기 시스템과 단일 처리기 시스템에서의 경쟁 조건의 해결방안

□ 단일 처리기 시스템에서 경쟁 조건의 해결방안

- 단일 처리기 시스템에서는 자원의 독점과 무한 대기를 방지하기 위하여 세마 포어를 사용함
- 공유 메모리 혹은 공유 파일의 사용시 다른 프로세스가 사용하지 못하도록 배제하는 상호 배제(mutual exclusion)의 최소화
- 최소 임계 영역(Critical Section)의 access를 통하여, 한 프로세스가 한 자원을 오랜 시간 동안 독점하지 못하게 함

□ 다중처리기 시스템에서 경쟁 조건의 해결방안

- 다중처리기 시스템에서는 위의 단일 처리기 시스템에서의 경쟁 조건 해결방안을 기본적으로 사용함
- 스핀락(spinlock)¹ timeout 개념을 사용하여, 특정 프로세스가 특정 자원을 lock하고 일정시간(60초) 동안 release하지 않는 경우에 spinlock timeout을 통하여 문제를 해결함
- 위의 경우에 극단적으로는 system panic²을 call하여 lock을 release하지 않은 프로세스나 CPU에 대한 자세한 조사를 통하여, 자원의 독점이 발생하는 현상을 수정할 수 있음

경쟁 조건(Race Condition)

4. Race Condition 억제 방법 및 예방 방법

□ Race Condition 억제 방법

- 임시 파일을 만들지 않음
- Race Condition은 프로세스가 실행 중 임시파일을 쓰는 경우에 발생 메모리에서 모든 작업을 수행하고 임시파일을 사용 않으면 Race Condition은 발생하지 않음
- 자원에 대한 경쟁상황을 만들지 않으며 각자 할당된 메모리를 사용하게 함
- Create와 Open을 확실히 구분하여 사용함
- Race Condition은 두 시스템 사이의 콜 사이에 정상적으로 처리될 것이란 생각 때문에 발생
- 즉, 파일이 존재하지 않는 경우에만 파일을 생성하도록 프로그래밍 하여야 함
- 수시로 링크 정보를 검사하여 비정상적인 링크를 제거함
- 슈퍼유저 소유의 중요한 파일에 대한 링크는 원천적으로 사용하지 못하게 통제

경쟁 조건(Race Condition)

□ Race Condition 에 대한 예방 방법

1) 유닉스의 버그에 대한 분석 필요

- Race Condition은 결국 유닉스의 버그로써 발생원인에 대한 이해가 중요함
- 유닉스 관리자는 보안 지침에 따라 보안 취약점에 대해 정기 점검을 실시
- 개발자의 유닉스 버그에 대한 이해도 강조 및 교육 실시가 중요함

2) 충분한 자원 산정: 임시파일 사용을 가능한 억제하게 충분한 메모리 자원 확보

3) 정기 보안 점검 및 취약성 분석 강화: 취약성 Tool 을 이용하여 심볼릭 링크, 임시파일에 대한 사전점검 강화시킴

경쟁 조건(Race Condition)

5. 시스템 성능향상을 위한 경쟁 조건의 해결의 trade off 및 고려사항

□ 경쟁 조건 해결의 trade off

- 경쟁 조건의 해결을 위하여 특정 프로세스가 특정 자원을 무한정 또는 일정시간 이상 독점할 수 없게 구현하지만, 이는 자원에 대한 잦은 lock과 release의 반복으로 프로세스가 충분하게 시스템 자원을 사용하지 못하고, 지속적인 wait-lock이 반복되어 오히려 전체적인 시스템 성능이 떨어질 수 있음
- 자원에 대한 획득-lock-release-wait 등의 상호관계를 고려한 적정선의 자원 독점 사용 시간과 강제종료 조건을 적절하게 배합해야 함

□ 고려사항

- 경쟁 조건을 통하여 여러 프로세스간에 deadlock이 발생할 수 있음
- 한 프로세스의 자원 독점이 길어지면 길어질수록, deadlock의 발생 가능성이 더욱 커지게 됨

세마포어(Semaphore)

1. 비동기 병행 프로세스의 개요

□ 비동기 프로세스의 정의

- 멀티 프로세싱 환경 하에서 하나의 자원을 다수의 프로세스가 비동기적으로 공유하면서 사용하는 환경

□ 비동기 병행 프로세스 관련 중요 키워드

- 경쟁 조건(Race Condition): 2개 이상의 프로세스 공유 메모리에서 읽기/쓰기를 수행, 실행 순서에 따라 결과값이 달라질 수 있는 상황
- 임계 영역(Critical Section): 위험 지역 각 프로세스가 공유 데이터를 액세스하는 프로그램 코드 부분
- 상호 배제(Mutual Exclusion): 한 프로세스가 그 데이터를 액세스할 때, 다른 프로세스들은 해당 데이터를 사용하지 못하도록 하는 운영 기법

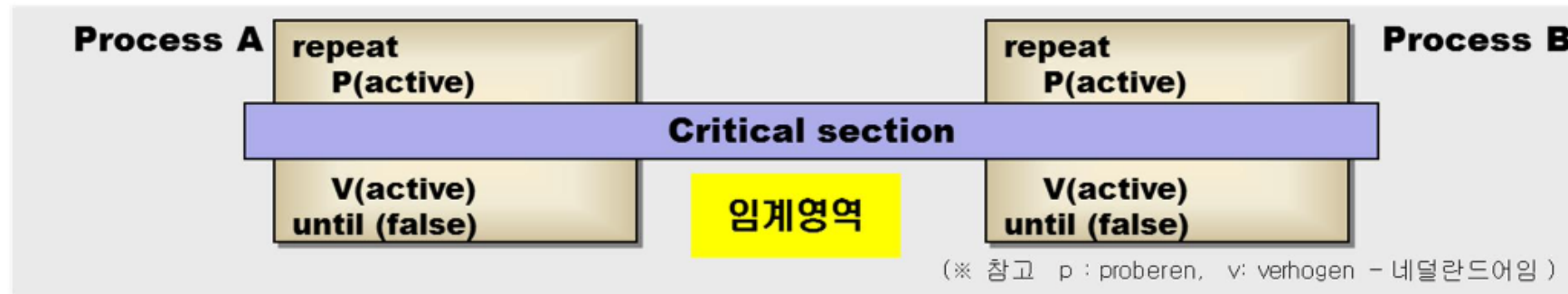
세마포어(Semaphore)

2. 병행처리를 위한 프로세스 동기화 기법, 세마포어의 개요

□ 세마포어(Semaphore) 의 정의

- 운영체계의 자원을 경쟁적으로 사용하는 다중 프로세스에서, 행동을 조정하거나 또는 동기화 시키는 기술
- 멀티프로그래밍 환경에서 정수 변수를 사용하여 공유 자원(resource)에 대한 접근을 제한하는 기법
- 한 순간에 하나의 작업에 의하여 접근되어야 하는 긴급 영역과 상호 배제 원리를 지키기 위한 방법

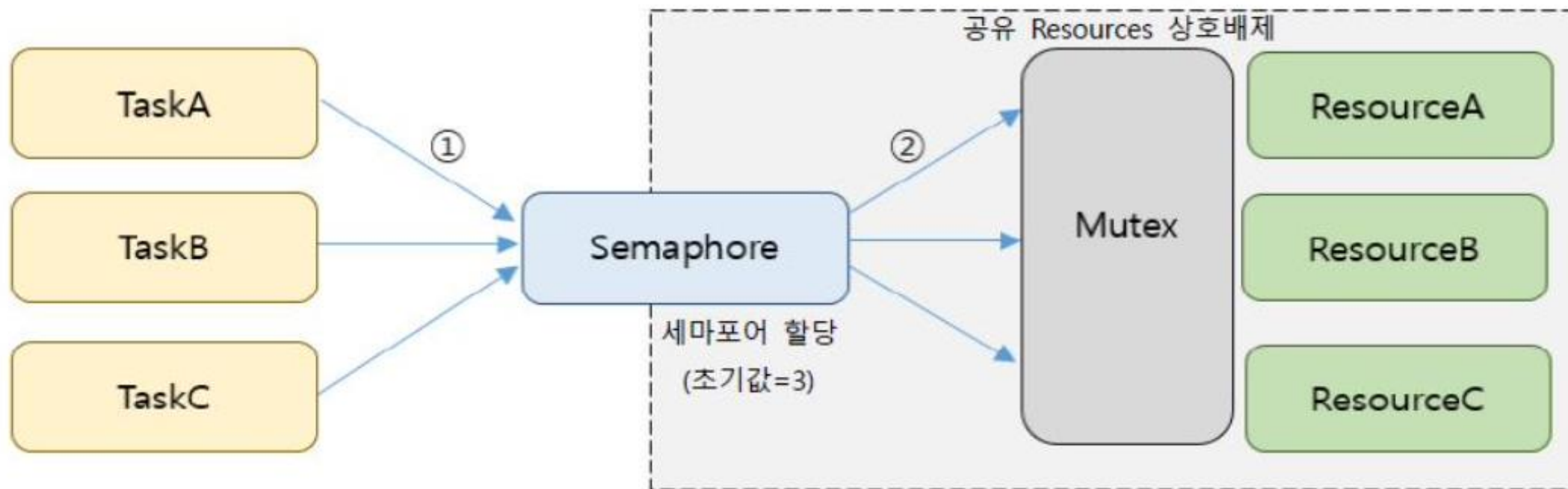
□ 세마포어를 이용한 상호배제 기법



- active가 1이면 Critical section을 실행중인 프로세스가 없음을 의미
- active가 0이면 Critical section을 실행중인 프로세스가 있음을 의미

세마포어(Semaphore)

□ 세마포어를 이용한 리소스 공유 메커니즘



- N개의 Task와 M개의 Resource를 공유할 때, 세마포어(계수형) 초기값을 N개로 설정

세마포어(Semaphore)

□ 세마포어 작동원리

- 일종의 정수형 변수인 Semaphore S에 하나의 대기큐를 할당하고 초기화

P연산 - Wait

```

procedure P(S)      --> 최초 S값은 1임
  while S=0 do wait  --> S가 0면 1이 될 때까지 기다려야 함
    S := S-1         --> S를 0로 만들어 다른 프로세스가 들어 오지 못하도록 함
end P
  
```

수행

V연산 - Signal

```

procedure V(S)      --> 현재상태는 S가 0임
  S := S+1           --> S를 1로 원 위치시켜 해제하는 과정으로 다른 프로세스가 들어 올 수 있음
end V
  
```

완료

세마포어(Semaphore)

□ 세마포어 유형

유형	내용
Binary Semaphore	- Semaphore 변수가 0과 1 두 종류의 값만 갖는 경우 - 상호배제나 프로세스 동기화 목적으로 사용
Counting Semaphore	- Semaphore 변수가 0 이상의 모든 정수 값을 가질 수 있는 경우 - 초기에 동시에 진입 할 수 있는 프로세스의 개수를 정의 가능 - 생산자와 소비자 문제 해결

세마포어(Semaphore)

3. Event Count/Sequence 를 이용한 프로세스 동기화

□ Event Count/Sequence 를 이용한 프로세스 동기화 개념

- 은행의 서비스 순서가 기재된 티켓과 같은 형태로 프로세스들 간의 동기화를 수행하는 형태

□ Semaphore 와 Event Count/Sequence 비교

구분	특징 및 설명
Semaphore	- 긴급 영역과 상호배제 원리를 지키기 위한 방법 - 무기한 연기의 문제점 - Semaphore 큐에서 대기중의 프로세스들의 Wake up 순서는 결정되지 않음
Event Count/Sequence	- Busy Waiting 문제 해결 - FIFO 스케줄링에 의해서 Block 된 순서대로 Wake up 됨 - 무기한 연기의 문제점 해결

- Busy Waiting: 하나의 프로세스가 CPU Scheduling에 들어가면 Critical Section에 들어가려는 다른 프로세스가 루프를 돌며 대기하는 프로세스가 많을 때를 말함.

세마포어(Semaphore)

4. 상호배제 매커니즘 간 비교

구분	Critical Section	Mutex	Semaphore
동기화 객체	유저모드	커널모드	
특징	쓰레드 동기화, 고속	프로세스/쓰레드 모두사용가능, 저속	
Wait	EnterCriticalSection	Wait	P 연산
Signal	LeaveCriticalSection	Mutex	V 연산

5. 세마포어와 뮤텍스¹ 비교

구분	세마포어(Semaphore)	뮤텍스(Mutex)
원리	화장실이 n 칸(접근할 수 있는 최대 허용치 만큼 동시에 사용자 접근을 할 수 있게 함), 세마포어 카운트가 0 이면 대기	화장실에 들어가기 위한 열쇠를 한 사람이 가지고 있다면, 그 사람 만이 들어갈 수 있다. 대기열(큐)에 기다리게 됨. → 세마포어의 일종

세마포어(Semaphore)

목적	- Dead Lock을 피하기 위한 기술	- Critical Section을 가진 쓰레드들이 running time이 서로 겹치지 않게, 단독으로 실행하게 하는 기술
특징	- 공유 리소스에 접근할 수 있는 최대 허용치 만큼 동시에 사용자 접근을 할 수 있게 하는 기술 - 동기화 대상이 여러 개일 경우에 사용	- 뮤텝스는 한 번에 하나의 쓰레드 만이 실행되도록 하는 재 입장할 수 있는 코드 섹션 직렬화된 접근이 가능하게 할 때 사용 - 첫 번째 쓰레드가 해당 세션을 빠져나올 때까지 기다림

스핀락(Spinlock)

1. Busy-Waiting 대기, Spinlock의 개요.

□ 교착상태(Deadlock)의 정의

- 임계 구역에 진입이 불가능할 때 진입이 가능할 때까지 루프를 돌면서 재시도하는 방식으로 구현된 락
- [특징] 문맥교환 최소화, 바쁜 대기(Busy Waiting) 수행, 짧은 임계영역 유용, SMP 시스템에 유용

2. Spinlock 프로세스 및 함수

□ Spinlock 의 프로세스



스핀락(Spinlock)

□ Spinlock 의 API

API	설명
spin_lock_init()	- runtime으로 spinlock 초기화
spin_lock()	- spinlock 설정
spin_unlock()	- spinlock 해제
spin_lock_irqsave()	- interrupt disable 시킨 후 spinlock 설정, cpu flag 레지스터 보관
spin_lock_irq()	- interrupt disable 시킨 후 spinlock 설정
spin_lock_bh()	- softirq를 disable 시킨 후 spinlock 설정
spin_unlock_irqrestore()	- spin_lock_irqsave에서 저장한 cpu 레지스터 복구
spin_unlock_irq()	- spin_lock_irq으로 설정된 spinlock 해제
spin_unlock_bh()	- spin_lock_bg으로 설정된 spinlock 해제
spin_lock_init()	- runtime으로 spinlock 초기화

- 스핀락 외에 대표적인 동기화 기법으로 세마포어가 존재 , 스핀락은 Busy Waiting을 통해 짧은 Critical Section 에 유용, 세마포어는 긴 Critical Section에 유용한 기법

스핀락(Spinlock)

3. Spinlock 과 세마포어(Semaphore) 비교

구분	Spinlock	Semaphore
동작	- A task Critical Section 접근 - B task가 공유자원 사용일 경우 <u>A task는 busy waiting</u> - B task Lock 해제 - A task 공유자원 접근	- A task Critical Section 접근 - B task가 공유자원 사용일 경우 <u>A task를 Wait Queue에 넣고 Sleep상태로 변경</u> - B task Lock 해제 - 스케줄러에 의해 A task 깨우기 - A task 공유자원 접근
사용현황	- 짧은 Critical Section (짧은 시간의 lock)	- 긴 Critical Section (오랜 시간의 lock)
인터럽트	- 사용가능	- 사용불가 (인터럽트 Context Switching에서 Sleep상태로 들어가면 안됨)
장점	- Context Switching 줄여 효율성 증가	- 불필요한 CPU부하 발생하지 않음
단점	- 비효율적으로 사용시 Busy Waiting CPU부하	- Context Switching 발생하여 부하 발생

감사합니다!

컴퓨터구조 및 운영체제(CAOS)

3. 운영체제(Operation System) part3

(7)	경쟁 조건(Race Condition)	88
(8)	세마포어(Semaphore).....	91
(9)	스핀락(Spinlock)	96
(10)	교착상태(Deadlock)	98
(11)	wait-die 와 wound-wait	102
(12)	뱅크스(Banker's) 알고리즘	103
(13)	지역성(Locality)	105
(14)	쓰레싱(Thrashing).....	107

교착상태(Deadlock)

1. 다중 프로세싱 환경에서 무한자원대기, 교착상태의 개요.

□ 교착상태(Deadlock)의 정의

- 다중 프로그램 환경에서 두 개 이상의 프로세스가 아무리 기다려도 자원을 사용할 수 없는 무한 대기상태
- Multi Processing 환경에서 다수의 프로세스가 특정 자원의 할당을 무한정 기다리고 있는 상태
- 교착상태에 있는 프로세스 들은 결코 실행을 끝낼 수 없으며, 시스템 자원이 묶여 있어서 다른 작업을 시작하는 것도 불가능 함.

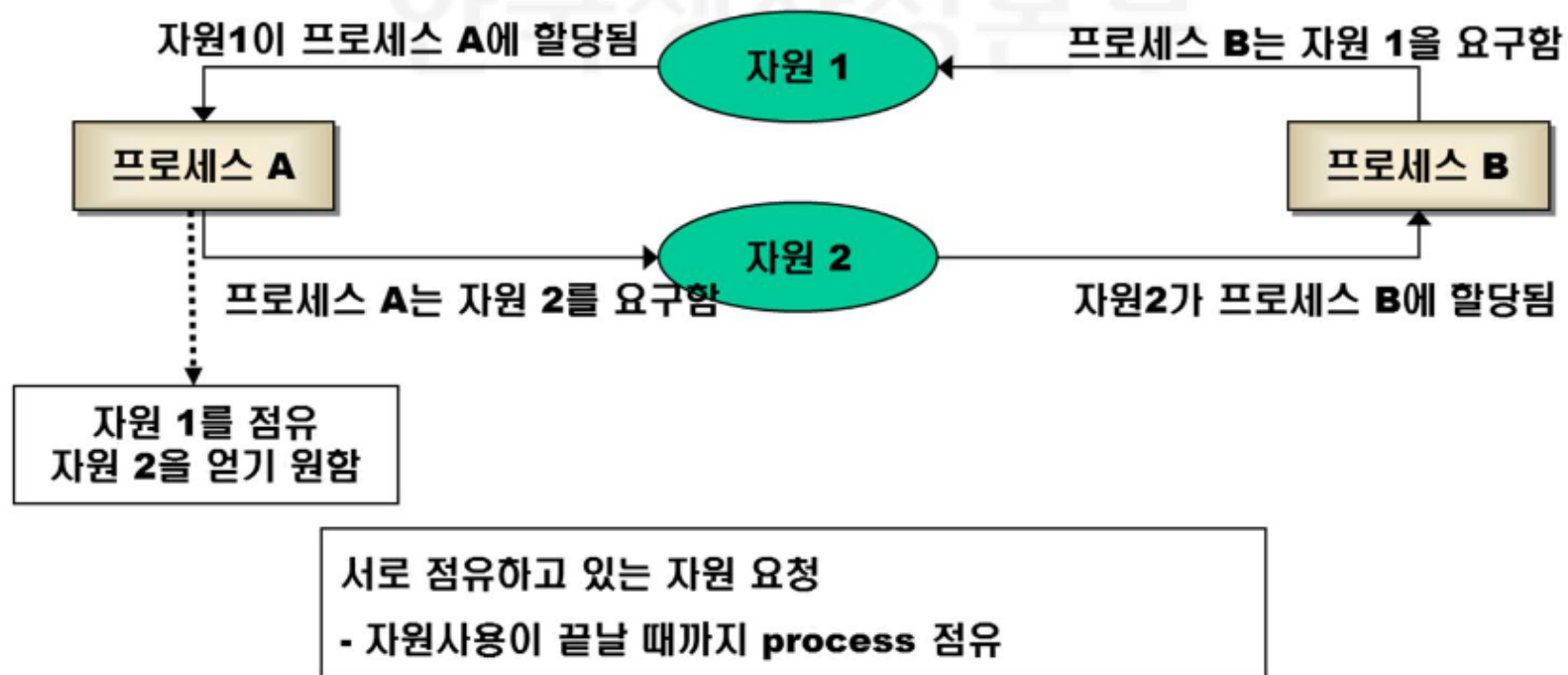
□ 교착상태와 무한대기 비교

구분	Deadlock(교착상태)	Starvation(무한대기)
정의	- 다수의 프로세스가 아무 일도 못하고 특정 사건을 기다리며 무한대기	- 특정 프로세스가 자원을 할당 받기 위하여 무한정 대기하는 상태
발생원인	- 상호배제, 점유와 대기, 비선점, 환형대기	- 자원의 편중된 분배정책
해결방안	- 예방, 회피, 발견, 회복	- Aging 기법(프로세스 우선순위 큐 이동)

교착상태(Deadlock)

2. 교착상태 개념도 및 발생 조건

□ 교착상태 개념도



교착상태(Deadlock)

□ 교착상태의 발생 원인

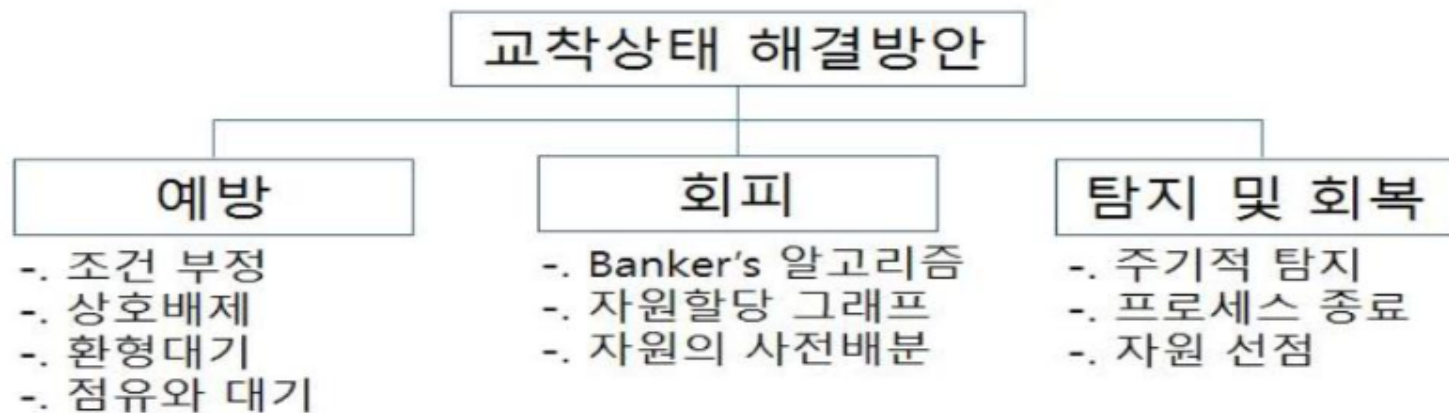
- 교착상태는 한 시스템에서 다음 네 가지 조건이 **동시에 성립될 때 발생**

원인	내용
상호배제 (Mutual Exclusion)	- 프로세스들이 자원을 배타적으로 점유하여 다른 프로세스가 그 자원을 사용하지 못함
점유와 대기 (Block & wait)	- 프로세스가 어떤 자원을 할당 받아 점유하고 있으면서 다른 자원 요구
비선점 (Non preemption)	- 프로세스에 할당된 자원은 사용이 끝날 때 까지 강제로 빼앗을 수 없으며, 점유하고 있는 프로세스 자신만이 해제 가능
환형대기 (Circular wait)	- 프로세스간 자원 요구가 하나의 원형을 구성

교착상태(Deadlock)

4. 교착상태 해결방안

□ 교착상태 해결방안



해결방안	내용
예방	- 교착상태 4가지 필수조건을 부정하여 사전에 예방
회피	- 사용자가 자원이 어떻게 요청될 지에 대한 추가 정보를 제공하도록 요청 - 자원 할당 그래프를 바탕으로 프로세스가 일정 기간 내에 안정적으로 종료될 수 있다는 확신이 있는 경우 자원 할당, 그 외 요구는 거절
탐지	- 교착상태 발생을 허용하고 발생시 원인을 규명하여 해결
회복	- 교착상태 발견 후 환형 대기를 배제시키거나 자원을 중단시켜 해결

교착상태(Deadlock)

□ 교착상태 예방(Prevention)

구분	내용	단점
상호배제 조건의 부정	- 공유할 수 없는 자원을 사용할 때 성립	
점유와 대기조건의 부정	- 프로세스가 자원을 요청할 때는, 다른 자원들을 점유하지 않을 것을 보장함	- 자원낭비 및 비용증가 - 자원공유 불가능 - Starvation 발생가능
비선점 조건의 부정	- 어떤 자원을 가진 프로세스가 더 이상 자원할당 요구가 받아지지 않으면 점유자원을 반납	- 비용증가 - Starvation 발생가능 - 일부 자원은 안전하게 선점 불가
환형대기 조건의 부정	- 모든 프로세스에게 각 자원의 유형별로 할당순서를 부여하는 방법	- 새로운 자원 추가 시 재구성 필요

교착상태(Deadlock)

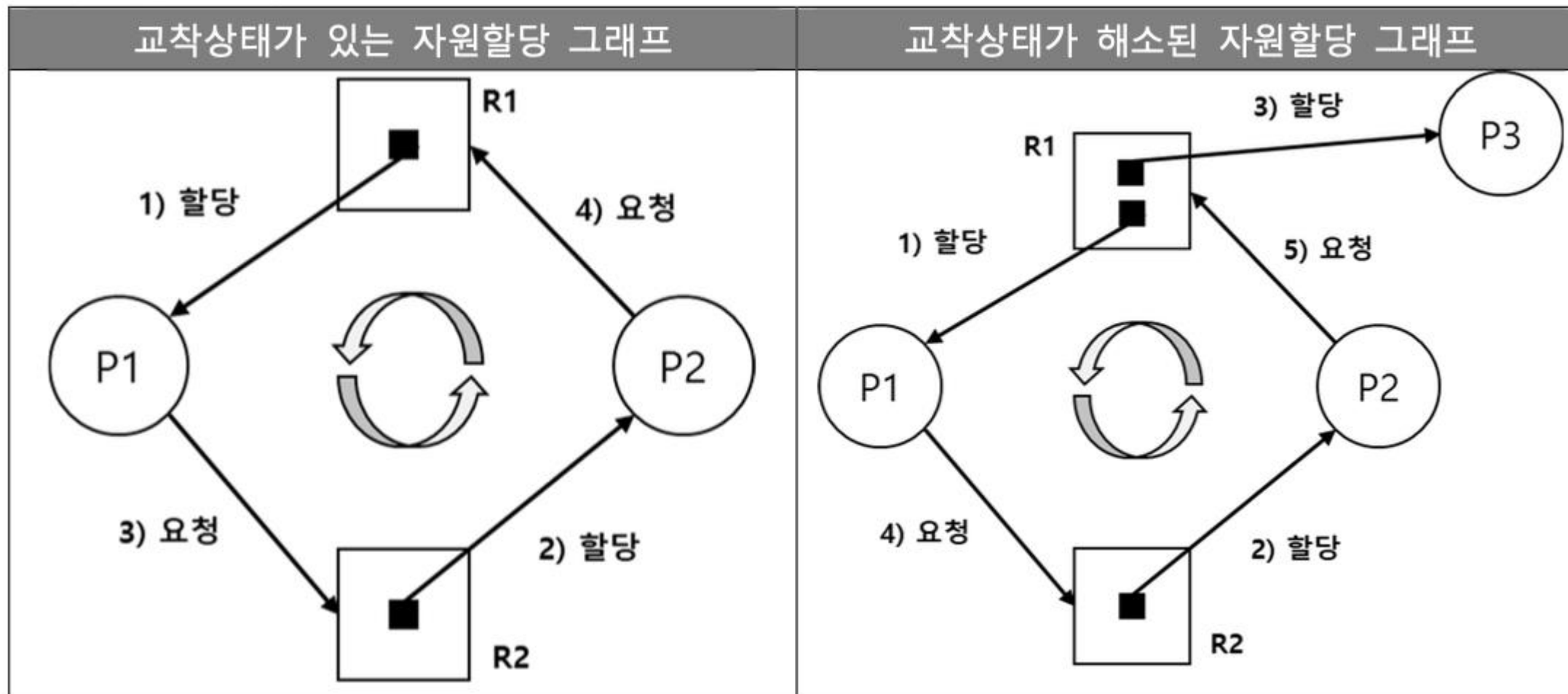
□ 교착상태 회피(Avoidance)

- 사용자가 자원이 어떻게 요청될 지에 대한 추가 정보를 제공하도록 요청함
- 자원할당 그래프를 바탕으로 프로세스가 일정 기간 내에 안정적으로 종료될 수 있다는 확신이 있을 경우에 자원을 할당하며 그 외 요구는 만족될 때까지 계속 거절함

알고리즘	설명
자원할당 그래프 알고리즘	- 종류별 자원이 한 개인 경우 자원 종류마다 한 개의 자원 요청가능 - 자원할당 그래프+예약간선(claim edge) - 프로세스가 자원 요청하면 예약간선은 요청간선으로 전환 - 이 요청간선을 할당간선으로 변환해도 cycle이 없을 때만 할당
Banker's 알고리즘	- 종류별 자원이 여러 개인 경우 프로세스의 최대자원 요구량, 현재 할당된 자원의 수 등을 이용하여 프로세스의 자원 할당이 안전한지 여부를 확인 후 할당

교착상태(Deadlock)

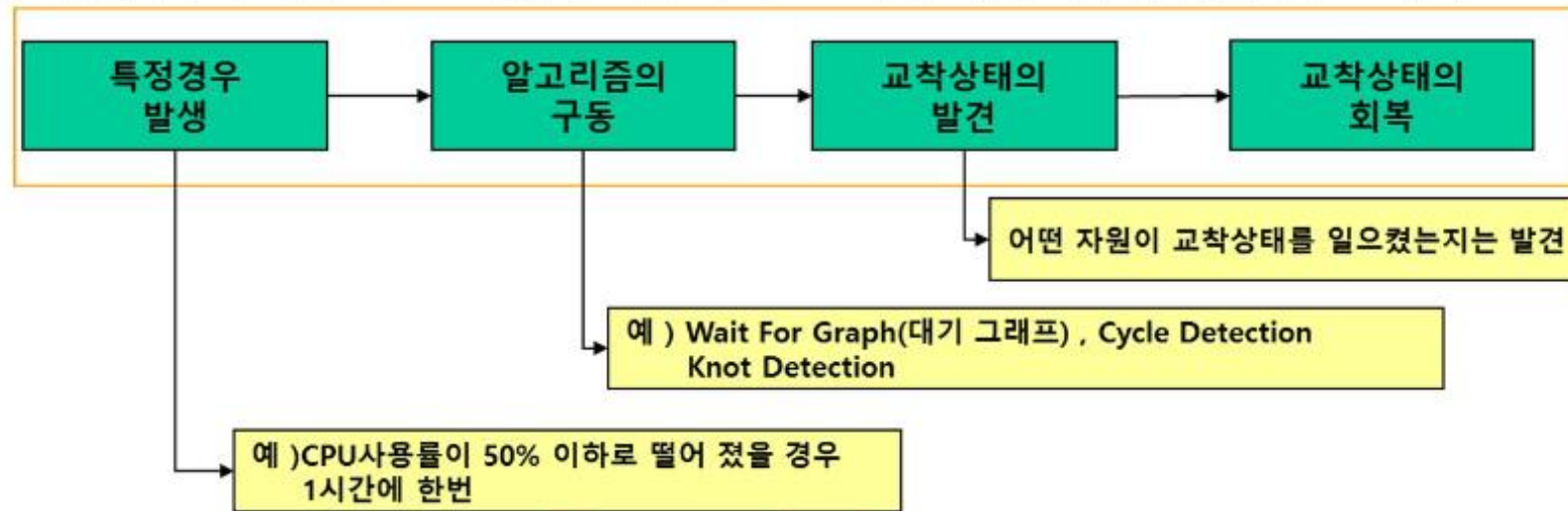
6. 자원할당 그래프



교착상태(Deadlock)

□ 교착상태의 발견(Detection)

- 시스템의 상태를 감시하는 알고리즘을 통하여 교착상태를 검사하는 알고리즘



□ 교착상태의 회복(Recovery)

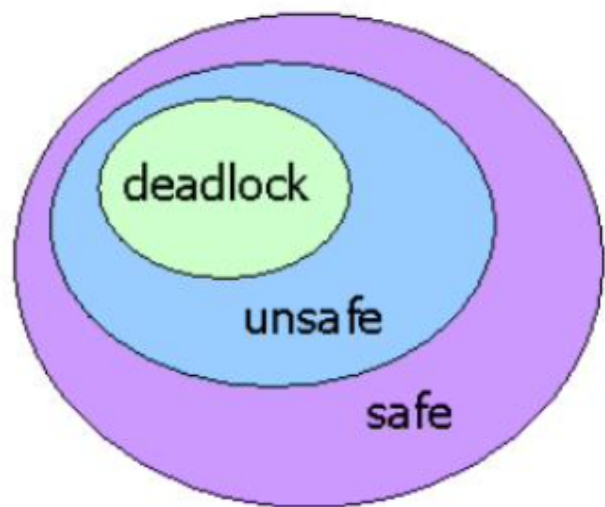
- 교착상태(Deadlock)가 없어질 때까지 프로세스를 순차적으로 제거(Kill)
- 프로세스 종료 비용 최소화 : 우선순위, 진행비용, 복귀비용 등
- 자원의 우선순위 할당 : 희생자 선택, 복귀, 기아상태

뱅크스 알고리즘

1. Banker's 알고리즘의 개요

□ Banker's 알고리즘의 정의

- 운영체제는 자원의 상태를 감시하고, 사용자 프로세스는 사전에 자신의 작업에서 필요한 자원의 수를 제시하는 교착상태 회피 알고리즘
- 운영체제는 안정상태를 유지할 수 있는 요구 만을 수락하고 불안전 상태를 초래할 사용자의 요구는 나중에 만족될 수 있을 때까지 계속 거절하는 교착상태 회피 알고리즘



- 안전상태: 모든 프로세스가 교착상태를 발생시키지 않고 종료할 수 있도록 하는 자원 할당 순서가 존재하는 경우
- 안전 상태 $\neq$ 교착 상태
- 불안전 상태 = 또는 $\neq$ 교착상태

뱅크스 알고리즘

2. Banker's 알고리즘의 구조

□ Banker's 알고리즘의 자료구조

자료구조	설명	수식
Available	- 사용 가능한 자원의 수	Available[j]=K 자원종류 Rj 중 K 개 사용가능
Max	- 프로세스 별 최대 자원의 요구	Max[i,j]=K Pi 는 자원종류 Rj 중 최대 K 개 요청
Allocation	- 현재 프로세스 별 할당되어 있는 자원 수	Allocation[i,j]=K Pi 는 현재 자원종류 Rj 를 K 개 할당받고 있음
Need	- 프로세스 별 남아있는 자원의 수	Need[i,j]=(Max-Allocation) Pi 는 자원종류 Rj 를 K 개 더 요구함

뱅크스 알고리즘

□ Banker's 알고리즘의 사례

[조건] 최대 요구자원 수에 대한 정보가 있을 경우를 가정

최대 요구량(Max)

프로세스	R1	R2
P1	2	2
P2	1	4
P3	2	1

현재 점유량 (Allocation)

프로세스	R1	R2
P1	1	1
P2	1	2
P3	1	1

리소스 총량

R1	R2
4	5

뱅크스 알고리즘

현재 사용 가능한 리소스의 양을 구함(Available)

리소스 총량

R1	R2
4	5

현재 점유량 (Allocation)

프로세스	R1	R2
P1	1	1
P2	1	2
P3	1	1
합산	3	4

현재 여유 자원 (Available)

R1	R2
1	1

추가 요구량을 구함(Request = Max - Allocation)

최대 요구량(Max)

프로세스	R1	R2
P1	2	2
P2	1	4
P3	2	1

추가 요구 자원 (Request)

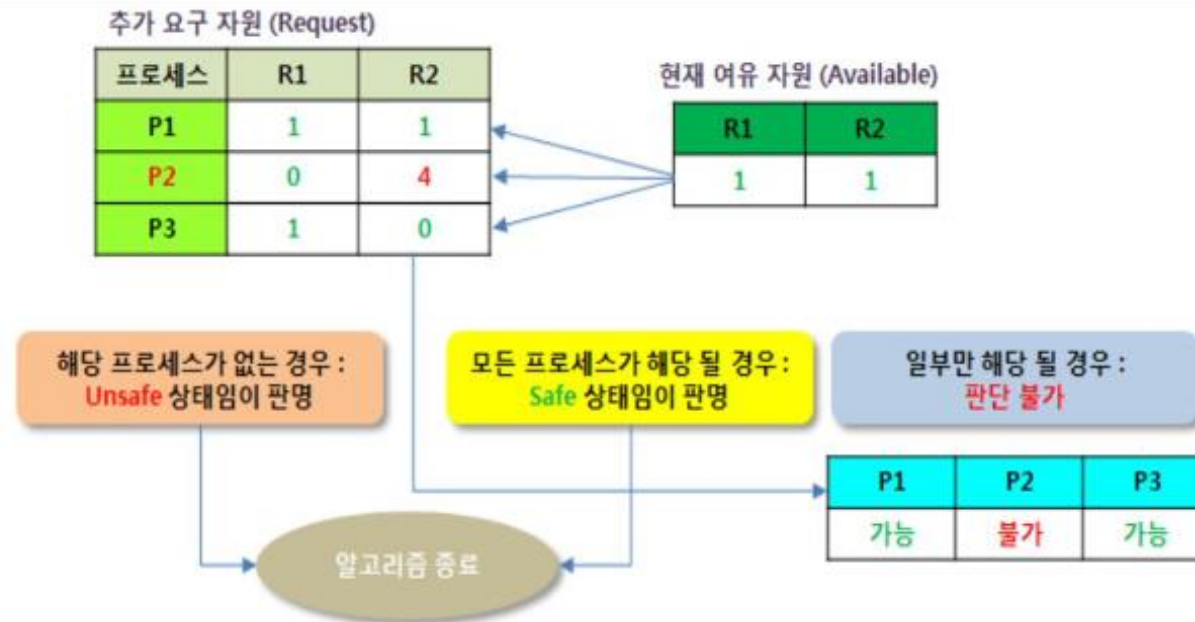
프로세스	R1	R2
P1	1	1
P2	0	2
P3	1	0

현재 점유량 (Allocation)

프로세스	R1	R2
P1	1	1
P2	1	2
P3	1	1

뱅크스 알고리즘

추가 요구자원이 현재 여유자원보다 적은 프로세스(i)를 찾음
수행을 끝내기 위한 충분한 자원을 확보한 프로세스 검사
(Request(i) $\leftarrow$ Available(i))



뱅크스 알고리즘

수행 가능한 프로세스가 점유한 자원을 여유 자원으로 바꿈
(끝내고 반환한 경우로 가정)한 후에 3 번부터 반복
($Available = Available + Allocation$)



지역성(Locality)

1. Cache Hit율을 극대화하기 위한 Locality의 개요

□ 지역성(Locality)의 정의

- 기억 장치 내의 정보를 균일하게 Access하는 것이 아닌, 어느 한 순간에 특정 부분을 집중적으로 참조하는 특성
- CPU가 주기억장치의 특정 부분(지역)에 위치한 프로그램 코드나 데이터를 빈번히 혹은 집중적으로 액세스하는 특성

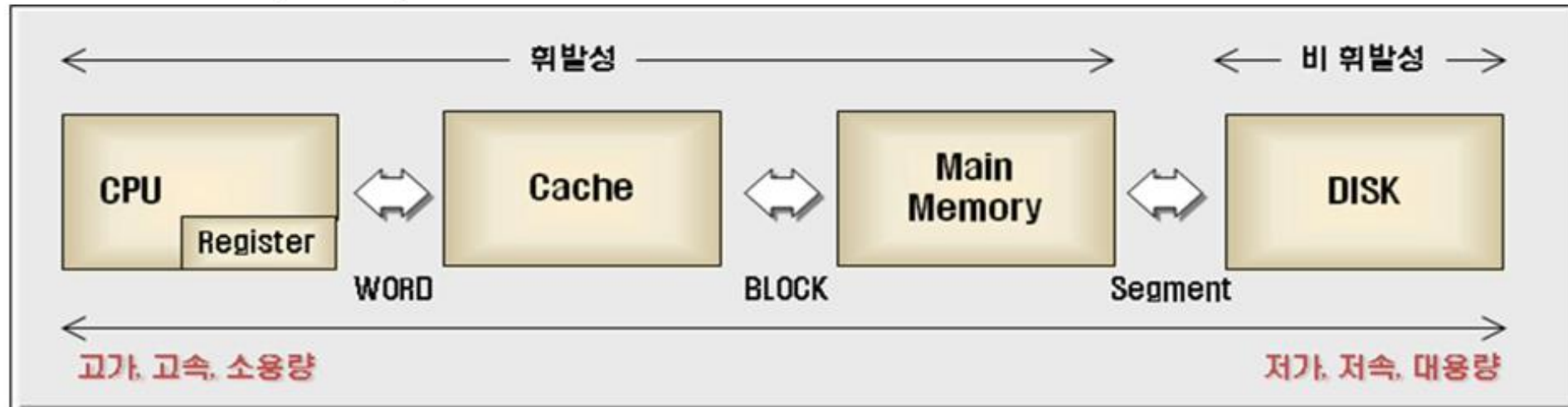
□ Locality의 전제조건

- 프로그램은 모든 코드나 데이터를 균등(시간적으로나, 공간적으로)하게 Access하지 않는다는 특성을 기본으로 함.

지역성(Locality)

□ Locality의 원리

- Cache Access Time의 최소화 + Cache Hit Rate의 최대화
- $\text{Cache Hit Ratio} = (\text{Hit 수}) / \text{주기억장치 접근의 총수}$



지역성(Locality)

2. Locality의 종류 및 관리방법

□ Locality의 종류

구분	내용	사례
시간적 지역성 (Temporal Locality)	- 최근에 참조된 기억장소가 가까운 장래에 계속 참조될 가능성이 높은 성질 - 최근 액세스된 프로그램 코드(명령어)나 데이터가 가까운 미래에 다시 액세스될 가능성이 높아지는 특성	- 순환(Looping) - 서브프로그램 - LRU(Least Recently Used)
공간적 지역성 (Spatial Locality)	- 최근에 참조된 기억장소와 가까운 기억정보가 가까운 장래에 계속 참조될 가능성이 높은 성질	- Array - Sequential Processing - Pre-fetch
순차적 지역성 (Sequential Locality)	- 분기(branch)가 없는 한 데이터가 기억장치에 저장된 순서대로 순차적으로 인출되고 실행될 가능성이 높음.	

※ LRU (Least Recently Used) : 사용되지 않고 가장 오래 있었던 블록교체

지역성(Locality)

□ Locality를 이용한 관리방법

구분	내용
기억장치 계층구조 (Hierarchy)	- CPU – Cache Memory, -Main Memory - Access time을 줄이고, 지역성을 이용한 Cache miss를 줄일 수 있음
Cache Access 시간	- Cache Hit ratio를 극대화 할 수 있음
Working Set	- 하나의 페이지가 자주 Access하는 페이지들의 집합을 말하고, Working set를 참조의 지역성을 바탕으로 메인 메모리에 배치하면 Page Fault도 줄이고, Thrashing도 예방가능

지역성(Locality)

3. Locality 원리의 활용 사례

□ Cache Memory

1) 재배치 알고리즘 구현에 이용

- LRU(Least Recently Used) : 최근에 가장 적게 사용된 Block을 교체 ← 시간적 지역성

2) Fetch 알고리즘 구현에 이용

- 선인출(Pre Fetch) : 필요한 정보와 예상 정보를 미리 인출하여 배치 ← 공간적 지역성

□ Virtual Memory

1) Thrashing 해결

- Working Set 기법: 지역성 원리를 이용한 Working Set로 인한 페이지 교환을 최소화
- 페이지부재 (Page fault)로 인한 페이지 교환을 최소화

2) 페이지 교체 (Replacement) 알고리즘 구현

스레싱(Thrashing)

1. 페이지 교체가 더 많은 시간을 소요하는 현상, Thrashing의 개요

□ Thrashing의 정의

- 기억장치의 페이지 부재가 비정상적으로 많이 발생하여 CPU가 프로그램 처리보다 페이지 교체에 더 많은 시간을 보내 성능이 급격히 줄어드는 현상
- 멀티 프로세싱 기능을 갖춘 시스템에서 가상 메모리의 페이지 부재(Page Fault)가 너무 많이 발생하여 프로세스 실행보다 페이지 교체에 더 많은 시간을 소모하는 현상

□ 원인

구분	내용
리소스 부족	- 낮은 CPU 사양 - 적은 메모리 사이즈 등
부적절한 페이지 교체 정책	- locality 및 페이지 빈도 등을 고려하지 않은 경우 - Memory Size가 너무 작은 경우
심한 다중 프로그래밍	- 프로세스 교환시 마다 페이지 부재처리 발생 - 수용 가능한 페이지 교체 수 초과 및 수용 가능한 프로세스 수 초과

스레싱(Thrashing)

□ 영향

- Thrashing 상황에서 프로세스는 실행보다 페이지 교체에 더 많은 시간을 허비
- System Performance 저하
- System이 비정상적으로 정지할 수 있음

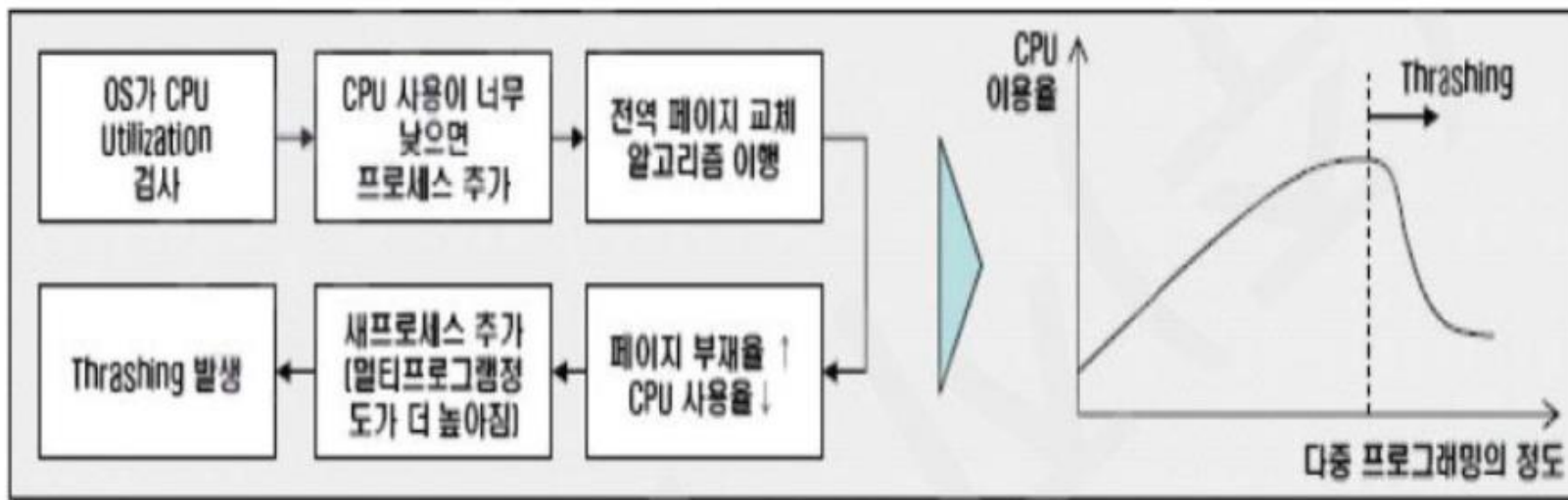
□ 발견방법

- Page Fault 조사: 각 프로세스 별 페이지 부재로 발생하는 PFF 검사
- Swapping 조사: 다중 프로그래밍시 다른 프로세스에 공간을 부여하기 위해 프로세스들을 주기억장치 에서 보조기억장치 또는 그 반대로 옮기는 과정 검사

스레싱(Thrashing)

2. 개념도 및 발생과정

□ 개념도



- 초기에는 멀티 프로그래밍 정도가 높아짐에 따라 CPU 이용률이 향상되지만, 어느 임계치보다 커지면 쓰레싱 발생

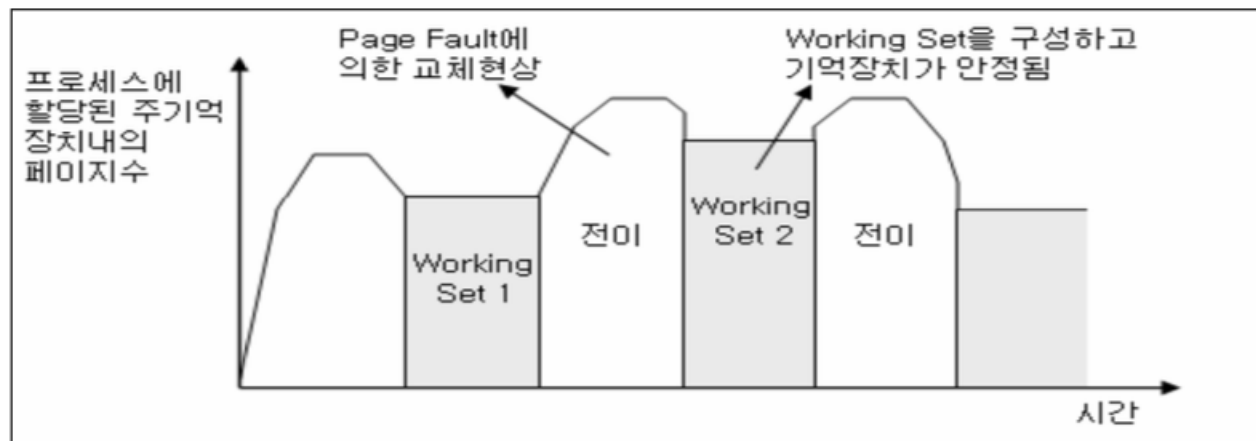
□ 발생과정

- CPU 사용 확인, 프로세스 추가, 전역 페이지 교체 알고리즘 이행, 페이지 부재율 상승/CPU 사용율 하락, 새로운 프로세스 추가

스레싱(Thrashing)

3. Working-Set Model

□ 개념



- 특정시간에 실행되는 프로그램에 Locality가 포함되는 페이지들의 집합
- 프로세스들은 주어진 시간 간격 동안 자신의 페이지들 중 일부를 더 자주 접근하려는 지역성(Locality)이 있으며, 이를 이용하면 페이지 부재율을 감소시킬 수 있음

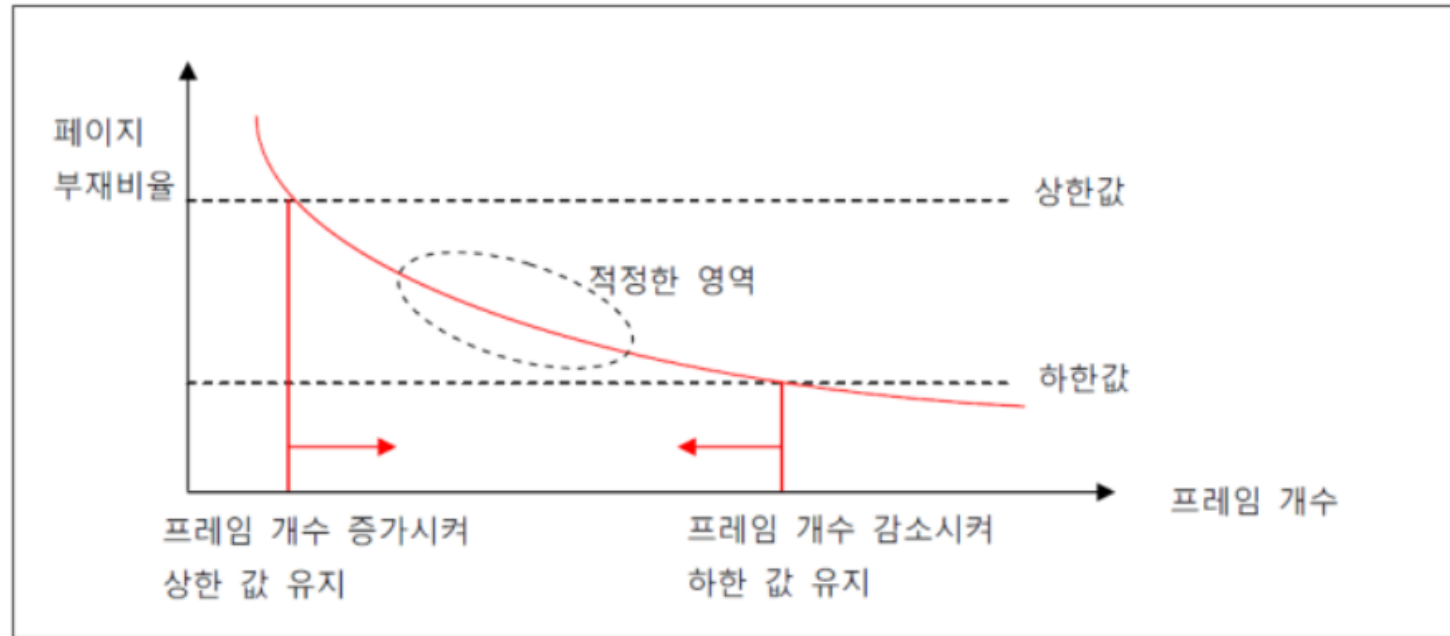
□ 특징

- 프로그램이 효과적으로 실행되기 위해서는 활동 프로세스의 Working Set이 메모리에 적재되어 있어야 함
- 매번 참조시마다 Resident Page Set 조정(PFF보다 overhead가 큼)

스레싱(Thrashing)

4. Page-Fault Frequency

□ 개념



- 현재 페이지 부재와 바로 전의 페이지 부재 사이의 시간을 관찰해 지금까지의 최소시간보다 크면 그 사이에 호출되지 않았던 페이지들을 모두 제거하고,
- 최적시간보다 작다면 들어오려는 페이지를 그 프로세스의 상주 페이지 집합에 첨가하는 방법
- 페이지 부재율 > 상한: 그 프로세스에게 Frame 추가 할당
- 페이지 부재율 < 하한: 그 프로세스로부터 Frame 회수

스레싱(Thrashing)

5. Thrashing 회피를 위한 고려요소 및 WSM, PFF 비교

□ Thrashing 의 회피를 위한 고려요소

- Pre-Paging: 과도한 페이지 부재를 방지하기 위해 관련된 모든 페이지를 사전에 한꺼번에 메모리에 로드
- Page Size: 페이지의 크기는 2의 거듭 제곱
- Inverted Page Table: 페이지 테이블을 위한 메모리 공간 줄이는 기법
- 프로그램 구조: 프로그램 개발 시 지역성 고려, 개발언어에 따른 특성 고려

□ Working Set Model 과 PFF 의 비교

구분	WSM	PFF
페이지집합 수정방식	매번 기억장치를 참조한 뒤 수정	페이지 폴트 발생시만 수정
쓰레싱 조절	Pre-paging 은 유용하나 조절 어려움 작업집합 추정이 어려움	직접적으로 쓰레싱을 방지하면서 PFF 측정 및 조절
오버헤드	매우 큼	적음

감사합니다!



컴퓨터구조 및 운영체제(CAOS)



5. 고가용성(High Availability)

5. 고가용성(High Availability)	139
(1) HA(High Availability).....	140
(2) FT(Fault Tolerant System).....	144
(3) RAID(Redundant Array of Inexpensive/Independent Disks).....	148

HA(High Availability)

1. 서비스 연속성 확보를 위한 HA의 개요

□ HA(High Availability) 의 정의

- 두 대 이상의 시스템을 하나의 클러스터로 묶어서 한 시스템의 장애시 클러스터 내의 다른 시스템이 신속하게 서비스를 Failover해 최소한의 서비스 중단을 이루는 메커니즘

□ HA의 필요성

- 서비스 다운타임을 최소화함으로써 가용성을 극대화
- 고가용성으로 기업의 비즈니스 연속성(Business Continuity) 확보
- 기업의 신뢰도 및 이미지 향상

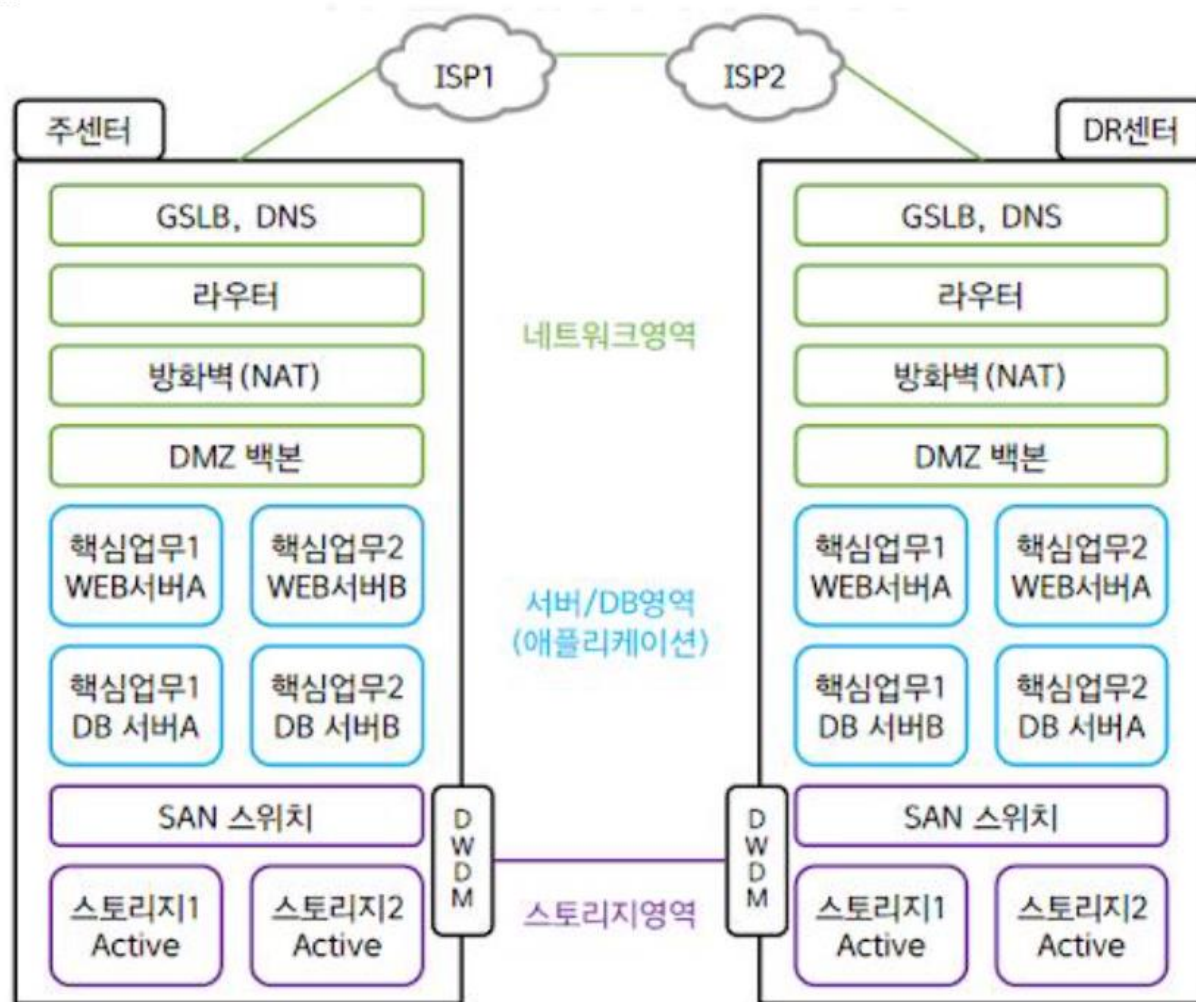
□ HA를 위한 요구사항

- RAS(Reliability, Availability, Serviceability)
- Dynamic Configuration and Upgrade)
- Disaster Recovery
- Single System Image
- 통합운영관리, 저렴한 소유 비용(TCO, Total Cost of Ownership)

HA(High Availability)

2. HA 구성도 및 구성유형

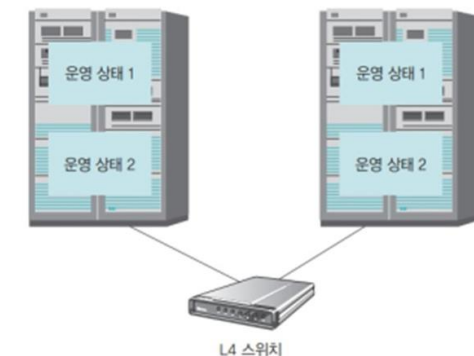
□ HA 구성도



HA(High Availability)

□ HA 구성유형

구 분	설 명
Hot Standby	- 가장 단순하면서 많이 사용되는 유형 - 가동 시스템과 평상시 대기 상태 또는 개발 시스템으로 운영되는 백업 시스템으로 구성 - 외장 디스크는 가동 시스템에서만 접근 가능하고, 장애 시에만 백업 시스템에서 접근 가능함
Mutual Takeover	- 2개 시스템이 각각의 고유한 가동 업무 서비스를 수행하다가 한 서버에 장애가 발생하면 상대 시스템의 자원을 Failover 하여 동시에 2개의 업무를 수행하는 방식 - 장애 발생 시 Failover 에 대비해 각 시스템 2개의 업무를 동시에 서비스할 수 있는 시스템 용량을 갖추도록 고려해야 함 - 외장 디스크는 해당 시스템에서만 접근 가능함
Concurrent Access	- 여러 개의 시스템이 동시에 업무를 나누어 병렬 처리하는 방식으로 HA 에 참여하는 시스템 전체가 Active 한 상태로 업무를 수행함 - 한 시스템에 장애가 발생하여도 다른 시스템으로 Failover 하지 않고 가용성을 보장함



HA(High Availability)

□ High Availability 동작 방식

장애 상황	HA 동작 방식
시스템 전체 장애 시	- 한 시스템으로부터 keep alive packet이 오지 않는 경우 백업 시스템은 상대 시스템이 Down 되었다고 판단하고 이미 정의된 자원(Volume Group, Filesystems, IP Address, Application)으로 Failover - Failover 순서 : 미리 정의된 Scripts에 의해 Network 자원 → Disk 자원 → Application 자원 순서로 진행 됨
Network Adapter 장애 시	- 가동 시스템 내의 Service Adapter 장애시 : 시스템에 Service adapter 와 Standby adapter를 설치 구성한 경우, Service adapter에 장애가 발생하면 Standby adapter가 Service adapter의 IP Address를 Failover - 가동 시스템 내의 전체 N/W Adapter 장애 시: 백업 시스템의 Standby Adapter로 가동 시스템의 Service IP Address가 Failover됨.
TCP/IP Network 장애 시	- Network 전체 장애가 발생하는 경우에는 특별한 동작을 하지 않음 - N/W 전체 장애 시에는 백업 시스템으로 Failover해도 동일한 N/W 문제가 존재함

HA(High Availability)

5. HA의 한계점 및 구축 시 고려사항

□ HA 한계점

- External Disk 자체 장애발생 시 HA Solution으로 해결 못함
- 장애 발생으로 시스템이 Down 되지 않는 경우 자동 Failover가 되지 않음
- 시스템성능이 저하되는 경우에 자동 감지가 불능
- DB 및 Application이 Down되는 경우에는 일반적으로 Failover하지 않음
- DB 및 Application 자체 Bug일 경우 Failover가 의미 없음
- HA구성에 따른 정보교환으로 시스템의 안정성, 보안성, 성능에 Overhead가 존재함

□ HA 구축 시 고려사항

- HA 구성 방식 및 대상 서버 결정
- 백업 서버 Capacity
- HA 대상 시스템들에 대한 OS 자원 및 사용자 자원 동기화
- 보호될 자원 결정 및 자원 동기화
- External Disk의 보호방안(2중화여부)

FT(Fault Tolerant System)

1. 결함 발생시 시스템 정지없이 연속적 기능을 보장하는 시스템, FTS의 개요

□ FTS(Fault Tolerant System)의 정의

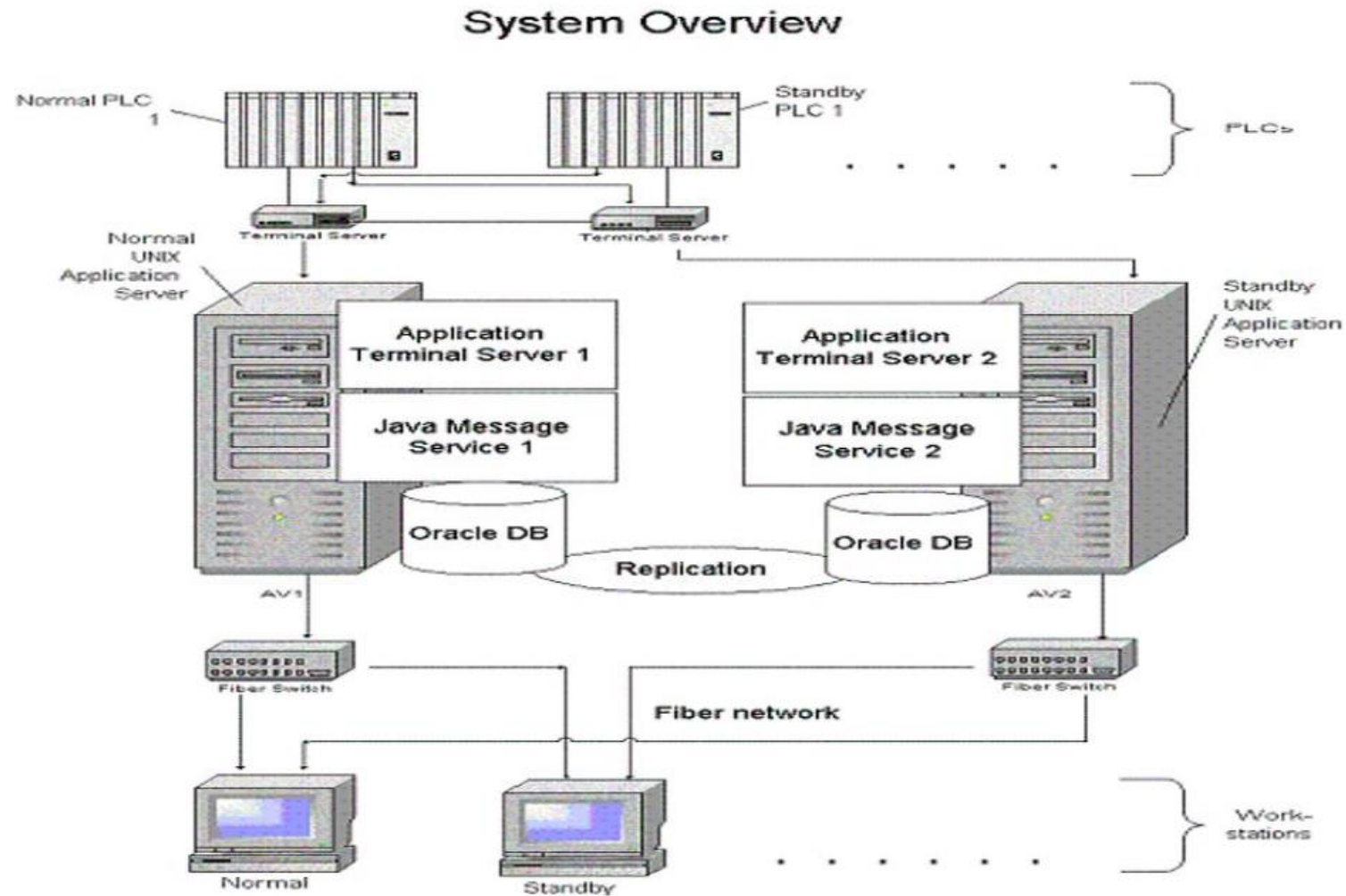
- 하드웨어 또는 소프트웨어의 결함, 오동작, 오류 등이 발생하더라도 설계 상에 명시된 기능을 지속적으로 수행할 수 있는 시스템
- 시스템의 가용성, 신뢰성, 안전성 보장

□ FTS의 필요성

- 데이터의 무결성과 일관성 유지
- 시스템 가동률 극대화, 데이터 안전 및 보존 유지
- 시스템 고장 등으로 인한 경제, 인명 손실을 막고 사회적 혼란에 대한 대처 방안 필요
- 트랜잭션 처리, 실시간 제어, 인간의 안전에 관련된 응용 분야에서 그 중요성이 증대

FT(Fault Tolerant System)

□ 결함허용시스템의 개념도



FT(Fault Tolerant System)

3. 가용성 저해 원인과 결함 해결 단계 및 주요 기능

□ 가용성 저해 원인

구분	내용	해결방안
응용 시스템	- 부적절한 Source code - 비 효율적 분기 logic - 잘못된 SQL - Resource 미 반환	- 표준 개발프로세스 정립 및 개발자 역량 강화 - 단위 테스트, 모듈 테스트
DBMS	- DB performance 튜닝 부족 - Dead Lock, Latch 경합	- 주기적인 모니터링 기반 DBMS - 최적화 - DB 성능 진단 및 튜닝
H/W	- 물리적 H/W resource 부족, Fault (firmware, patch) 유지보수 미흡	- 시스템 중요도에 따른 투자 검토
N/W	- 대량 트래픽에 의한 N/W Resource초과, - DDOS, 해킹 등의 보안 침해 - 물리적 Fault	- 보안 솔루션 - 계획적인 N/W자원 활용 - QoS 도입 - 이중화

FT(Fault Tolerant System)

□ 결함의 해결 단계

단계	내용
결함감지 (Fault Detection)	- 하드웨어로 구성된 비교기(Compare Logic)를 통하여 수행됨 - 시스템 내에서 Fault가 발생되면 해당 모듈 또는 시스템은 Fault 상태로 들어가게 되고, OS는 각 모든 하드웨어 모듈들의 상태를 분석하여 어느 모듈이 Fault를 유발시켰는가를 분석하여 알아냄
결함진단 (Fault Diagnosis)	- Fault가 일시적(Transient)인 것인지 영구(Hard)적인 것인지 판단하여 영구적인 경우 해당 모듈을 시스템 구성에서 제거
결함통제 (Fault Isolation)	- 결함으로 인한 오류 파급 차단
결합복구 (Fault Recovery)	- Fault를 유발한 모듈을 시스템에서 제거하여 시스템을 재구성

FT(Fault Tolerant System)

□ 결합허용시스템의 주요 기능

기능	상세내용
중복성	- 시스템의 구성요소 백업, 이중성으로 결함 발생시 대신 동작 - 복구 작업의 지원
결함검출	- 결함 복구를 위한 결함 발생 인식
결함위치확인	- 결함의 내용과 위치 검출
결함격리	- 결함 발생 부분의 격리
재구성	- 결함 발생시에도 정상적 작동 보장
보수	- 고장 부분을 정상 작동에 방해 없이 교체 및 보수
회복	- 교체 및 보수 후 시스템의 동작을 방해 없이 추가
데이터 보존	- 정보의 손상 및 유실 방지

FT(Fault Tolerant System)

4. 결함허용 기법

□ 하드웨어 측면

기법	상세내용
Checkpoint 기법	- 한 프로세스를 주 프로세서와 보조 프로세서에 중복 할당(중복성의 원리)
Hot standby 방식	- 두 개의 프로세서를 동기 상태에서 프로세스 수행
Triple modular Redundancy	- 3개 이상의 프로세서가 같은 입력에 대하여 동일한 연산 수행
RAID	- 디스크 미러링, 패리티 비트 분산 저장을 통해 결함 허용

□ 데이터베이스 측면

기법	상세내용
Rollback (Undo)	- 트랜잭션 작업 철회
Log File	- Commit 안된 작업을 로그 파일로 회복
Check Point	- 중간 검사시점을 두어 결함 발견시점 이후만 회복
Shadow Paging	- 트랜잭션 실행 동안 수행 페이지 테이블과 그림자 페이지 테이블을 지속 유지

FT(Fault Tolerant System)

□ 소프트웨어 측면

기법	상세내용
Check Pointing	- S/W 수행 중에 검사 시점을 설정하여 오류발생이 발견되면 발생 이전의 검사시점으로 되돌아가서 재 수행
Recovery Block	- 재 수행 (Rollback & Retry)에 근거 - 검사시점에서 오류가 발견되면 지정된 이전 검사점으로 되돌아가서 같은 기능을 가진 다른S/W모듈을 수행
Conversation	- 재 수행(Rollback & Retry)에 근거한 Recovery Block의 확장형 - 복수의 프로세서 정보를 교환하는 프로세서들 간에 적용 가능한 기법
Distributed Recovery Block	- Recovery Block 기법을 분산환경으로 확장 - H/W 결함과 S/W 결함을 동일한 방법으로 대처
N Self-checking Programming	- 두 개 이상의 Self-checking 컴포넌트가 수행되면서 하나는 주어진 기능을 수행하고 다른 컴포넌트는 대기상태
N version Programming	- H/W 결함 허용 기법의 Triple Modular Redundancy와 유사 - N개의 독립적인 S/W 모듈의 수행결과를 비교하여 다수의 수행결과를 채택

FT(Fault Tolerant System)

□ 데이터 측면

기법	내용
Parity Code	- 2진 정보 내의 "1"의 개수를 홀수 또는 짝수로 규정하여 결함 감지
M of N Code	- N bit의 정보 길이에서 "1"의 개수가 반드시 M 개가 되게 하여 결함 감지
Checksum	- 한 블록의 정보 내용 합을 추가
Berger Code	- 한 정보의 "1"의 개수를 2진 정보로 추가
Hamming Error Correcting Code	- 결함 감지 및 단일 결함의 교정까지 수행하는 코드 기법

FT(Fault Tolerant System)

5. FTS 설계 고려사항과 활용분야

□ 결함허용시스템 설계 고려사항

- 처리방식에 따라 하드웨어 구조와 적용할 기법이 달라짐
- 결함허용 정도에 따라 적용 기법과 소요 자원이 달라짐
(일반 응용분야는 단순 결함감지만으로 가능, 고신뢰도 분야는 신속한 결함값 및 복구까지 필요)
- 분야별 결함허용 기법을 조화롭게 활용하여 FTS 구성함

□ FTS 활용분야

구분	내용
고 신뢰도	- 핵반응 제어, 우주비행 제어 등
장기 안정성	- 무인 우주항공 제어 등
높은 가용성	- 전화교환망, 항공예약시스템 등

- 하드웨어 기법 중심, Mirrored Memory, Mirrored Disk, On-line Repairing 기법 사용

FT(Fault Tolerant System)

□ High Availability와 Fault Tolerant System과 비교

구분	FTS	HA
Failover Time	0초	30 ~ 300초
동시성 유지보수	필요	불필요
시스템 비용	10 ~ 20배	2배 이상
어플리케이션	제한적	대부분 범용 제품
운영체제	전용 OS	범용 OS
하드웨어	전용 하드웨어	범용 하드웨어

RAID

1. 시스템의 고가용성과 성능 향상을 위한 솔루션의 핵심, RAID 개요

□ RAID(Redundant Array of Inexpensive/Independent Disks)의 정의

- 여러 개의 디스크에 중복된 데이터 저장과 데이터의 동시 저장을 통한 성능향상과 안정성 향상을 위한 솔루션

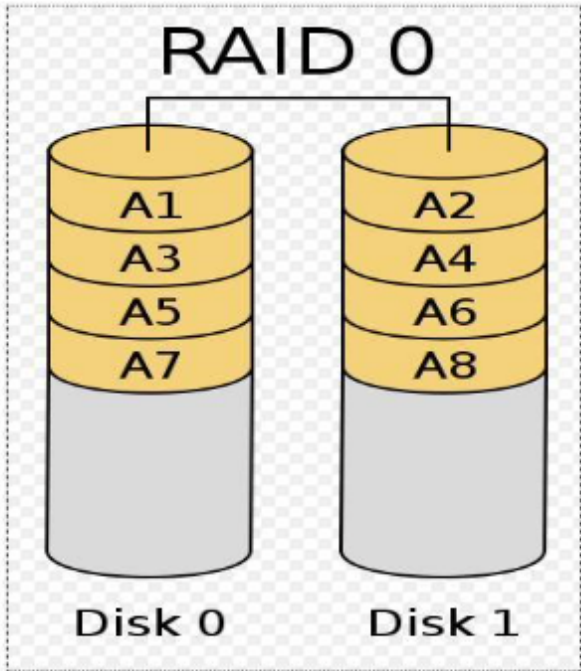
□ RAID의 등장배경

- 제한된 용량의 디스크로 대용량의 데이터 저장공간을 만들기 위함
- 디스크의 안정성의 향상을 통한 고가용성 확보 요구
- 시스템의 병목구간인 디스크에 대한 성능 개선 요구

RAID

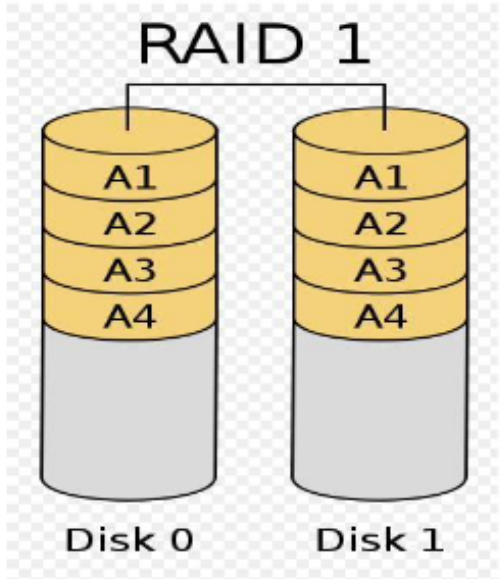
2. RAID의 종류 및 기술 특징

□ RAID 0의 구성도 및 특징

구성도	항목	내용
 RAID 0 Disk 0: A1, A3, A5, A7 Disk 1: A2, A4, A6, A8	특징	- Striping - 중복이 없이 데이터를 여러 개의 디스크에 분할해서 저장 - 패러티나 미러링이 없으므로 디스크의 안정성 향상 없음
	최소 수량	2개 (n : disk 수량)
	공간효율	1
	읽기 향상	n배 향상
	쓰기 향상	n배 향상
	고장 허용	Zero

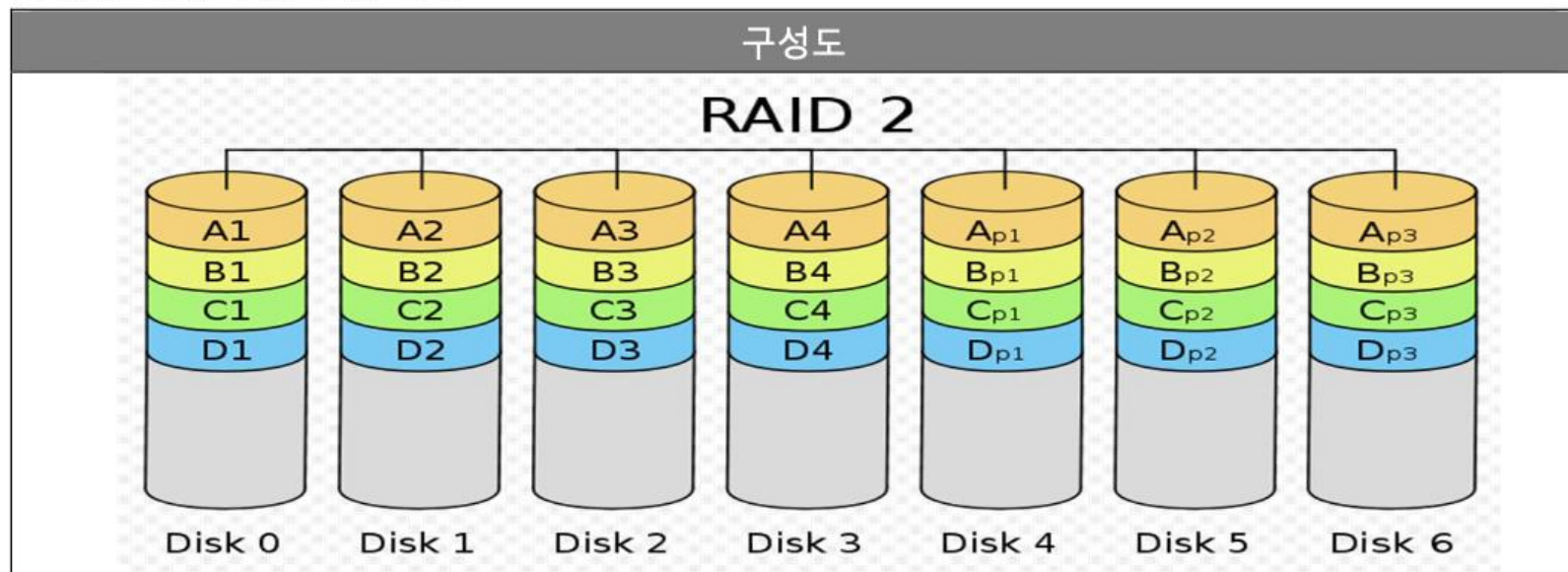
RAID

□ RAID 1의 구성도 및 특징

구성도	항목	내용
 The diagram illustrates RAID 1 (Mirroring) with two disks, Disk 0 and Disk 1. Each disk contains four data blocks labeled A1, A2, A3, and A4. The data is mirrored across both disks, meaning each block is stored on both Disk 0 and Disk 1. The title 'RAID 1' is centered above the disks.	특징	- Mirroring - 디스크 중복 쓰기를 통한 안정성의 2배 향상 - Parity와 Striping이 없고, 디스크의 공간효율 2배 하락
	최소 수량	2개 (n : disk 수량)
	공간 효율	1/n
	읽기 향상	n배
	쓰기 향상	1배
	고장 허용	n-1 disk

RAID

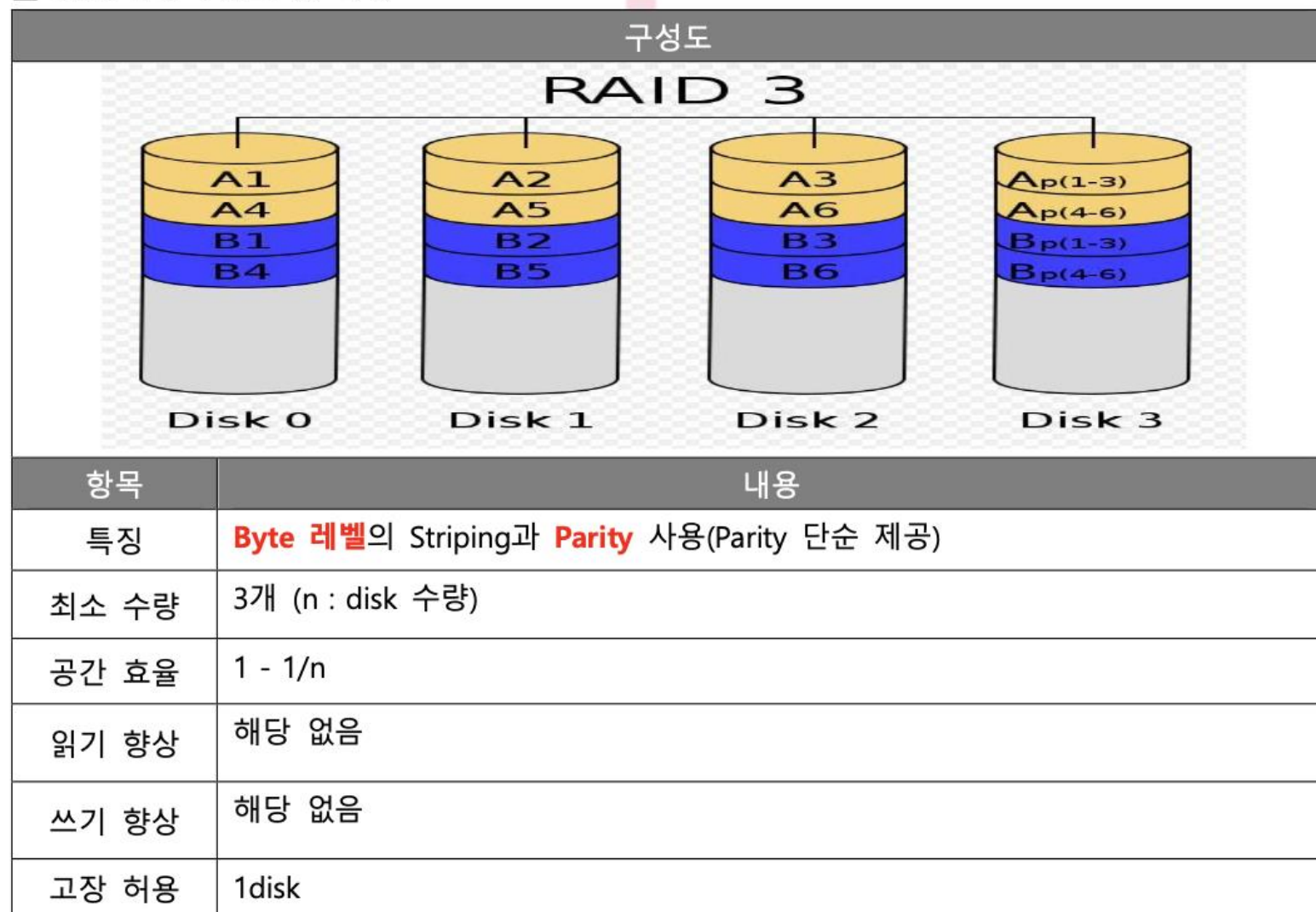
□ RAID 2의 구성도 및 특징



항목	내용
특징	자료를 비트 별로 각 디스크에 순서적으로 저장하는 방식 Hammring-code Parity 사용(Parity 단순 제공)
최소 수량	3개 (n : disk 수량)
공간 효율	$1 - 1/n \cdot \log_2(n-1)$
읽기 향상	해당 없음
쓰기 향상	해당 없음
고장 허용	1disk (단 오류디스크가 복구기록 코드 안에 있을 때)

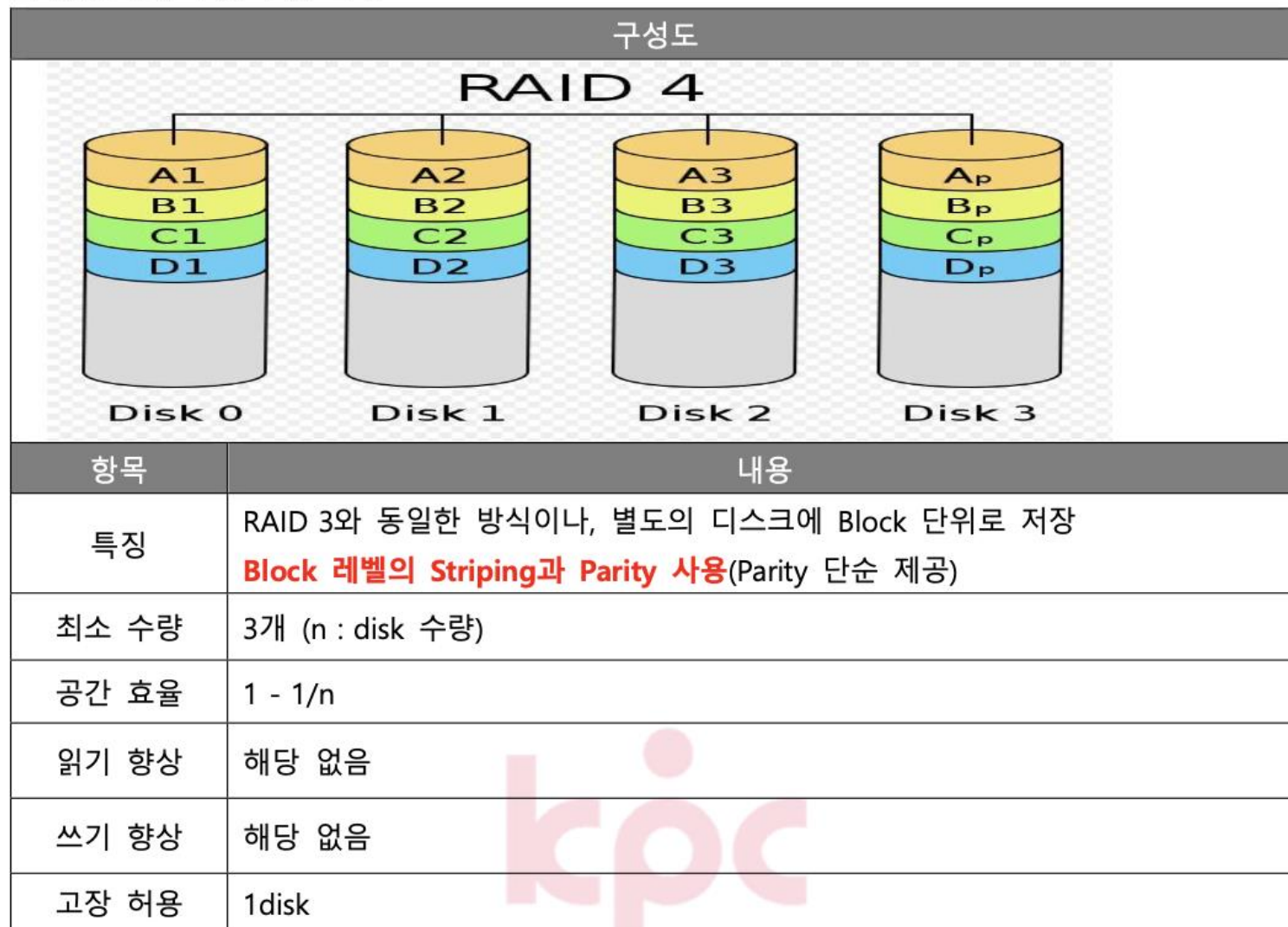
RAID

□ RAID 3의 구성도 및 특징



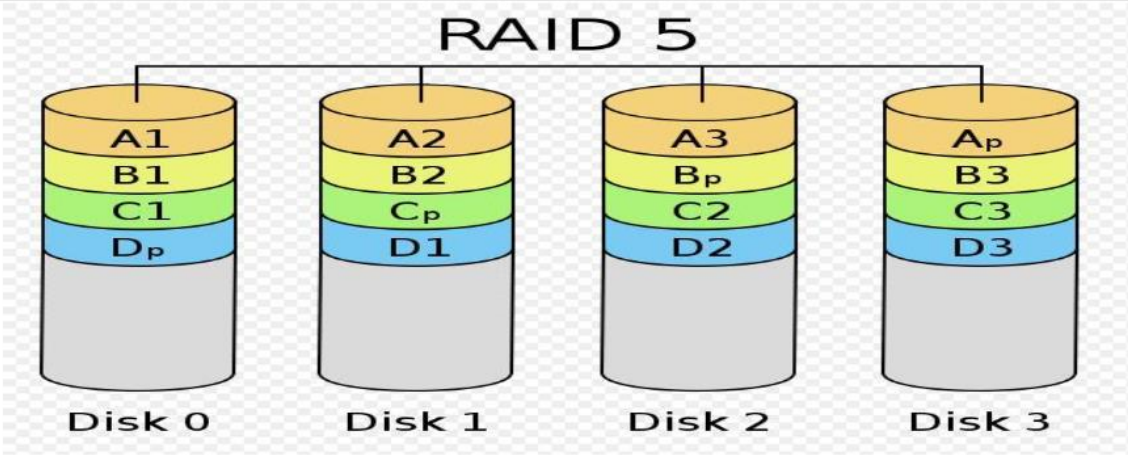
RAID

□ RAID 4의 구성도 및 특징



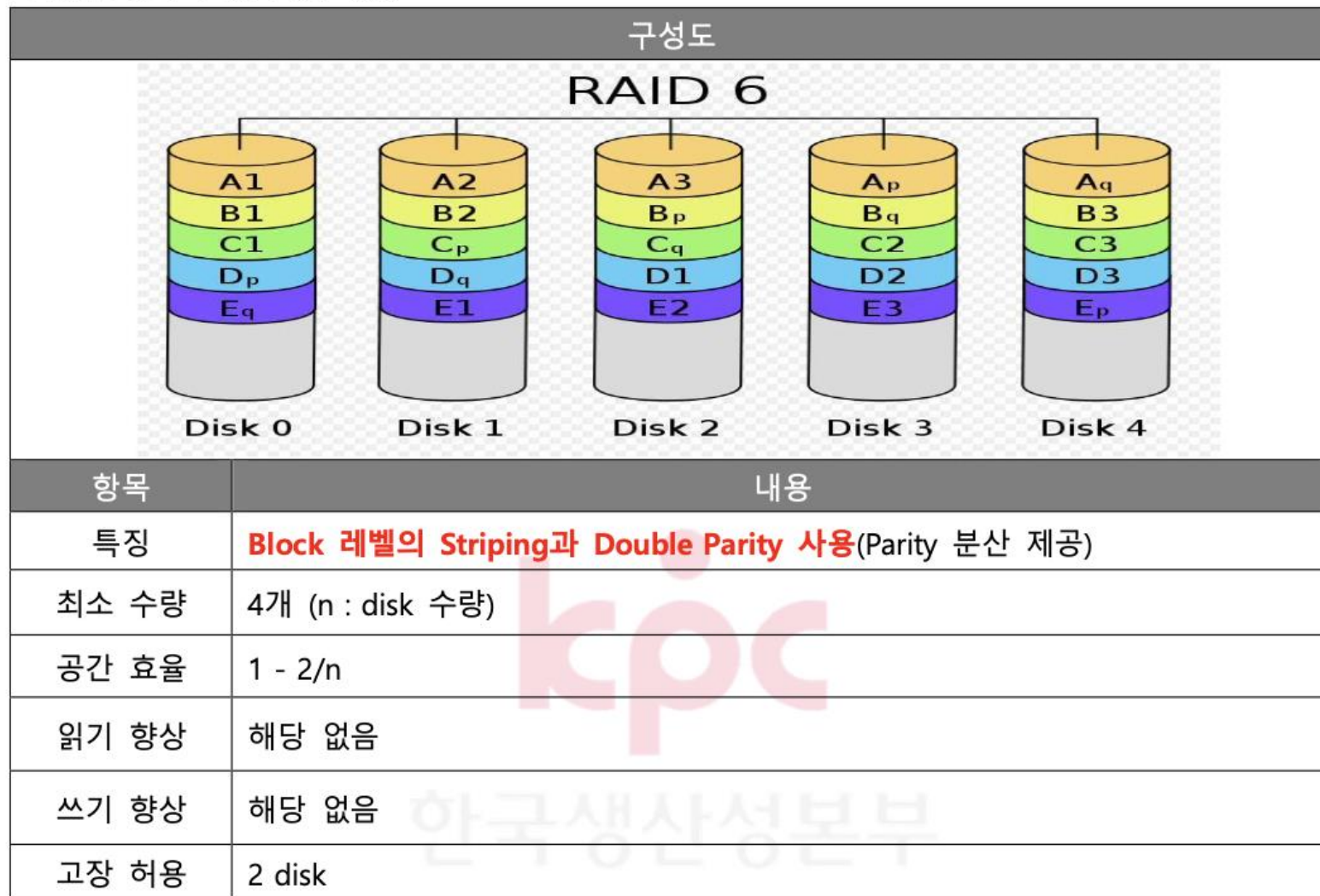
RAID

□ RAID 5의 구성도 및 특징

구성도	
RAID 5  Disk 0 Disk 1 Disk 2 Disk 3	
항목	내용
특징	데이터와 Parity를 분산하여 각 디스크에 저장 RAID 4의 병목현상 해결 Block 레벨의 Striping과 Parity 사용(Parity 분산 제공)
최소 수량	3개 (n : disk 수량)
공간 효율	$1 - 1/n$
읽기 향상	n-1 배
쓰기 향상	Variable (상황에 따라 다름)
고장 허용	1disk

RAID

□ RAID 6의 구성도 및 특징



RAID

3. Hybrid RAID 기술 특징 및 RAID 구성방법 비교

□ Hybrid RAID 기술 특징

항 목	RAID 0+1	RAID 1+0	RAID 5+1
개요	- 디스크의 Striping후 Mirroring을 실행함	- 디스크의 Mirroring후 Striping을 실행	- Parity를 이용하여 구성된 디스크를 Striping
특징	- Striping이 실행된 그룹의 디스크 2개가 문제 발생시 데이터의 보장이 안됨(최소 4개)	- 0+1의 Striping이 실행된 그룹의 2개 디스크의 고장에 대한 개선이 이루어진 모델(최소 4개)	- 최소 6개의 디스크가 사용되어 많은 디스크 소모

RAID

□ RAID 구성방법 비교

항 목	Software 기반 구성	Hardware 기반 구성	Firmware/driver 기반 구성
개요	- 대부분 OS가 제공하는 기능을 이용하여 구성하며 논리적 또는 물리적으로 구성이 가능	- 일반적으로 BIOS의 기능을 이용하여 구성	- RAID컨트롤러가 존재하는 외장 장치 기반으로 구성
장점	- 비용이 저렴	- SW 기반 구성에 비하여 성능이 뛰어남 - 부하가 적음	- 가장 성능이 뛰어나고 모든 RAID를 완벽하게 지원
단점	- CPU에 부하가 발생되고 성능이 떨어짐.	- 모든 RAID를 완벽하게 구현하지 못함. - BIOS에 종속 됨	- 비용이 가장 많이 소요됨

감사합니다!



컴퓨터구조 및 운영체제(CAOS)



6. 임베디드 시스템

6. 임베디드 시스템.....	154
(1) 임베디드 시스템(Embedded System)	155
(2) 임베디드 운영체제(Embedded OS).....	158
(3) 실시간 운영체제(Real Time Operating System).....	161
(4) 우선순위 역전(Priority Inversion).....	166
(5) 워치독 타이머(Watchdog Timer).....	170

우선순위 역전

1. 실시간 스케줄링의 문제점 Priority Inversion(우선순위 역전)의 개요

□ Priority Inversion의 정의

- 우선 순위가 높은 태스크가 READY상태(실행 가능)로 바뀌었지만 더 낮은 우선순위의 태스크가 CPU를 점유하고 있어서 실행되지 못하는 상태
- 높은 우선순위의 프로세스가 낮은 우선순위의 프로세스로 인하여 수행이 블록(block)되어 있는 상태

□ Priority Inversion의 문제점

- RTOS 환경에서 중요한 태스크의 수행이 요구된 시간에 끝내지 못함
- 중요한 태스크의 무한정 대기로 인한 전체 시스템의 이상현상 발생 가능함.

우선순위 역전

□ Priority Inversion의 원인

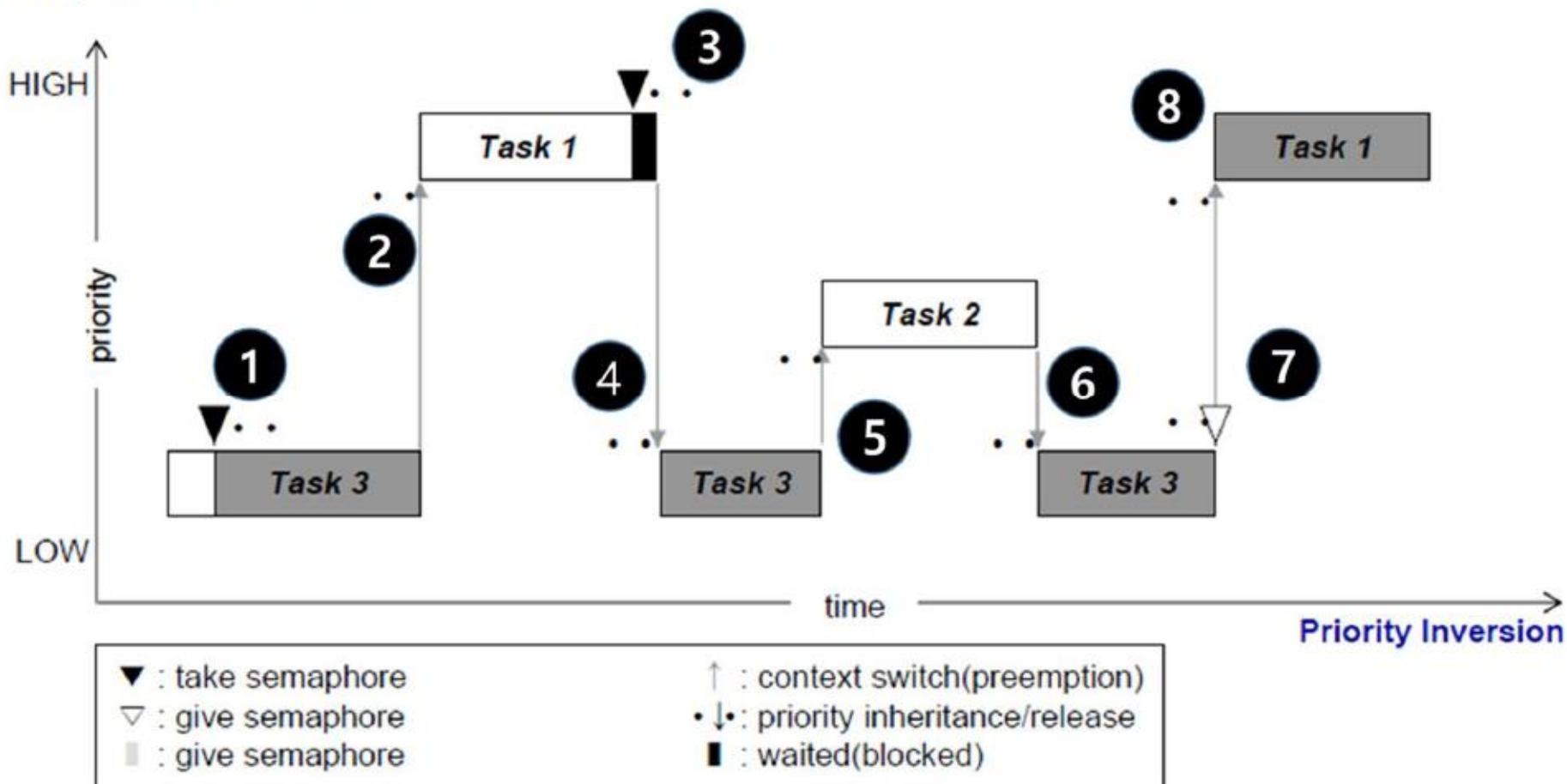
- 스케줄링과 동기화 사이의 상호작용 결과로 발생
- 스케줄링 규칙에서 실행 되어야 하는 스레드와 동기화에서 실행되어야 하는 스레드가 서로 다른 경우, 결과적으로 두 스레드의 우선순위가 역전되어 나타남

- 1) 비선점 스케줄링에서 낮은 우선순위의 태스크가 자원을 점유
- 2) 실시간 운영체제에서 상호배제를 위한 세마포어를 이용
- 3) 공유 자원의 장기 소유
- 4) 자원 점유 후 릴리즈시 낮은 우선순위의 태스크가 자원을 점유하는 경우

우선순위 역전

2. 사례 기반 Priority Inversion 설명

□ Priority Inversion Flow



- task1, 2, 3이 각각 high, medium, low priority를 가진다고 가정 했을 때 Priority Inversion 발생

우선순위 역전

□ Priority Inversion 발생과정

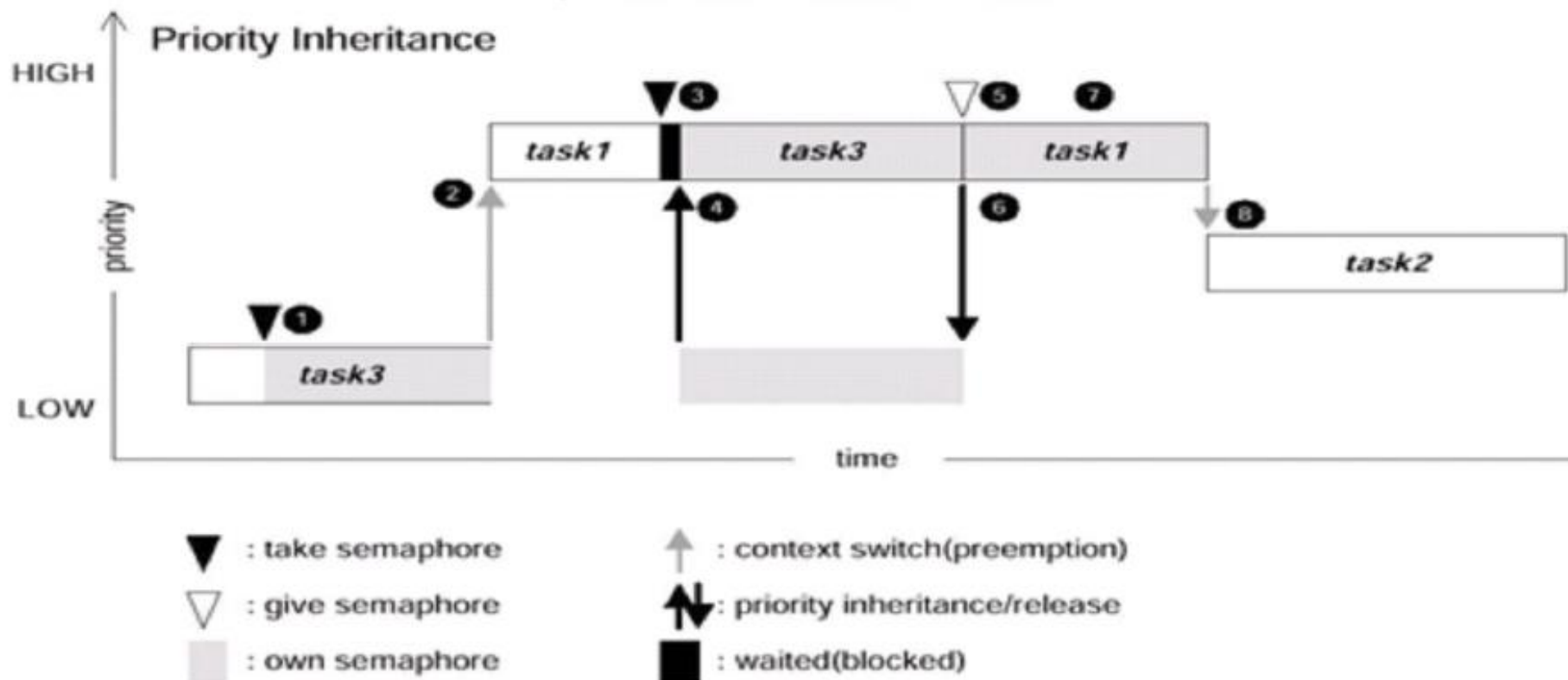
순서	우선순위역전 발생 상태
1	- task3이 공유자원을 액세스 위해 binary semaphore를 가지고 수행
2	- Scheduler에 의해 task1이 수행
3	- task1은 task3가 먼저 획득한 Semaphore를 얻으려(take)하고, task3가 그 Semaphore를 반환할 때(give)까지 Waiting 상태로 됨
4	- Scheduler에 의해 task3 수행
5	- Scheduler에 의해 task2가 수행 - 이 시점에서 task1의 priority가 task2보다 높음에도 불구하고 task2가 먼저 수행 되는 Priority Inversion 문제가 발생
6	- task2의 수행이 종료 되면, task3가 재 수행
7	- task3가 Semaphore 반환
8	- task1 수행

- task1이 가장 높은 우선순위를 가졌으나 결국 가장 나중에 수행

우선순위 역전

3. 우선순위 상속(Priority Inheritance) 이용한 Priority Inversion 해결 방법

□ 우선순위 상속(Priority Inheritance) 처리 개념도



- 특정 스레드가 더 낮은 우선순위의 스레드가 소유하고 있는 뮤텍스에서 대기 중인 경우, 소유자의 우선순위가 대기자의 우선순위로 증가 시켜 해결

우선순위 역전

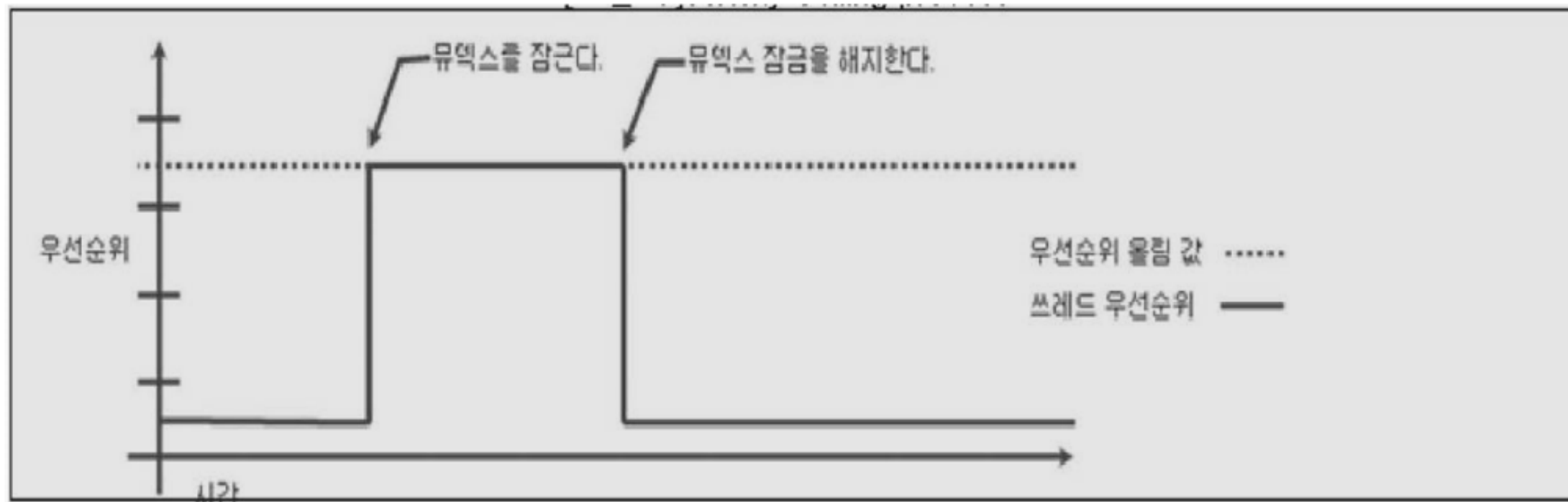
□ 우선순위 상속(Priority Inheritance)에 의한 처리 프로세스

순서	Priority Inheritance 처리 방법
1	- task3이 공유자원에 액세스하기 위해 binary semaphore를 가지고 수행
2	- Scheduler에 의해 task1이 수행
3	- task1은 task3가 먼저 획득한 Semaphore를 얻으려(take)하고, task3가 그 Semaphore를 반환할 때(give), Waiting 상태로 됨
4	- Scheduler에 의해 task3 수행되는데, task3의 priority를 task1이 waiting 상태인 동안, task1의 레벨로 높임(Priority Inheritance)으로써 task3의 priority가 task2보다 높기 때문에 선점(preemption) 없이 수행
5	- Task3이 Semaphore를 반환
6	- Task1이 Semaphore를 얻어서 수행됨과 동시에, task3의 priority가 다시 예전의 값으로 복귀
7	- Task1 수행
8	- Task2 수행

우선순위 역전

4. 우선순위 올림(Priority Ceiling) 이용한 Priority Inversion 해결 방법

□ 우선순위 올림(Priority Ceiling)의 개념도



- 어떤 쓰레드가 무텍스를 소유하고 있는 동안은 지정된 우선순위에서 동작

우선순위 역전

□ 우선순위 올림(Priority Ceiling)의 우선순위역전 해결 원리

- 어떤 쓰레드가 priority ceiling 속성을 가지는 뮤텁스를 Lock 설정 하면, 쓰레드의 우선순위는 뮤텁스의 우선순위 올림 값으로 자동으로 상승
- 단, 쓰레드의 우선순위가 뮤텁스의 우선순위 올림 값보다 작은 경우에만 우선순위 변경됨
- 올림 값 보다 더 높은 우선순위 가지는 쓰레드가 잠글 경우 규칙 깨짐
- 따라서 뮤텁스를 이미 잠근 쓰레드는 뮤텁스를 잠그려고 시도하는 다른 쓰레드들에 의해 선점되지 않고 자신의 작업을 종료 가능

우선순위 역전

□ 우선 순위 역전 해결 방안 비교

항목	Priority Inheritance	Priority Celling
특징	- 낮은 우선순위를 높여줌	- 우선순위 한정
장점	- 자동으로 우선순위 동적 변경	- 우선순위 변동에 대한 부하 적음
단점	- 세마포어 이용 자료공유로 완벽한 해결방법이 못됨 - 우선순위 변경에 따른 오버헤드	- 우선순위 한정 범위 설정 어려움

워치독 타이머(Watchdog Timer)

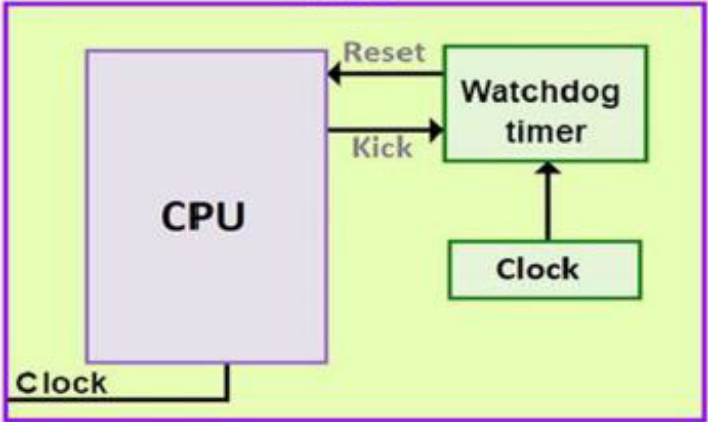
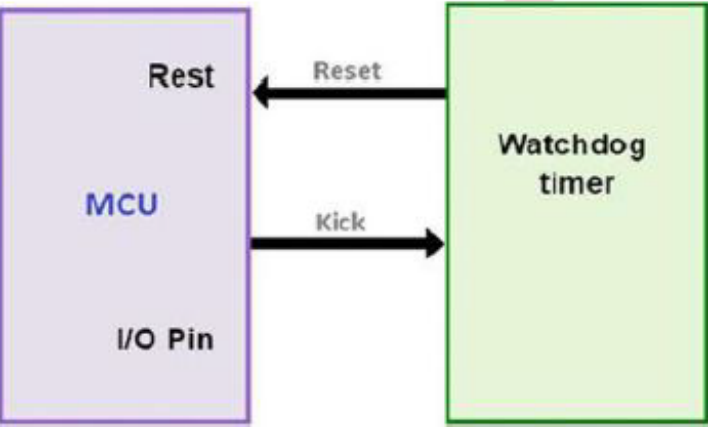
1. 워치독 타이머(Watchdog Timer)의 개요

□ 워치독 타이머(Watchdog Timer)의 정의

- 시스템이 중단되거나 소프트웨어 오류로 무한루프에 빠지는 오작동 상태를 감시하고, 시스템을 리셋하여 복구하기 위한 장치 또는 기능
- 정상 작동 중의 시스템은 시간이 경과하거나 "타임아웃"이 되는 것을 막기 위해, 정기적으로 워치독 타이머를 초기화함

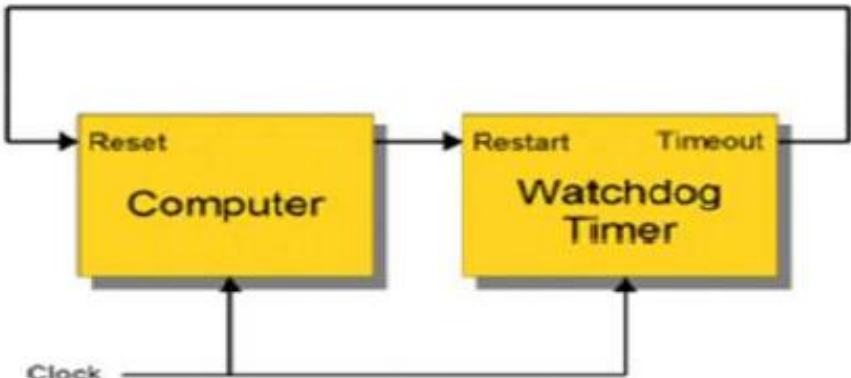
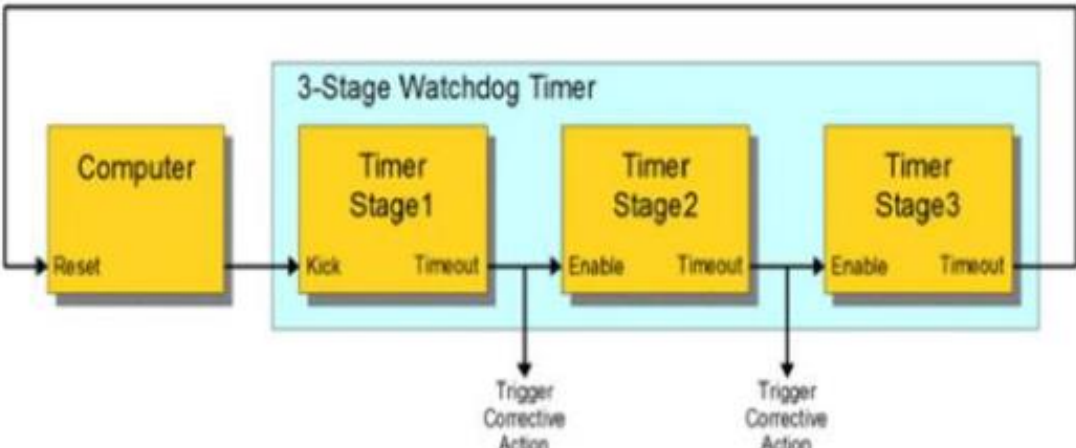
워치독 타이머(Watchdog Timer)

□ 워치독 타이머의 유형

유형	개념도	설명
내부 워치독 타이머		- 대부분의 마이크로 컨트롤러에는 내부 WDT가 내장됨 - 별도의 비용이 들지 않음 - 상대적으로 신뢰성이 낮음(SW적인 문제로 동작 오류 발생 가능)
외부 워치독 타이머		- 마이크로 컨트롤러 외부에 별도의 하드웨어 필요 - 비용 증가 - 프로세서에 대한 물리적 리셋핀(reset pin)이 포함 - 상대적으로 신뢰성이 높음

워치독 타이머(Watchdog Timer)

□ 워치독 타이머 하드웨어 설계 방식

구분	하드웨어 설계방식	설명
단단계 워치독 (Single-stage)		- 시스템 리셋 목적으로만 사용 - MCU에 통합된 형태나 CPU에 직접 연결된 칩 또는 외부 확장카드 형태로 구성
다단계 워치독 (Multi-Stage)		- 둘 이상의 타이머가 단계적으로 구성 - 차례로 시정 조치 작동, 마지막 단계에서 리셋

감사합니다!